Bike System

Carrera	**TECNICATURA SUPERIOR en DESARROLLO DE SOFTWARE**		
Ciclo Lectivo	**2026**	**Año**	3º
Unidad Curricular	Prácticas Profesionalizantes II	**Docente**	Kappes Nicolas
Nombre y apellido	German Domingues- Exequiel Rios	**Fecha**	13-10-2026
Calificación			

Instituto Tecnológico El Molino (ITEC)

Desarrollo de Software

Prácticas Profesionalizantes II

Bike System

Domingues German - Rios Exequiel

Kappes Nicolas

Ciclo lectivo 2026

Fecha de entrega: 13-10-2026

EL MOLINO
INSTITUTO TECNOLÓGICO

Índice

Introducción **4**
Fundamentos del proyecto **4**
Alcance y Límites **5**
Objetivo general y Objetivos Específicos **6**
Objetivo General 6
Objetivos Específicos 7
Relevamiento y análisis de Requerimientos **7**
Requerimientos Funcionales 7
Requerimientos No Funcionales 9
Diagramas UML **11**
Figura 1: Diagrama de Contexto 11
Figura 2: Diagramas de casos de usos. 12
Descripción de los casos de usos. **14**
Diagrama entidad-relación(E/R). **61**
Figura 3: diagrama entidad relación. 61
Diagrama de clases. **62**
Figura 4: diagrama de clases. 62
Relaciones y Multiplicidades del Diagrama de Clases. 62
Stack Tecnológico **63**
Modelo lógico de la base de datos. **64**
Modelo fisco de la base de datos(DDL). **81**
Arquitectura del sistema. **92**
Diseño y prototipo **93**
Interfaces 93
Flujo de navegación entre pantallas 94
Figura 5: flujo de navegación entre pantallas 94
Figura 6: pantalla de inicio de sesión 94
Figura 7: Ventana principal de operaciones 95
Figura 8: Directorio y gestor de clientes 96
Figura 9: Formulario de alta y edición de clientes. 97
Figura 10: Bicicletas asociadas a su cliente. 98
Figura 11: Historial de servicios de la bicicleta 98
Figura 12: Punto de venta. 99
Figura 13: Historial de ventas y comprobantes emitidos. 100
Figura 14: Gestión de garantías y services. 101
Figura 15: Tablero Kanban de reparaciones en taller. 102
Figura 16: Detalle y asignación de repuestos. 103
Figura 17: Gestión de inventario y catálogo de productos. 104
Figura 18: Formulario de ajuste manual de inventario. 104
Figura 19: Gestión de Proveedores y Pagos Comerciales. 105
Figura 20: Panel de reportes financieros. 106

Figura 21: Panel de creación de usuarios. 106
Figura 22: ventana modal para crear usuarios. 107
Figura 23: Bitácora de auditoría y respaldo de base de datos. 108
Estructura del Proyecto 108
Figura 24: estructura de carpetas. 111
Diagrama de arquitectura de flujo del sistema. 112
Figura 25: flujo del sistema 112
Descripción técnica de Funcionalidades. 112
Módulo: Login / Autenticación 113
Implementación en Capa Frontend (React) 113
Módulo: Home / Dashboard de Operaciones 115
Implementación en Capa Frontend (React) 115
Consulta SQL Agregada de Métricas de Dashboard 116
Módulo: Gestión de Clientes 116
Implementación en Capa Frontend (React) 117
Consulta SQL Optimizada con Trigramas 117
Módulo: Gestión de Parque Rodado y Bicicletas 118
Consulta SQL de Historial Clínico de Servicios 119
Módulo: Punto de Venta (Ventas y Presupuestos) 119
Implementación en Capa Frontend (React) 120
Código Backend Representativo (Transacción ACID y Prevención de Deadlocks) 121
Módulo: Gestión de Garantías y Primer Service 123
Lógica de Negocio y Regla de los 90 Días en PostgreSQL 123
Módulo: Taller de Reparaciones y Órdenes de Trabajo 124
Implementación en Capa Frontend (React) 126
Código Backend: Consumo de Repuesto en Taller y Descuento Atómico 126
Módulo: Gestión de Inventario y Stock 127
Módulo: Gestión de Proveedores y Compras 129
Módulo: Reportes y Estadísticas 130
Consulta SQL de Consolidación Financiera y Rentabilidad 131
Módulo: Configuración del Sistema y Respaldo 132
Arquitectura General y Decisiones de Diseño Transversales 133
Arquitectura de Capas y Separación de Responsabilidades 133
Transaccionalidad ACID y Consistencia de Inventario 134
Mecanismo de Seguridad, Hash Bcrypt y Tokens JWT 135
Middlewares de Autorización y Roles (RBAC) 135
Empaquetado y Distribución Desktop (Electron Builder y Monorepo) 136
Conclusiones y Trabajo Futuro 137
Conclusiones 137
Evolución y Maduración del Diseño (2025 → 2026) 139
Decisiones técnicas y aprendizajes 140
Limitaciones y desafíos encontrados 140
Limitaciones previstas en el alcance: 140
Desafíos técnicos durante el desarrollo: 141

Trabajo Futuro **141**
Nuevas funcionalidades (alta prioridad) 141
Mejoras sobre funcionalidades existentes 141
Mejoras técnicas y de calidad 142
Reflexión Final **142**
Anexos **143**

Introducción

Iniciando este proyecto teníamos bastante claro con quién trabajariamos, Diego nos dio la oportunidad de conocer las fortalezas y debilidades de su negocio desde su punto de vista para lograr comprender su problemática. La empresa comercial y taller mecánico **"DN Bikes"**, de Diego Nagel, se encuentra radicada en la ciudad de Esperanza, provincia de Santa Fe. Desarrolla sus actividades comerciales y de servicio técnico orientadas al sector ciclismo tanto en el ámbito recreativo como urbano y deportivo, atendiendo a clientes de Esperanza y de localidades aledañas.

En el área de venta comercial, se enfoca en la comercialización directa al público de bicicletas nuevas (rodados infantiles, paseo, MTB, gravel y competición), componentes, repuestos e indumentaria deportiva.

En el área de taller mecánico, ofrece diagnóstico técnico, armado de rodados a medida, reparaciones generales, service preventivo de transmisión y frenos, centrado de ruedas y mantenimiento oficial de garantías de post-venta.

La empresa está compuesta por su titular y administrador Diego Nagel cuenta con un equipo de trabajo compuesto personal de ventas y atención en mostrador, y mecánicos especializados en el sector de taller.

En contacto con la empresa pudimos dialogar sobre las distintas áreas de mejora y profundizar en el conocimiento del funcionamiento del negocio. Con las conclusiones del relevamiento y la entrevista realizada, consideramos que aportamos valor directo al establecimiento desarrollando un sistema informático integral de gestión comercial, taller y control de stock, ya que hasta el momento operaban mediante anotaciones manuales en cuadernos de taller y planillas de cálculo de Excel dispersas.

Fundamentos del proyecto

En la actualidad, DN Bikes gestiona gran parte de sus operaciones cotidianas a través de registros informales en papel y planillas de cálculo. Esta modalidad, aunque permitió resolver transitoriamente el día a día, presenta graves limitaciones operativas: extravío frecuente de órdenes de reparación, falta de trazabilidad en el historial de servicios realizados a cada rodado, desconocimiento del stock real en mostrador frente a las piezas consumidas en

taller, y ausencia de herramientas automatizadas para consolidar la facturación y el balance de caja al cierre de cada jornada laboral.

Durante las entrevistas realizadas al personal y al titular, se identificó como necesidad prioritaria centralizar y sistematizar toda la información operativa del negocio en una única herramienta confiable. La digitalización no solo previene pérdidas económicas por repuestos no registrados o errores en cobros, sino que agiliza de forma sustancial la atención al cliente en el mostrador.

El desarrollo de un sistema informático a medida para bicicletería y taller permitirá registrar ordenadamente las ventas, controlar el inventario con descuentos atómicos en tiempo real, gestionar el parque rodado de los clientes con su historial clínico completo, administrar las órdenes de trabajo del taller mediante un flujo claro de estados, y realizar un seguimiento estricto de las garantías post-venta de 90 días para el primer service gratuito. Además, la posibilidad de generar reportes analíticos y balances automáticos aportará información estratégica para las compras y la toma de decisiones comerciales.

En resumen, la implementación de Bike System representa una mejora sustancial en la eficiencia operativa, el control financiero y la calidad de atención de DN Bikes, dotando al negocio de una plataforma tecnológica moderna y adaptada a sus necesidades específicas.

Alcance y Límites

Este proyecto tiene como objetivo el desarrollo integral de un sistema de gestión para bicicletería y taller mecánico, en formato de aplicación de escritorio (Desktop) para el sistema operativo Windows, desarrollada sobre Electron, React, Express y PostgreSQL. La aplicación está diseñada bajo una arquitectura de escritorio conectada con persistencia centralizada en la nube, garantizando la sincronización en tiempo real de inventario y órdenes de trabajo entre las terminales de mostrador y taller, con resguardo de datos centralizado y transaccionalidad ACID.

El sistema contempla funcionalidades clave tales como el control integral de stock y catálogo de productos, registro de ventas al mostrador con cálculo de totales y emisión de comprobantes térmicos y PDF, gestión de clientes y parque rodado vinculado a su historial clínico (diferenciando las bicicletas de clientes del catálogo de rodados comerciales), administración de órdenes de trabajo de taller mediante un Tablero Kanban interactivo (con columnas *Recibida*, *En Reparación* y *Lista*, más historial de *Entregadas*), seguimiento automatizado del primer service oficial gratuito dentro de los 90 días posteriores a la compra, registro de pagos a proveedores mayoristas, administración del flujo de fondos y balances financieros consolidados (evolución arquitectónica respecto a la caja manual preliminar de 2025), control de acceso mediante roles (Empleado, Administrador y Superadmin), y generación de respaldos (backups) en caliente de la base de datos PostgreSQL.

Como límites y exclusiones de esta fase del desarrollo, no se contempla la conexión directa en tiempo real con pasarelas de pago bancarias mediante webhooks online (las cobranzas se registran declarativamente en efectivo, transferencia o tarjeta en mostrador), ni la emisión directa de Factura Electrónica vinculada a los servidores web de AFIP/ARCA, la cual queda proyectada para una etapa posterior. Asimismo, no se incluye una plataforma de comercio electrónico web B2C ni una aplicación móvil orientada a clientes particulares; la atención se concentra exclusivamente en el salón de ventas y taller físico del establecimiento.

Objetivo general y Objetivos Específicos

Objetivo General

Desarrollar un sistema informático de escritorio para la gestión integral de bicicletería y taller mecánico, que permita registrar, controlar y optimizar de manera centralizada la administración de stock, ventas de mostrador, clientes y rodados, reparaciones de taller, garantías oficiales, compras y finanzas en la empresa DN Bikes.

Objetivos Específicos

- Diseñar un módulo de inventario que permita gestionar el catálogo de bicicletas, repuestos y accesorios, con alertas automáticas de stock mínimo y trazabilidad de movimientos en kardex.
- Desarrollar un módulo de punto de venta para agilizar la facturación en mostrador, permitiendo múltiples medios de pago y emisión de comprobantes en PDF y formato ticket.
- Implementar la gestión de clientes y su parque de bicicletas, registrando marca, modelo para mantener un historial clínico de servicios.
- Crear un módulo de taller mecánico que gestione órdenes de trabajo mediante un tablero de estados, asignación de repuestos con descuento automático de inventario y cálculo de mano de obra.
- Automatizar el seguimiento del primer service de mantenimiento preventivo gratuito con vigencia de 90 días tras la venta de una bicicleta nueva.
- Registrar las compras y pagos a distribuidores mayoristas de repuestos para mantener actualizados los costos y saldos de proveedores.
- Proveer herramientas de control de caja diaria con arqueo de valores, registrando ingresos y egresos de efectivo.
- Generar reportes analíticos y balances financieros para evaluar la rentabilidad del negocio y los productos más vendidos.
- Implementar un esquema de seguridad con autenticación basada en JWT, control de acceso por roles (Empleado, Administrador y Superadmin) y mecanismos de respaldo y restauración de la base de datos PostgreSQL

Relevamiento y análisis de Requerimientos

Requerimientos Funcionales

Los siguientes requerimientos funcionales describen las acciones y funcionalidades que el sistema de gestión deberá permitir al usuario realizar:

Gestión de clientes y parque rodado:

- RF1: Registrar, consultar y actualizar datos de clientes (nombre, apellido, DNI, teléfono, dirección).
- RF2: Vincular bicicletas al cliente titular, registrando marca, modelo, rodado, color y número de serie del cuadro.
- RF3: Consultar el historial clínico completo de servicios y reparaciones mecánicas realizadas a una bicicleta.

Gestión de productos y stock:

- RF4: Administrar el catálogo de productos (bicicletas nuevas, repuestos, indumentaria, accesorios) con código de barras, precio de costo, precio de venta y stock mínimo.
- RF5: Alertar automáticamente cuando el stock de un repuesto o producto se encuentre por debajo del umbral mínimo de seguridad.
- RF6: Registrar ajustes manuales de inventario con motivo de auditoría y responsable asignado.
- RF7: Registrar todos los movimientos de entrada y salida de inventario en una tabla de kardex unificado.

Gestión de taller y reparaciones:

- RF8: Registrar órdenes de reparación al ingresar un rodado al taller, indicando el cliente, la bicicleta y el diagnóstico inicial.
- RF9: Gestionar el flujo de la reparación mediante estados: Pendiente, En Proceso, Esperando Repuesto, Lista y Entregada.
- RF10: Asignar repuestos a una orden de trabajo, descontados de manera automática del inventario general.
- RF11: Liquidar el costo total de la reparación sumando el valor de los repuestos consumidos y la mano de obra del mecánico.

Punto de venta y facturación:

- RF12: Registrar ventas comerciales de mostrador con buscador predictivo de productos, cálculo automático de subtotales e impuestos, y selección de cliente o consumidor final.
- RF13: Soporta múltiples métodos de pago (efectivo, transferencia bancaria, tarjeta de débito y crédito).

- RF14: Generar e imprimir comprobantes de venta y tickets de entrega en formato PDF y ticket térmico.

Garantías y primer service:

- RF15: Habilitar automáticamente una garantía oficial con primer service gratuito de 90 días ante la venta de una bicicleta nueva.
- RF16: Validar la vigencia temporal de la garantía y redimirse en taller, registrando el mantenimiento preventivo realizado.

Gestión de compras y proveedores:

- RF17: Registrar facturas de compra a proveedores mayoristas, incrementando el stock de productos y actualizando costos.
- RF18: Registrar pagos efectuados a proveedores mayoristas con número de comprobante y forma de pago.

Control de caja diaria y finanzas:

- RF19: Gestionar el balance y flujo de fondos diario y periódico, consolidando automáticamente la recaudación por ventas de mostrador y servicios mecánicos.
- RF20: Calcular la rentabilidad neta del establecimiento en tiempo real confrontando la recaudación total frente a los egresos por pagos a proveedores.

Reportes y estadísticas:

- RF21: Generar balances financieros de ingresos por ventas y taller versus egresos por compras y gastos operativos.
- RF22: Visualizar estadísticas de productos con mayor rotación, marcas más ingresadas al taller y recaudación por período.

Seguridad y administración del sistema:

- RF23: Controlar el acceso mediante usuario y contraseña protegida con hash criptográfico.
- RF24: Asignar roles operativos (Administrador y Empleado), restringiendo el acceso a balances financieros y configuración.
- RF25: Exportar e importar copias de respaldo completas de la base de datos PostgreSQL.

Requerimientos No Funcionales

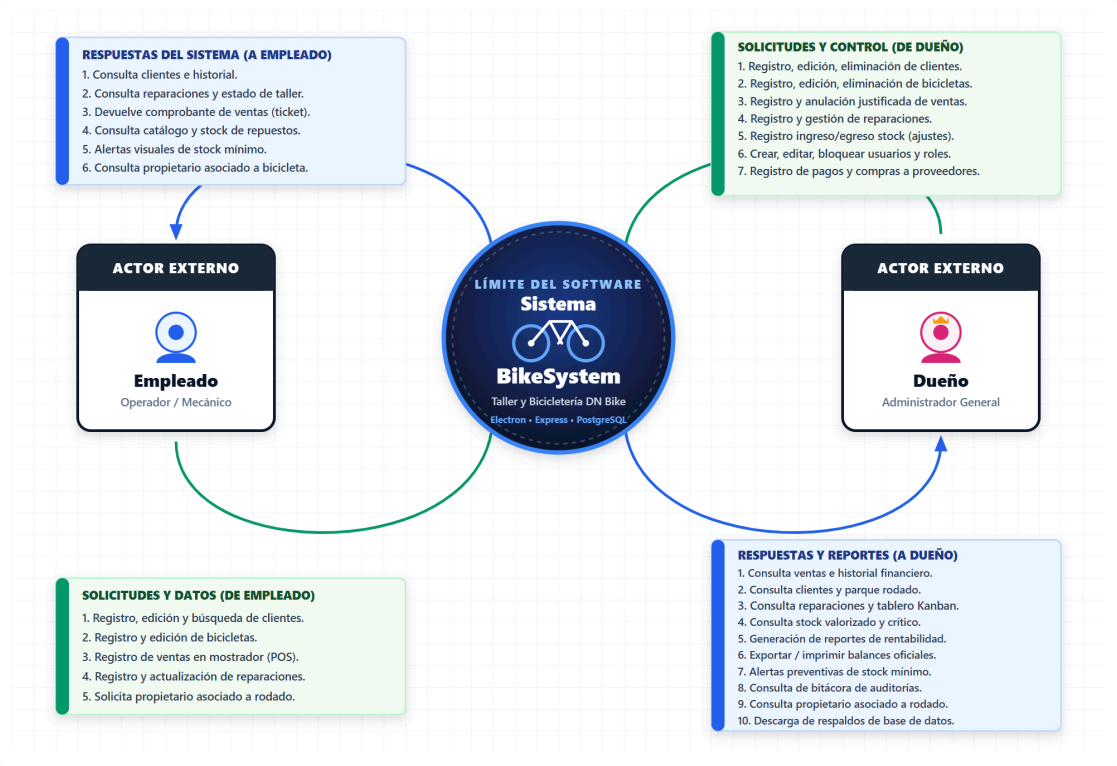
Los requerimientos no funcionales determinan aspectos técnicos, operativos y de

calidad del sistema:

- Usabilidad: La interfaz gráfica debe ser clara, moderna e intuitiva, accesible para usuarios con conocimientos técnicos básicos, permitiendo realizar operaciones comunes en pocos clics con una curva de aprendizaje mínima.
- Disponibilidad y Conectividad: El sistema debe operar con alta disponibilidad mediante conexión segura a la base de datos PostgreSQL alojada en la nube, asegurando latencias de respuesta menores a 200 milisegundos para operaciones de mostrador, taller y control de stock multi-terminal.
- Rendimiento: Las búsquedas predictivas de productos y clientes deben responder en menos de 100 milisegundos, y las operaciones de guardado y cálculo de transacciones deben ejecutarse de forma inmediata sin demoras perceptibles.
- Integridad Transaccional: Toda operación crítica que altere múltiples entidades (como una venta con cobro y descuento de stock, o una orden de taller con consumo de repuestos) debe ejecutarse bajo transacciones atómicas ACID en PostgreSQL, asegurando consistencia absoluta.
- Seguridad: Las contraseñas se almacenan cifradas mediante el algoritmo de hash Bcrypt con salt. La sesión se gestiona con tokens JWT y las rutas de la API están protegidas según el perfil de privilegios asignado.
- Portabilidad y Empaquetado: La aplicación debe empaquetarse como ejecutable de escritorio para Windows mediante Electrón y NSIS, facilitando su instalación directa en las terminales del negocio.
- Almacenamiento y Respaldo: Los datos se almacenan en el motor relacional PostgreSQL 16. El sistema debe permitir generar respaldos completos en formato SQL para proteger la información frente a eventualidades.
- Mantenibilidad: El código se estructura bajo una arquitectura modular y desacoplada (Frontend en React, Backend en Express, Base de Datos en PostgreSQL), facilitando futuras extensiones sin afectar el núcleo del sistema.

Diagramas UML

Figura 1: Diagrama de Contexto



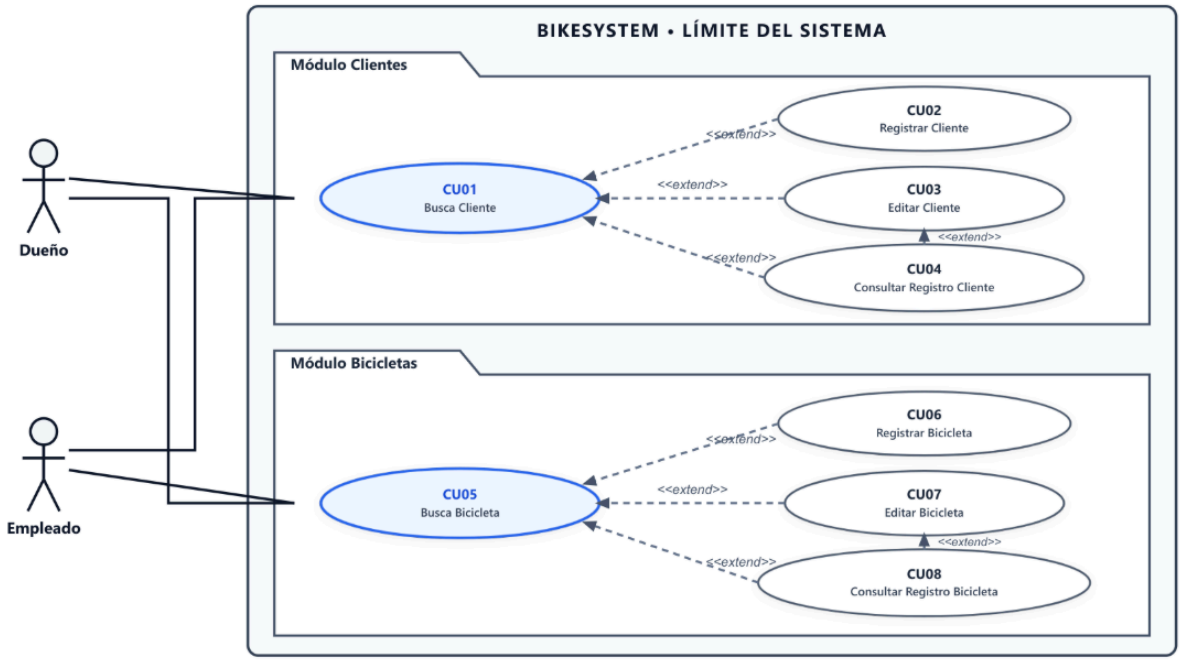
Nota: El Diagrama de Contexto delimita las fronteras del sistema, identificando a los usuarios que interactúan directamente con la aplicación y los periféricos externos vinculados.

Los actores y elementos identificados son:

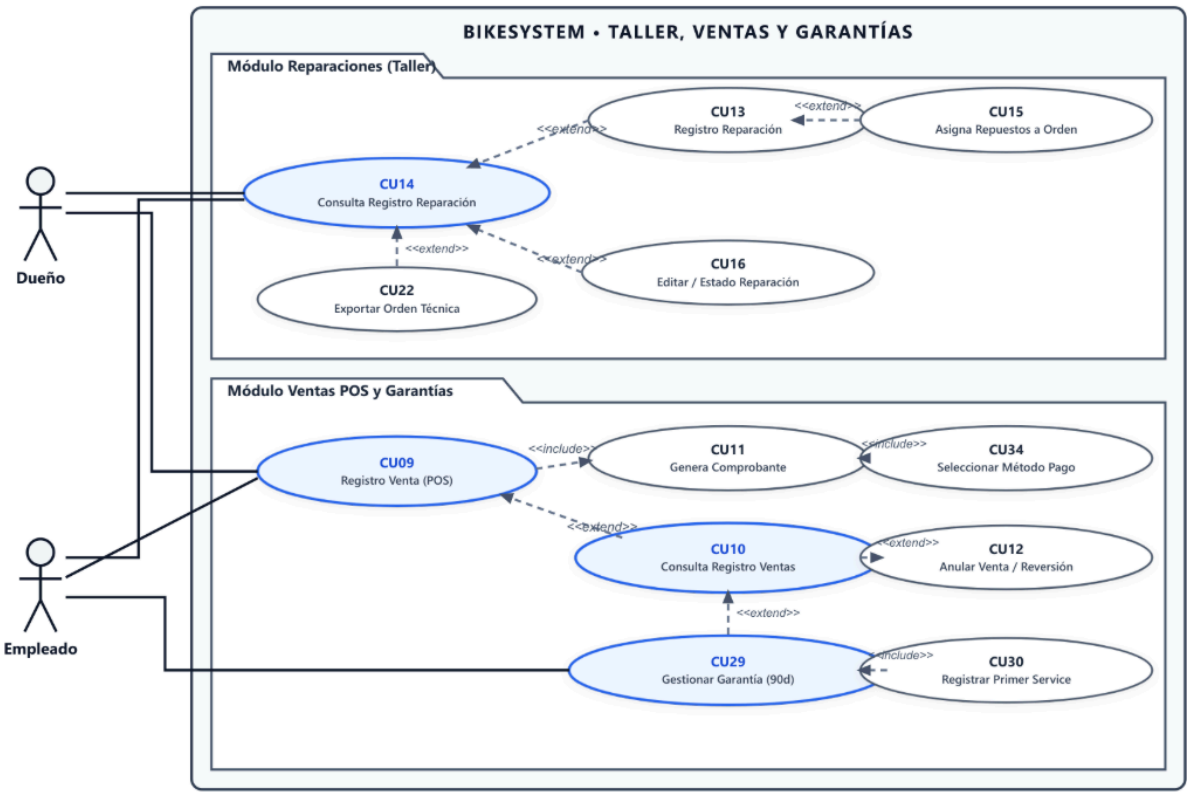
- Dueño / Administrador: Usuario con acceso completo a todas las funciones: configuración, gestión de usuarios, auditoría de eventos, balances financieros, compras a proveedores y respaldos de base de datos.
- Empleado / Mecánico: Usuario con acceso operativo restringido: registro de clientes, facturación en punto de venta (POS), tablero de reparaciones en taller y visualización de inventario.
- Impresora Térmica: Periférico de hardware local para la emisión rápida de comprobantes y tickets de trabajo.
- Sistema de Archivos: Destino local para la exportación de comprobantes en PDF y archivos de respaldo de PostgreSQL.

Figura 2: Diagramas de casos de usos.

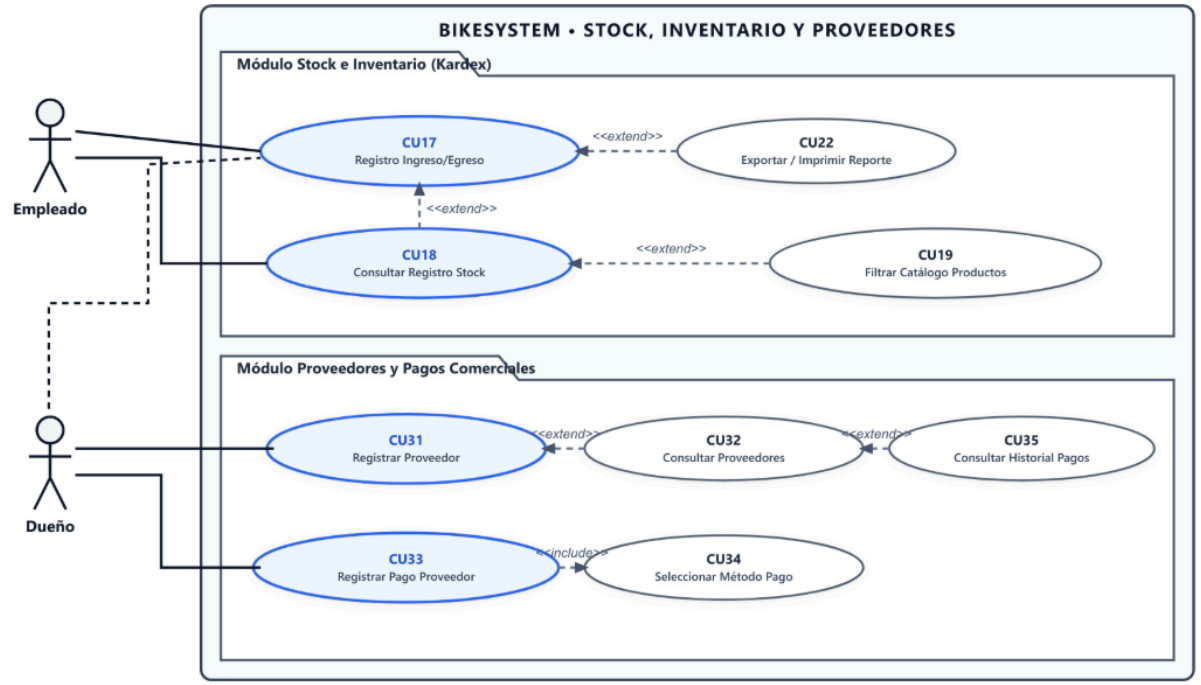
Módulo cliente y bicicleta.



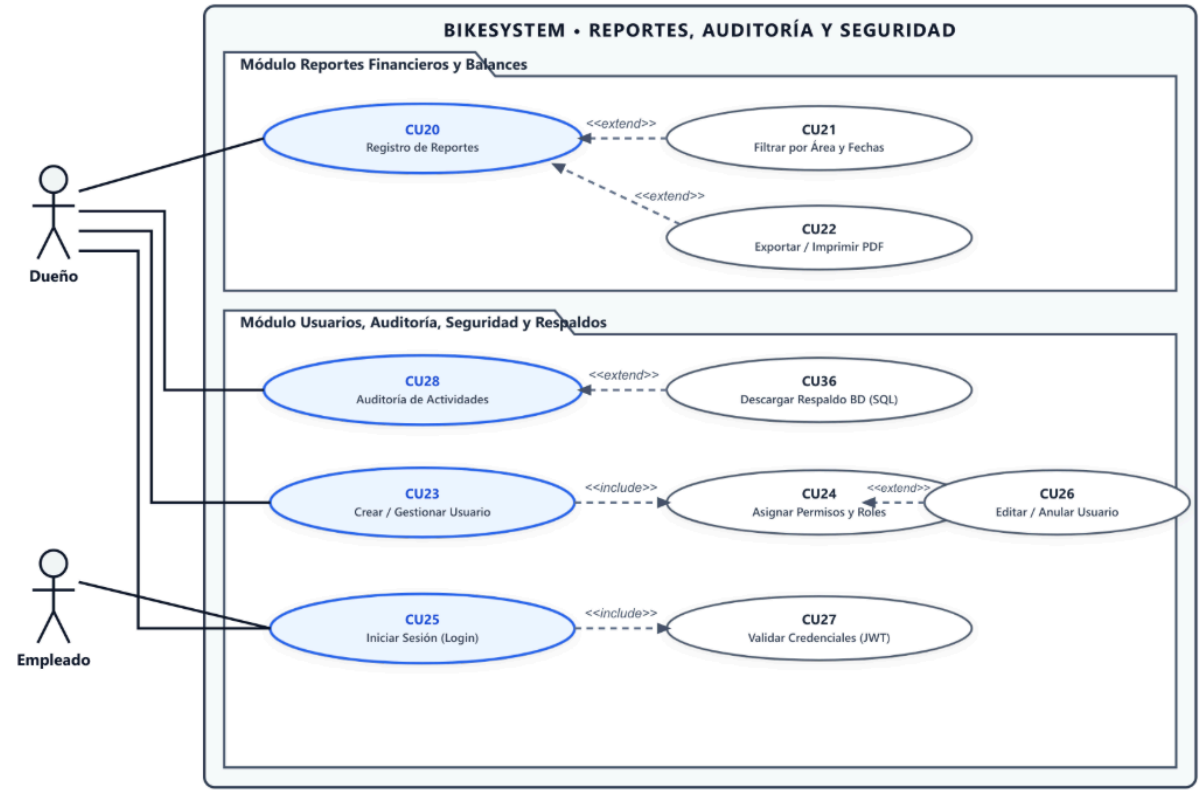
Módulo reparacion y venta.



Módulo stock y proveedores.



Módulos Reportes, usuarios, seguridad, auditoría, seguridad.



Descripción de los casos de usos.

CU01: Buscar Cliente

CU01	**Buscar Cliente**		
Descripción	Permite al usuario localizar un cliente registrado mediante el ingreso de datos como nombre, apellido, teléfono o DNI, con el fin de visualizar su ficha, editar su información o seleccionarlo para operaciones de venta o servicio técnico.		
Actores	Dueño, Empleado		
Pre condiciones	El operador ha iniciado sesión y cuenta con permisos para acceder al módulo de clientes. Deben existir clientes previamente registrados en la base de datos.		
Post condiciones	El sistema despliega en pantalla la lista de clientes coincidentes con el criterio de búsqueda ingresado.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario accede al módulo de clientes en la barra lateral.	El sistema presenta la vista de gestión de clientes con el campo de búsqueda rápida activado.
	2	Ingresa el criterio de búsqueda (nombre, apellido, teléfono o número de DNI).	El frontend dispara la consulta predictiva mediante debounce a GET /api/clientes?q=....
	3	El sistema ejecuta la búsqueda con índices trigram en PostgreSQL.	Retorna la lista de clientes coincidentes en menos de 100 ms y renderiza la tabla de resultados.
	4	El usuario selecciona el cliente deseado de la grilla.	El sistema resalta el registro y habilita las acciones de consultar ficha, editar datos o asociar a orden de venta/taller.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Criterio de búsqueda en blanco.	El sistema restablece la grilla mostrando el listado general paginado.

	2	Sin coincidencias en la base de datos.	Muestra un mensaje informativo en pantalla: 'No se encontraron clientes coincidentes' con botón para 'Registrar nuevo cliente'.
Rendimiento	El sistema deberá realizar la búsqueda fonética y numérica de forma inmediata (< 150 ms) aprovechando los índices GIN Trigram (pg_trgm) en PostgreSQL.		
Frecuencia	Alta (múltiples veces al día)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	La búsqueda admite coincidencia parcial insensibles a mayúsculas/minúsculas y acentos gracias a los índices GIN Trigram (pg_trgm). **Relaciones:** <<extend>> CU03 (Editar Cliente), <<extend>> CU04 (Consultar Registro de Clientes).		

CU02: Registrar Cliente

CU02	**Registrar Cliente**		
Descripción	Permite dar de alta un nuevo cliente en el sistema ingresando sus datos personales (nombre, apellido, DNI, teléfono, email y dirección) para futuras transacciones comerciales o recepciones de taller.		
Actores	Dueño, Empleado		
Pre condiciones	El usuario cuenta con permisos operativos para crear registros en el módulo de clientes.		
Post condiciones	El nuevo cliente queda almacenado de forma permanente en la tabla Cliente, disponible para ventas y servicios, y se asienta el evento en la bitácora de actividades.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario presiona el botón 'Nuevo Cliente' en la barra superior del módulo.	El sistema abre el modal interactivo con el formulario de captura.

	2	Completa los campos obligatorios: Nombre, Apellido, DNI, Teléfono y opcionales: Email y Dirección.	El cliente valida en tiempo real formatos numéricos de DNI/teléfono y sintaxis de correo electrónico.
	3	Presiona el botón 'Guardar Cliente'.	Envía una solicitud HTTP POST a /api/clientes con el payload JSON.
	4	El backend valida unicidad de DNI y almacena el registro en PostgreSQL.	Retorna HTTP 201 Created, cierra el modal, refresca la grilla y muestra notificación de éxito: 'Cliente registrado correctamente'.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Faltan campos obligatorios requeridos.	El formulario bloquea el botón de envío y resalta los campos con error: 'Debe completar los campos obligatorios'.
	2	DNI ya registrado en la base de datos.	El servidor responde con HTTP 400 Bad Request: 'Ya existe un cliente registrado con ese número de DNI'.
Rendimiento	La persistencia y validación de unicidad de DNI debe ejecutarse en menos de 200 ms.		
Frecuencia	Alta (al dar de alta nuevos clientes)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	El DNI debe ser numérico único en el padrón, excepto en el registro especial del sistema 'Consumidor Final' que posee DNI nulo o genérico. **Relaciones:** <<extend>> CU01 (Buscar Cliente).		

CU03: Editar Cliente

CU03	Editar Cliente		
Descripción	Permite modificar los datos personales y de contacto de un cliente existente, como su teléfono, dirección, correo electrónico o apellido, garantizando integridad referencial con sus órdenes previas.		
Actores	Dueño, Empleado		
Pre condiciones	El cliente debe existir en la base de datos y haber sido seleccionado desde la grilla o búsqueda.		
Post condiciones	Los datos del cliente quedan actualizados en PostgreSQL y se registra la acción en la bitácora inmutable de auditoría.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario localiza al cliente y hace clic en la acción 'Editar' (ícono de lápiz).	El sistema despliega el modal de edición precargado con los datos actuales del cliente.
	2	El operador modifica los campos que requieren actualización.	El sistema verifica en tiempo real las restricciones de formato en los campos editados.
	3	Presiona el botón 'Guardar Cambios'.	El frontend envía una petición HTTP PUT a /api/clientes/:id.
	4	El servidor actualiza el registro en la tabla Cliente y genera el asiento de auditoría.	Retorna confirmación HTTP 200 OK, actualiza la fila correspondiente en la tabla visual y notifica: 'Datos actualizados correctamente'.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	DNI modificado coincide con otro cliente distinto.	El sistema aborta la actualización y alerta: 'El DNI ingresado pertenece a otro cliente registrado'.
	2	Fallo de conexión o persistencia.	El sistema muestra: 'Error al actualizar los datos del cliente. Intente nuevamente'.
Rendimiento	La actualización de los datos del cliente en PostgreSQL debe realizarse en menos de 200 ms.		

Frecuencia	Media (cuando un cliente actualiza datos de contacto)
Importancia	**Media**
Urgencia	Puede esperar
Comentarios	Las modificaciones de clientes mantienen intacto su identificador primario (id_cliente) asegurando la integridad de facturas y órdenes históricas. **Relaciones:** <<extend>> CU04 (Consultar Registro de Clientes), <<extend>> CU01 (Buscar Cliente).

CU04: Consultar Registro de Clientes

CU04	**Consultar Registro de Clientes**		
Descripción	Permite visualizar el catálogo completo de clientes registrados en el establecimiento, ordenados alfabéticamente o por fecha de alta, con indicadores de compras y bicicletas asociadas.		
Actores	Dueño, Empleado		
Pre condiciones	Existen clientes registrados en la base de datos.		
Post condiciones	Se visualiza la grilla general de clientes con paginación, totalizadores y accesos directos a ficha detallada.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador hace clic en 'Clientes' en el menú de navegación.	El sistema realiza una petición GET a /api/clientes con parámetros de paginación.
	2	El backend consulta PostgreSQL y retorna el listado ordenado.	La interfaz renderiza la tabla interactiva mostrando Nombre, Apellido, DNI, Teléfono, Email y Dirección.
	3	El usuario puede navegar entre páginas o seleccionar un cliente para ver su ficha clínica.	El sistema permite expandir el panel lateral con las bicicletas y órdenes vinculadas al cliente.
Excepciones	#	Acción (actor)	Reacción (sistema)

	1	No existen clientes cargados en el sistema.	Muestra una ilustración de estado vacío con la leyenda: 'No hay clientes registrados aún. Cree el primero con el botón superior'.
Rendimiento	La carga inicial de la grilla paginada debe responder en menos de 250 ms.		
Frecuencia	Alta (visualización permanente en salón)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	La grilla implementa paginación en servidor (20 registros por página) para garantizar fluidez operativa incluso con miles de clientes registrados. **Relaciones:** <<extend>> CU01 (Buscar Cliente).		

CU05: Busca Bicicleta

CU05	**Busca Bicicleta**		
Descripción	Permite localizar rodados en el parque del taller mediante el ingreso de id, modelo, marca, color o por el nombre del propietario asignado.		
Actores	Dueño, Empleado		
Pre condiciones	El usuario ha iniciado sesión. Deben existir bicicletas registradas en el sistema.		
Post condiciones	El sistema presenta en pantalla las bicicletas coincidentes con los criterios de búsqueda.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario ingresa al módulo de Bicicletas en la barra de navegación.	El sistema despliega el catálogo de rodados registrados.
	2	Ingresa el texto de búsqueda en la barra superior (ej. 'Trek Marlin' o '12').	El frontend consulta GET /api/bicicletas?q=... de manera asíncrona.

	3	PostgreSQL busca coincidencias en Bicicleta y realiza JOIN con Cliente.	La vista actualiza la grilla con las bicicletas encontradas y los datos de sus dueños.
	4	El usuario selecciona la bicicleta deseada.	Habilita la consulta de ficha técnica, edición o apertura de una nueva orden de servicio.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	No se encuentran bicicletas con el criterio ingresado.	El sistema informa: 'No se encontraron bicicletas coincidentes con el criterio de búsqueda'.
Rendimiento	La búsqueda predictiva por cuadro, marca o titular debe responder en menos de 180 ms.		
Frecuencia	Alta (al ingresar o retirar rodados del taller)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	El campo de número de serie admite caracteres alfanuméricos y es el identificador técnico principal para el seguimiento de garantías. **Relaciones:** <<extend>> CU06 (Registrar Bicicleta), <<extend>> CU08 (Consultar Registro de Bicicleta).		

CU06: Registrar Bicicleta

CU06	**Registrar Bicicleta**
Descripción	Permite registrar una nueva bicicleta en el taller vinculándola obligatoriamente a un cliente titular, detallando marca, modelo, color, rodado, tipo y número de serie único.
Actores	Dueño, Empleado
Pre condiciones	El cliente dueño debe existir previamente en el sistema. El usuario tiene permisos operativos.
Post condiciones	La bicicleta queda dada de alta en la tabla Bicicleta asociada a su propietario y se asienta el evento en bitácora.

Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa el botón 'Registrar Bicicleta' en el módulo correspondiente.	El sistema abre el modal con el formulario técnico de alta.
	2	Selecciona al propietario mediante el selector predictivo de clientes (invoca CU01).	El sistema asocia el id_cliente al formulario.
	3	Completa marca, modelo, color, rodado (26, 27.5, 29, etc.), talle y número de serie.	El formulario valida que los campos técnicos no se encuentren vacíos.
	4	Presiona 'Guardar Bicicleta'.	Envía una petición POST a /api/bicicletas.
	5	El backend valida que el número de serie no esté duplicado y almacena en PostgreSQL.	Retorna HTTP 201 Created, cierra el modal y notifica: 'Bicicleta registrada con éxito'.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Número de serie ya existente en la base de datos.	El sistema rechaza la operación informando: 'El número de serie ingresado ya se encuentra registrado para otra bicicleta'.
	2	No se ha seleccionado un cliente titular.	Bloquea el guardado indicando: 'Debe seleccionar un cliente propietario registrado'.
Rendimiento	El alta técnica y vinculación al cliente debe registrarse en menos de 200 ms.		
Frecuencia	Alta (al ingresar nuevos rodados a servicio técnico)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	Cada bicicleta debe tener exactamente un cliente titular asignado (relación 1:N entre Cliente y Bicicleta en el modelo de datos). **Relaciones:** <<extend>> CU05 (Busca Bicicleta).		

CU 07: Editar Bicicleta

CU 07	**Editar Bicicleta**		
Descripción	Permite modificar las características registradas de una bicicleta (color, rodado, talle, notas técnicas o transferencia de titular a otro cliente).		
Actores	Dueño, Empleado		
Pre condiciones	La bicicleta debe existir en el sistema y haber sido seleccionada previamente.		
Post condiciones	Los atributos técnicos de la bicicleta quedan actualizados en PostgreSQL y se registra la actividad en bitácora.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador selecciona la opción 'Editar' en la fila de la bicicleta.	Se abre el formulario modal con los datos cargados en campos editables.
	2	Modifica los atributos requeridos (ej. actualización de color o corrección de rodado).	El sistema valida la integridad de los nuevos datos.
	3	Presiona 'Guardar Cambios'.	El cliente envía solicitud HTTP PUT a /api/bicicletas/:id.
	4	El servidor persiste la actualización en la base de datos.	Retorna código HTTP 200 OK y muestra mensaje: 'Datos de bicicleta actualizados correctamente'.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Número de serie editado coincide con otra bicicleta existente.	El sistema muestra: 'Conflicto: el número de serie ya pertenece a otro rodado registrado'.
	2	Campos obligatorios incompletos.	Resalta los campos faltantes e impide el envío.

Rendimiento	La actualización técnica de la bicicleta debe completarse en menos de 200 ms.
Frecuencia	Media (correcciones en ficha técnica del rodado)
Importancia	**Media**
Urgencia	Puede esperar
Comentarios	La modificación técnica de la bicicleta no altera su historial de reparaciones previas, preservando la trazabilidad mecánica. **Relaciones:** <<extend>> CU08 (Consultar Registro de Bicicleta), <<extend>> CU05 (Busca Bicicleta).

CU08: Consultar Registro de Bicicleta

CU08	**Consultar Registro de Bicicleta**		
Descripción	Permite visualizar el legajo integral de una bicicleta, detallando todas las intervenciones de taller recibidas, mecánicos intervinientes, repuestos instalados y garantías vigentes.		
Actores	Dueño, Empleado		
Pre condiciones	La bicicleta debe encontrarse dada de alta en el sistema.		
Post condiciones	El sistema despliega la historia clínica completa de la bicicleta, incluyendo ficha técnica, propietario e historial cronológico de servicios.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador localiza la bicicleta y selecciona 'Ver Historial' o hace clic en su tarjeta.	El frontend solicita GET /api/bicicletas/:id junto con GET /api/reparaciones?bicicleta=:id.
	2	El backend realiza las consultas y compila el historial clínico del rodado.	Retorna los datos de la bicicleta, datos del titular y el listado de reparaciones históricas.
	3	La interfaz despliega la vista detallada con línea de tiempo.	El usuario revisa las fechas de servicios anteriores, diagnósticos previos y piezas reemplazadas.

Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Bicicleta sin órdenes de taller registradas.	El sistema muestra la ficha técnica e indica en el panel de historial: 'Sin reparaciones previas registradas'.
Rendimiento	La recuperación del legajo técnico e historial de servicios debe completarse en menos de 250 ms.		
Frecuencia	Alta (diagnóstico y revisión de antecedentes técnicos)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	La historia clínica de reparaciones es inmutable hacia el pasado, sirviendo como respaldo técnico ante reclamos de garantía o reventa. **Relaciones:** <<extend>> CU05 (Busca Bicicleta).		

CU 09: Registro Venta (POS)

CU 09	**Registro Venta (POS)**		
Descripción	Permite facturar una venta de salón en el Punto de Venta (POS), agregando productos o bicicletas al carrito, calculando subtotales, IVA y total neto, seleccionando el medio de pago y actualizando el inventario en tiempo real.		
Actores	Dueño, Empleado		
Pre condiciones	Operador autenticado. Existen productos registrados con stock disponible. Se cuenta con un cliente seleccionado o Consumidor Final.		
Post condiciones	Se genera la venta en Venta y Detalle_Venta en transacción atómica, se descuenta el stock físico en Productos, se crea el asiento en Movimiento_Stock y se emite el comprobante.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El vendedor accede a la vista de Ventas / Punto de Venta (POS).	El sistema presenta el buscador de catálogo y el

			panel lateral del carrito de compras.
	2	Selecciona el cliente comprador (por defecto asigna 'Consumidor Final').	El sistema asocia el comprador a la orden actual.
	3	Busca productos por código de barras o nombre y los agrega al carrito definiendo cantidades.	Calcula automáticamente el total sumando los precios unitarios por cantidad.
	4	Presiona 'Confirmar Venta' y selecciona la modalidad de pago (CU34).	Abre la confirmación transaccional y verifica disponibilidad física de stock.
	5	Confirma la operación.	El backend ejecuta una transacción SQL (BEGIN ... COMMIT), inserta la cabecera, los ítems, decrementa el stock en Productos y registra en Kardex.
	6	El sistema dispara automáticamente la emisión del comprobante (CU11).	Genera el ticket térmico/PDF de venta y reinicia el carrito para la siguiente transacción.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Stock insuficiente en uno o más productos del carrito.	El sistema aborta la transacción, notifica el ítem con déficit y no permite confirmar la venta.
	2	Fallo durante la persistencia en base de datos.	La transacción realiza ROLLBACK automático, manteniendo el stock intacto y alertando al operador.
Rendimiento	La transacción atómica (inserción de venta, detalles y descuento físico de stock) debe completarse en menos de 200 ms bajo nivel de aislamiento ACID.		
Frecuencia	Muy Alta (operación continua en mostrador)		
Importancia	**Vital**		

Urgencia	Inmediatamente
Comentarios	La operación se ejecuta en una transacción atómica con nivel de aislamiento serializable para evitar lecturas fantasma o ventas concurrentes de stock negativo. **Relaciones:** <<include>> CU11 (Genera Comprobante), <<extend>> CU10 (Consulta Registro de Ventas).

CU10: Consulta Registro de Ventas

CU10	**Consulta Registro de Ventas**		
Descripción	Permite consultar el listado histórico de operaciones comerciales, con filtros avanzados por fecha, cliente, método de pago, rango de montos y estado (completada o anulada).		
Actores	Dueño, Empleado		
Pre condiciones	Existen ventas registradas en la base de datos.		
Post condiciones	Se visualiza el historial cronológico de comprobantes con opciones de filtrado y reimpresión.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario ingresa a la pestaña 'Historial de Ventas' dentro del módulo Ventas.	El sistema efectúa petición GET a /api/ventas con parámetros de fecha.
	2	Aplica filtros si es necesario (ej. ventas del día de hoy o ventas abonadas por Transferencia).	El sistema actualiza la grilla y recalcula el total acumulado en pantalla.
	3	El usuario puede hacer clic en cualquier comprobante para ver su desglose completo de ítems.	El sistema abre el modal de detalle con los productos, precios unitarios y medio de pago.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	No se registran ventas para el período seleccionado.	Muestra aviso: 'No se encontraron ventas para los filtros especificados'.

Rendimiento	La consulta del historial y filtros de fecha debe retornar en menos de 250 ms.
Frecuencia	Alta (control diario de caja y facturación)
Importancia	**Importante**
Urgencia	Inmediatamente
Comentarios	Los usuarios con rol Empleado solo pueden visualizar el historial de ventas del turno actual, mientras que el Administrador/Dueño tiene acceso histórico sin restricciones. **Relaciones:** <<extend>> CU09 (Registro Venta).

CU11: Genera Comprobante

CU11	**Genera Comprobante**		
Descripción	Produce el comprobante de venta conteniendo el membrete de DN Bikes, número de ticket correlativo, datos del cliente, detalle de ítems, totales, método de pago y leyenda de garantía legal.		
Actores	Dueño, Empleado		
Pre condiciones	La venta ha sido confirmada y persistida exitosamente en el sistema.		
Post condiciones	Se genera un documento de comprobante listo para envío a impresora térmica (80mm) o guardado en formato PDF digital.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El sistema finaliza la persistencia de CU09 y activa la generación del comprobante.	El frontend solicita los datos del ticket a GET /api/ventas/:id/ticket.
	2	El backend compila la plantilla con numeración correlativa y código de barras/QR.	Retorna la estructura maquetada del comprobante.
	3	El operador elige entre imprimir ticket térmico o descargar copia PDF.	Electron invoca el servicio nativo de impresión o guarda el archivo en el sistema de archivos local.

Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Impresora térmica desconectada o fuera de línea.	El sistema ofrece guardar el comprobante en formato PDF en el disco rígido sin interrumpir la operatoria.
Rendimiento	El renderizado del comprobante y envío al spooler térmico (80 mm) o generación del PDF debe tardar menos de 300 ms.		
Frecuencia	Muy Alta (se ejecuta con cada venta confirmada)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	El comprobante impreso incluye obligatoriamente la fecha de emisión y el plazo legal de 90 días corridos de garantía para bicicletas y componentes nuevos. **Relaciones:** <<include>> CU34 (Seleccionar Método de Pago), <<extend>> CU22 (Exporta / Imprime Reporte).		

CU12: Editar / Anular Venta

CU12	**Editar / Anular Venta**		
Descripción	Permite revocar una operación de venta completada previamente, reintegrando físicamente la mercadería al inventario comercial y ajustando los balances financieros de caja.		
Actores	Dueño / Administrador		
Pre condiciones	El operador debe contar con rol ADMIN. La venta a anular debe estar en estado 'completada'.		
Post condiciones	El estado de la venta cambia a 'anulada', se restituyen automáticamente las cantidades al stock de Productos, se asienta la reversión en Kardex y se registra el evento en auditoría.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Dueño/Administrador localiza la venta en el historial y presiona 'Anular Venta'.	El sistema solicita confirmación con diálogo modal requiriendo motivo de la anulación.

	2	El operador ingresa el motivo (ej. 'Devolución de cliente' o 'Error en medio de pago') y confirma.	El frontend envía una petición HTTP PUT a /api/ventas/:id/anular.
	3	El backend abre una transacción SQL, actualiza estado = 'anulada' y restituye el stock mediante sumas en Productos.	Genera un asiento de tipo 'REVERSION' en Movimiento_Stock y asienta el log forense en Bitacora_Actividad.
	4	El sistema confirma la anulación.	Muestra notificación: 'Venta anulada correctamente. Stock restituido al inventario'.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Usuario sin privilegios de administrador intenta anular.	El middleware RBAC bloquea la petición con HTTP 403 Forbidden: 'Solo los administradores pueden anular ventas'.
	2	La venta ya se encontraba previamente anulada.	El backend rechaza la solicitud: 'La venta ya ha sido anulada con anterioridad'.
Rendimiento	La reversión atómica y restitución de stock físico debe ejecutarse en menos de 200 ms.		
Frecuencia	Baja (exclusivo ante devoluciones o errores de facturación)		
Importancia	**Media**		
Urgencia	Alta		
Comentarios	Una venta anulada no se elimina físicamente de la base de datos (eliminación lógica); permanece inmutable para respaldar la auditoría contable. **Relaciones:** <<extend>> CU10 (Consulta Registro de Ventas).		

CU13: Registro Reparación (Ingreso)

CU13	**Registro Reparación (Ingreso)**

Descripción	Permite recepcionar una bicicleta para servicio técnico o reparación mecánica, registrando la descripción de la falla reportada por el cliente, presupuesto estimado, fecha estimada de entrega y mecánico asignado.		
Actores	Dueño, Empleado / Mecánico		
Pre condiciones	El cliente y la bicicleta deben estar dados de alta en el sistema.		
Post condiciones	Se crea un nuevo registro en la tabla Reparacion en estado 'recibida', se asienta el evento y se emite la orden de trabajo.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario hace clic en 'Nueva Reparación' en el módulo Taller.	Se despliega el formulario modal de ingreso a taller.
	2	Selecciona al cliente y elige la bicicleta correspondiente entre los rodados registrados de ese titular.	El sistema carga los datos técnicos del rodado seleccionado.
	3	Describe el síntoma o trabajo solicitado (ej. 'Service integral, purgado de frenos hidráulicos Shimano, centrado de ruedas').	El operador define el costo estimado de mano de obra y fecha límite de entrega.
	4	Presiona 'Ingresar al Taller'.	Envía una solicitud POST a /api/reparaciones con estado inicial 'recibida'.
	5	El sistema guarda la orden en PostgreSQL y genera el número de orden correlativo.	Retorna confirmación, agrega la tarjeta a la primera columna del Kanban y emite el ticket de recepción para el cliente.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	La bicicleta seleccionada no pertenece al cliente indicado.	El sistema valida la integridad de la clave foránea e impide la creación.

	2	Falta detallar la descripción de la falla.	El formulario alerta: 'Debe ingresar el detalle del trabajo o falla a reparar'.
Rendimiento	La apertura de la orden de trabajo y emisión del ticket de taller debe tardar menos de 250 ms.		
Frecuencia	Alta (ingreso cotidiano de bicicletas al taller)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	Al ingresar, toda reparación adquiere automáticamente el estado inicial 'recibida', con timestamp de ingreso registrado por el servidor. **Relaciones:** <<extend>> CU14 (Consulta Registro de Reparación).		

CU14: Consulta Registro de Reparación

CU14	**Consulta Registro de Reparación**		
Descripción	Permite visualizar y coordinar el flujo de trabajo del taller mecánico mediante un tablero Kanban dinámico con 5 estados: Recibida, En Reparación, En Espera de Repuestos, Terminada y Entregada.		
Actores	Dueño, Empleado / Mecánico		
Pre condiciones	Existen órdenes de servicio técnico registradas en el sistema.		
Post condiciones	Se presenta el tablero interactivo Kanban organizado en columnas según el estado operativo de cada bicicleta.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El mecánico accede a la vista principal del módulo Taller.	El frontend solicita GET /api/reparaciones.
	2	El backend consulta PostgreSQL ordenando las órdenes por prioridad y fecha.	El frontend distribuye las tarjetas en las 5 columnas del tablero Kanban.

	3	El usuario puede filtrar por mecánico asignado, número de orden o bicicleta.	El tablero actualiza dinámicamente las tarjetas visibles.
	4	Al hacer clic sobre una tarjeta, se abre el panel lateral con la ficha completa de la orden.	Permite consultar diagnóstico, repuestos consumidos y costos acumulados.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	No hay reparaciones activas en el tablero.	Muestra el tablero con columnas vacías y mensaje de bienvenida a la jornada de taller.
Rendimiento	El tablero Kanban debe renderizar el estado de todas las órdenes activas en menos de 200 ms.		
Frecuencia	Muy Alta (tablero operativo en tiempo real en taller)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	El tablero Kanban actualiza sus columnas automáticamente sin recargar la página completa, optimizando los tiempos del taller mecánico. **Relaciones:** <<extend>> CU13 (Registro Reparación), <<extend>> CU16 (Editar / Estado Reparación), <<extend>> CU22 (Exporta / Imprime Reporte).		

CU15: Asigna Repuestos a Orden

CU15	**Asigna Repuestos a Orden**
Descripción	Permite imputar repuestos mecánicos del inventario (cables, cadenas, pastillas de freno, cubiertas, etc.) a una orden de trabajo, recalculando el importe total de materiales.
Actores	Dueño, Empleado / Mecánico
Pre condiciones	La orden de reparación debe encontrarse en estado activo ('recibida', 'en_reparacion' o 'en_espera'). Repuestos con stock disponible.

Post condiciones	Se registran los repuestos en Detalle_Reparacion, se decrementa el stock en Productos, se asienta el egreso en Kardex y se actualiza el costo total de la orden.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El mecánico abre la orden de reparación en el tablero Kanban y selecciona la pestaña 'Repuestos / Materiales'.	El sistema despliega el selector predictivo de repuestos disponibles en stock.
	2	Busca el repuesto requerido (ej. 'Pastillas Shimano B01S') e indica la cantidad utilizada.	El sistema verifica en tiempo real que exista stock suficiente disponible.
	3	Presiona 'Agregar Repuesto a la Orden'.	El cliente envía una solicitud POST a /api/reparaciones/:id/detalles.
	4	El backend ejecuta una transacción SQL: inserta en Detalle_Reparacion, decrementa stock en Productos y crea asiento en Movimiento_Stock.	Retorna la orden actualizada con el nuevo total consolidado (mano de obra + repuestos).
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Stock insuficiente del repuesto seleccionado.	El sistema alerta: 'Stock insuficiente para imputar la cantidad solicitada. Stock actual disponible: X'.
	2	La orden se encuentra en estado 'entregada' o 'cancelada'.	El sistema bloquea la operación: 'No se pueden añadir repuestos a órdenes finalizadas o cerradas'.
Rendimiento	El descuento de stock y cómputo de costos de materiales debe procesarse en menos de 180 ms.		
Frecuencia	Alta (durante la ejecución mecánica de servicios)		
Importancia	**Importante**		
Urgencia	Inmediatamente		

Comentarios	Los repuestos imputados a una orden de taller descuentan stock inmediatamente del inventario general para prevenir discrepancias físicas en el salón. **Relaciones:** <<extend>> CU13 (Registro Reparación).

CU 16: Editar / Estado Reparación (Kanban)

CU16	**Editar / Estado Reparación (Kanban)**		
Descripción	Permite arrastrar o cambiar el estado operativo de una bicicleta entre las columnas del Kanban (Recibida → En Reparación → En Espera → Terminada → Entregada), notificando el avance del servicio.		
Actores	Dueño, Empleado / Mecánico		
Pre condiciones	La orden de trabajo debe existir en el tablero Kanban.		
Post condiciones	El estado de la reparación se actualiza en PostgreSQL; si pasa a 'entregada', se asienta la fecha de entrega y el cobro final.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El mecánico arrastra la tarjeta de la bicicleta hacia otra columna (o selecciona el nuevo estado en el menú desplegable).	El frontend captura la transición y valida las reglas de flujo de estados.
	2	Si el estado destino es 'Entregada', solicita confirmación del cobro del saldo pendiente y método de pago (CU34).	El operador confirma la entrega y el cobro.
	3	El frontend envía una petición PUT a /api/reparaciones/:id/estado con el nuevo valor.	El backend actualiza la columna estado y asigna fecha_entrega = NOW() si corresponde.
	4	El sistema asienta el evento en la bitácora y confirma visualmente el movimiento en el tablero.	La tarjeta se fija en la nueva columna con indicador de color correspondiente.
Excepciones	#	Acción (actor)	Reacción (sistema)

	1	Transición de estado inválida.	El sistema rechaza la acción si se intenta pasar una orden cancelada a terminada sin reactivación.
	2	Intento de marcar como 'Entregada' con saldo impago no liquidado.	El sistema exige confirmar la liquidación o registrar el saldo pendiente.
Rendimiento	La transición de columna Kanban y actualización de timestamp de entrega debe procesarse en menos de 150 ms.		
Frecuencia	Muy Alta (a lo largo de todo el ciclo de trabajo mecánico)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	La fecha de entrega real (fecha_entrega) se congela automáticamente al momento de pasar al estado 'Entregada', habilitando la vigencia de la garantía técnica de mano de obra. **Relaciones:** <<extend>> CU14 (Consulta Registro de Reparación).		

CU17: Registro Ingreso/Egreso de Stock (Kardex)

CU17	**Registro Ingreso/Egreso de Stock (Kardex)**		
Descripción	Permite registrar movimientos físicos en el inventario comercial y de repuestos (entradas por compra a proveedor, salidas por merma/daño y ajustes de auditoría física).		
Actores	Dueño, Empleado		
Pre condiciones	Producto identificado en el catálogo del sistema.		
Post condiciones	Se genera un asiento inmutable en Movimiento_Stock, se recalcula stock_actual en la tabla Productos y se guarda el evento en bitácora.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador accede al módulo Stock y presiona 'Ajuste de Stock' o 'Registrar Movimiento'.	Se abre el modal interactivo de gestión de inventario.

	2	Selecciona el producto mediante el buscador predictivo y elige el tipo de movimiento: INGRESO, EGRESO o AJUSTE.	El sistema muestra el stock físico actual registrado en base de datos.
	3	Ingresa la cantidad a mover y el motivo de la operación (ej. 'Recepción factura Proveedor #1042' o 'Ajuste inventario mensual').	El sistema calcula el nuevo balance proyectado.
	4	Presiona 'Confirmar Movimiento'.	Envía una solicitud POST a /api/productos/:id/movimiento.
	5	El backend ejecuta una transacción atómica: inserta en Movimiento_Stock y actualiza el campo stock_actual.	Retorna HTTP 200 OK, actualiza la grilla y notifica: 'Movimiento de stock asentado correctamente'.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Egreso solicitado supera el stock físico disponible.	El sistema bloquea la operación con alerta: 'No se puede registrar un egreso que resulte en stock negativo'.
	2	Cantidad ingresada es cero o negativa.	Valida los campos obligando a ingresar valores enteros positivos mayores a cero.
Rendimiento	El asiento inmutable en Movimiento_Stock y actualización del balance de producto debe ejecutarse en menos de 180 ms.		
Frecuencia	Alta (ante recepción de mercadería, ajustes o mermas)		
Importancia	**Vital**		
Urgencia	Inmediatamente		
Comentarios	Cada alteración física de inventario deja constancia inmutable en la tabla de Kardex (Movimiento_Stock) con usuario, fecha, motivo y saldo resultante. **Relaciones:** <<extend>> CU18 (Consultar Registro de Stock), <<extend>> CU22 (Exporta / Imprime Reporte).		

CU 18: Consultar Registro de Stock

CU 18	**Consultar Registro de Stock**		
Descripción	Permite consultar el catálogo completo de repuestos, accesorios, indumentaria y bicicletas, monitoreando cantidades disponibles y detectando productos que requieren reposición urgente.		
Actores	Dueño, Empleado		
Pre condiciones	Existen productos cargados en la base de datos.		
Post condiciones	Se despliega la tabla de inventario con stock físico actual, precios de costo y venta, márgenes comerciales y alertas visuales de stock crítico.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa sobre 'Stock / Inventario' en la barra lateral.	El sistema realiza una solicitud GET a /api/productos.
	2	El backend consulta PostgreSQL y retorna todos los productos con sus cálculos de stock.	La interfaz renderiza la tabla interactiva.
	3	Los productos cuyo stock actual sea menor o igual al stock mínimo se resaltan automáticamente con una insignia de color rojo ('Stock Bajo').	El operador visualiza de forma instantánea las alertas de reposición.
	4	El usuario puede ordenar por código, nombre, categoría, stock o precio.	La grilla se reordena de inmediato en el cliente.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	No existen productos registrados.	Muestra pantalla vacía con botón para dar de alta el primer producto del catálogo.
Rendimiento	La consulta del catálogo con cálculo de stock crítico debe cargar en menos de 200 ms.		
Frecuencia	Alta (consulta continua durante la jornada)		

Importancia	**Importante**
Urgencia	Inmediatamente
Comentarios	El cálculo de stock bajo responde estrictamente a la regla de negocio: stock_actual ≤ stock_minimo. **Relaciones:** <<extend>> CU17 (Registro Ingreso/Egreso de Stock), <<extend>> CU19 (Filtrar Catálogo de Productos).

CU 19: Filtrar Catálogo de Productos

CU 19	**Filtrar Catálogo de Productos**		
Descripción	Permite filtrar el catálogo por categorías (Repuestos, Accesorios, Indumentaria, Bicicletas Nuevas), rango de precios, proveedor habitual o estado de inventario (Normal o Crítico).		
Actores	Dueño, Empleado		
Pre condiciones	Listado de inventario activo en pantalla.		
Post condiciones	La grilla visualiza únicamente los productos que cumplen con los criterios de filtrado seleccionados.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador selecciona una o más categorías en la botonera de filtros (ej. 'Repuestos' y 'Transmisión').	El frontend aplica el filtro combinado en memoria o solicita los datos al backend.
	2	Activa el interruptor 'Solo productos con Stock Crítico' si desea preparar una orden de compra.	La tabla se reduce a los artículos con déficit de existencias.
	3	El usuario consulta las cantidades para emitir pedidos de reposición a proveedores.	Habilita la exportación directa de la lista filtrada.
Excepciones	#	Acción (actor)	Reacción (sistema)

	1	La combinación de filtros no arroja ningún producto.	Se despliega la notificación: 'No hay productos que coincidan con los filtros seleccionados'.
Rendimiento	El filtrado reactivo combinado debe responder en menos de 100 ms en el cliente.		
Frecuencia	Alta (localización rápida de insumos en salón y taller)		
Importancia	**Media**		
Urgencia	Puede esperar		
Comentarios	Los filtros son acumulativos y se pueden limpiar con un solo clic mediante el botón 'Restablecer Filtros'. **Relaciones:** <<extend>> CU18 (Consultar Registro de Stock).		

CU20: Registro de Reportes

CU20	**Registro de Reportes**		
Descripción	Consolida las métricas comerciales y financieras del negocio: total facturado en ventas POS, total recaudado por servicios de taller mecánico, costos de repuestos, egresos pagados a proveedores y margen de ganancia líquida.		
Actores	Dueño / Administrador		
Pre condiciones	Usuario con rol ADMIN autenticado. Existen transacciones registradas en el sistema.		
Post condiciones	Se generan y despliegan en pantalla los tableros analíticos con gráficos de facturación, volumen de reparaciones de taller y balance neto de ingresos y egresos.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador ingresa al módulo 'Reportes y Métricas' en la navegación.	El sistema realiza peticiones paralelas a /api/reportes/balance, /api/reportes/ventas y /api/reportes/taller.

	2	El backend ejecuta consultas de agregación SQL SUM(), COUNT() y AVG() excluyendo registros anulados.	Retorna los consolidados financieros organizados por período y medio de pago.
	3	El frontend renderiza tarjetas KPI (Ingresos Ventas, Ingresos Taller, Gastos Proveedores, Rentabilidad Neta) y gráficos de evolución temporal.	El usuario analiza el desempeño económico de la bicicletería.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Acceso denegado por rol no autorizado.	El middleware bloquea con HTTP 403 Forbidden a usuarios con rol Empleado.
Rendimiento	La agregación analítica de ingresos, gastos y márgenes netos debe completarse en menos de 350 ms.		
Frecuencia	Periódica (al cierre de caja diario, semanal y balance mensual)		
Importancia	**Importante**		
Urgencia	Alta		
Comentarios	La rentabilidad neta se calcula mediante la fórmula contable: Balance Neto = (Total Ventas Válidas + Total Mano de Obra Taller) - Total Pagos a Proveedores. **Relaciones:** <<extend>> CU21 (Filtra Reportes por Área y Fechas), <<extend>> CU22 (Exporta / Imprime Reporte).		

CU21: Filtra Reportes por Área y Fechas

CU21	**Filtra Reportes por Área y Fechas**
Descripción	Permite parametrizar el rango de fechas (Hoy, Esta Semana, Mes en Curso o Rango Personalizado) y filtrar el reporte por área operativa: Solo Salón de Ventas, Solo Taller Mecánico o Balance General Consolidado.
Actores	Dueño / Administrador
Pre condiciones	Acceso al módulo de Reportes con permisos administrativos.

Post condiciones	Los gráficos, tablas y métricas se recalculan dinámicamente para el rango temporal y área de negocio seleccionada.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador define la fecha de inicio y la fecha de corte en el selector de calendario.	El frontend valida que la fecha desde sea menor o igual a la fecha hasta.
	2	Selecciona el área deseada (ej. 'Módulo Taller' para ver rendimiento mecánico).	Envía petición GET a /api/reportes/reparaciones?desde=...&hasta=...
	3	El backend recalcula las estadísticas y devuelve la serie temporal acotada.	La pantalla actualiza al instante las curvas gráficas y los totales discriminados.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Rango de fechas inconsistente (fecha 'desde' posterior a 'hasta').	El sistema muestra alerta: 'La fecha de inicio no puede ser posterior a la fecha de finalización'.
Rendimiento	El recálculo de series temporales por área debe responder en menos de 250 ms.		
Frecuencia	Periódica (análisis de rendimientos específicos)		
Importancia	**Media**		
Urgencia	Puede esperar		
Comentarios	Las consultas temporales emplean índices en el campo fecha de cada tabla para responder en menos de 200 ms. **Relaciones:** <<extend>> CU20 (Registro de Reportes).		

CU22: Exporta / Imprime Reporte (PDF/Térmico)

CU22	**Exporta / Imprime Reporte (PDF/Térmico)**
Descripción	Permite exportar balances contables, listados de stock para toma de inventario físico, tickets de taller u órdenes de venta en formatos listos para imprimir o archivar digitalmente.
Actores	Dueño, Empleado

Pre condiciones	Datos visualizados en pantalla en módulos de Reportes, Ventas, Taller o Stock.		
Post condiciones	Se genera un archivo PDF formal estructurado o se remite el documento al servicio de impresión del sistema operativo.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El usuario pulsa el botón 'Exportar a PDF' o 'Imprimir' en la vista actual.	El sistema abre el diálogo modal de opciones de impresión.
	2	Elige el diseño deseado (Informe A4 corporativo o Ticket térmico 80 mm).	El frontend genera el render optimizado con estilos de impresión CSS @media print.
	3	Presiona 'Guardar' o 'Imprimir'.	Electron genera el PDF directamente en el directorio seleccionado por el usuario o envía los bytes a la impresora predeterminada.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Error de permisos de escritura en el directorio de destino.	El sistema informa: 'No se pudo guardar el archivo. Verifique los permisos de la carpeta seleccionada'.
Rendimiento	La generación del informe estructurado o ticket térmico debe completarse en menos de 500 ms.		
Frecuencia	Periódica (presentación contable o emisión física)		
Importancia	**Media**		
Urgencia	Puede esperar		
Comentarios	Los documentos PDF generados incluyen marca temporal de emisión, número de página y sello de identificación del establecimiento comercial. **Relaciones:** <<extend>> CU20 (Registro de Reportes), <<extend>> CU11 (Genera Comprobante), <<extend>> CU14 (Consulta Registro Reparación), <<extend>> CU17 (Registro Stock).		

CU23: Crear / Gestionar Usuario

CU23	Crear / Gestionar Usuario		
Descripción	Permite dar de alta a empleados y encargados que operarán el sistema de la bicicletería, definiendo credenciales de acceso seguras y datos de contacto.		
Actores	Dueño / Administrador		
Pre condiciones	Usuario autenticado con rol ADMIN.		
Post condiciones	El nuevo operador queda registrado en la tabla Usuario con su contraseña protegida mediante hash Bcrypt y rol asignado.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador ingresa al módulo 'Usuarios y Seguridad' y presiona 'Nuevo Usuario'.	Se abre el formulario modal de alta de usuario.
	2	Ingresa nombre, apellido, nombre de usuario (único) y contraseña inicial segura.	El sistema valida que la contraseña cumpla los requisitos mínimos de longitud (≥ 6 caracteres).
	3	Selecciona el rol que tendrá el operador (invoca CU24).	Asigna 'ADMIN' o 'EMPLEADO'.
	4	Presiona 'Crear Usuario'.	Envía una solicitud POST a /api/usuarios.
	5	El backend genera un salt con Bcrypt (10 rondas), calcula el hash y persiste el registro en PostgreSQL.	Retorna confirmación HTTP 201 Created y asienta el evento en la bitácora.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Nombre de usuario ya existente en el sistema.	El servidor responde con HTTP 400 Bad Request: 'El nombre de usuario ingresado ya está en uso'.
	2	Contraseña demasiado débil o corta.	El frontend alerta: 'La contraseña debe tener al menos 6 caracteres'.

Rendimiento	El cálculo del hash Bcrypt (10 salt rounds) y persistencia del operador debe ejecutarse en menos de 250 ms.
Frecuencia	Baja (al incorporar nuevo personal al establecimiento)
Importancia	**Importante**
Urgencia	Alta
Comentarios	Bajo ninguna circunstancia se almacenan contraseñas en texto plano; se utiliza exclusivamente hashing unidireccional con algoritmo Bcrypt. **Relaciones:** <<include>> CU24 (Asigna Permisos y Roles), <<extend>> CU26 (Editar / Anula Usuarios).

CU24: Asigna Permisos y Roles (RBAC)

CU24	**Asigna Permisos y Roles (RBAC)**		
Descripción	Asigna el perfil de seguridad al operador según el esquema RBAC (Role-Based Access Control): 'ADMIN' (acceso irrestricto) o 'EMPLEADO' (acceso operativo a Ventas, Taller y Stock; sin acceso a balances, anulación de ventas ni respaldos de base de datos).		
Actores	Dueño / Administrador		
Pre condiciones	Usuario en proceso de alta o edición en el sistema.		
Post condiciones	El rol queda guardado en la columna rol de la tabla Usuario, determinando los accesos a rutas y vistas.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador selecciona el perfil en el control desplegable del formulario de usuario.	El sistema muestra la descripción de los privilegios correspondientes al rol.
	2	Confirma la asignación de permisos.	El backend valida la consistencia del enumerado ('ADMIN' o 'EMPLEADO') antes de persistir.

	3	El rol queda asociado al usuario.	Se incluirá en el payload del token JWT al momento del inicio de sesión.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Intento de asignar un rol no reconocido.	El sistema rechaza la operación con error de validación de esquema.
Rendimiento	La validación y asignación del rol de seguridad debe responder de forma inmediata (< 100 ms).		
Frecuencia	Baja (configuración de perfiles de acceso)		
Importancia	**Importante**		
Urgencia	Alta		
Comentarios	El principio de privilegio mínimo se aplica de forma estricta: los empleados no tienen visibilidad de datos financieros ni auditoría inmutable. **Relaciones:** <<include>> de CU23 (Crear / Gestionar Usuario).		

CU 25: Inicia Sesión (Login)

CU 25	**Inicia Sesión (Login)**		
Descripción	Autentica al operador mediante su nombre de usuario y contraseña personal, verificando las credenciales contra la base de datos y otorgando acceso seguro a la aplicación de escritorio.		
Actores	Dueño, Empleado		
Pre condiciones	El usuario debe estar registrado y en estado activo en la base de datos.		
Post condiciones	Se genera un token JWT firmado, se almacena en el estado seguro del cliente y se desbloquea la interfaz del sistema.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador ingresa nombre de usuario y contraseña en la pantalla de inicio y presiona 'Ingresar'.	El cliente envía solicitud POST a /api/login con las credenciales.

	2	El backend invoca CU27 para buscar el usuario y cotejar el hash con bcrypt.compare().	Si es correcto, genera un token JWT firmado con expiración de 8 horas conteniendo ID, username y rol.
	3	El sistema asienta el evento 'LOGIN' en la tabla Bitacora_Actividad.	Retorna el token y datos del perfil al cliente.
	4	El frontend guarda la sesión en AuthContext y redirige al Dashboard operativo.	Habilita la barra de navegación lateral con las opciones correspondientes a su rol.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Campos incompletos o vacíos.	El formulario alerta: 'Por favor complete su usuario y contraseña'.
	2	Usuario incorrecto o contraseña inválida.	El sistema muestra mensaje neutral de seguridad: 'Credenciales inválidas' (sin revelar si falló el usuario o la clave).
	3	Usuario dado de baja (inactivo).	El sistema responde: 'Cuenta inactiva. Comuníquese con el administrador'.
Rendimiento	La comprobación de credenciales, emisión de token JWT y redirección al Dashboard debe tardar menos de 280 ms.		
Frecuencia	Alta (al comenzar el turno laboral)		
Importancia	**Vital**		
Urgencia	Inmediatamente		
Comentarios	El token JWT utiliza el algoritmo HMAC-SHA256 con firma secreta configurable y tiempo de vida limitado a 8 horas continuas. **Relaciones:** <<include>> CU27 (Valida Credenciales).		

CU26: Editar / Anula Usuarios

CU26	Editar / Anula Usuarios		
Descripción	Permite modificar datos de operadores (cambio de rol, restablecimiento de contraseña) o suspender el acceso de un empleado mediante baja lógica sin destruir los registros históricos de sus ventas.		
Actores	Dueño / Administrador		
Pre condiciones	El operador a gestionar debe existir en la base de datos. No se permite anular la cuenta propia activa.		
Post condiciones	Los datos del usuario se actualizan o su estado cambia a `activo = false` (baja lógica), revocando el acceso futuro al sistema.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador hace clic en la acción de usuario y selecciona 'Editar' o 'Desactivar'.	El sistema despliega el modal de edición o solicitud de confirmación de baja.
	2	Si es edición, modifica los campos requeridos (o ingresa una nueva contraseña para restablecer).	El sistema valida los datos actualizados.
	3	Si es baja lógica, confirma la desactivación del usuario.	Envía solicitud HTTP PUT a `/api/usuarios/:id` con `activo: false`.
	4	El backend persiste el cambio y asienta el evento en auditoría.	Actualiza el listado de usuarios y muestra la insignia 'Inactivo'.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	El administrador intenta desactivar su propia cuenta activa.	El sistema bloquea la acción informando: 'No puede darse de baja a sí mismo mientras tiene la sesión activa'.
Rendimiento	La aplicación de baja lógica (activo = false) debe registrarse en menos de 150 ms.		
Frecuencia	Baja (cambios de puesto o desvinculaciones)		
Importancia	**Media**		

Urgencia	Puede esperar
Comentarios	Los usuarios nunca se eliminan físicamente (DELETE) para preservar la integridad referencial de todas las ventas, reparaciones y asientos de bitácora previos. **Relaciones:** <<extend>> CU23 (Crear / Gestionar Usuario).

CU 27: Valida Credenciales (JWT)

CU 27	**Valida Credenciales (JWT)**		
Descripción	Mecanismo centralizado de seguridad que verifica la firma digital, validez y vigencia temporal de los tokens JWT transmitidos en las cabeceras HTTP de la aplicación.		
Actores	Sistema Backend (Autenticación)		
Pre condiciones	La petición HTTP incluye la cabecera Authorization: Bearer <token>.		
Post condiciones	El middleware autoriza la ejecución del controlador inyectando el objeto req.usuario o deniega el acceso con HTTP 401/403.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El middleware verificarToken intercepta la solicitud entrante.	Extrae el token del encabezado Authorization.
	2	Ejecuta jwt.verify() con la clave secreta del entorno.	Comprueba que la firma sea legítima y que el token no haya expirado.
	3	Adjunta el payload verificado (ID, username y rol) a req.usuario.	Transfiere el control al siguiente middleware o controlador de la ruta.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Token ausente en la cabecera.	Retorna de inmediato HTTP 401 Unauthorized: 'Acceso denegado: token no suministrado'.
	2	Token expirado o firma manipulada.	Retorna HTTP 403 Forbidden: 'Token inválido o

			expirado. Debe reiniciar sesión'.
Rendimiento	La verificación criptográfica del token en cabeceras HTTP debe realizarse en menos de 5 ms en memoria sin estado.		
Frecuencia	Continua (en cada petición HTTP entrante a la API)		
Importancia	**Vital**		
Urgencia	Inmediatamente		
Comentarios	La verificación de tokens es completamente sin estado (stateless), lo que maximiza la velocidad de respuesta del servidor local. **Relaciones:** <<include>> de CU25 (Inicia Sesión).		

CU28: Busca Registro de Actividades (Auditoría)

CU28	**Busca Registro de Actividades (Auditoría)**		
Descripción	Permite inspeccionar el registro forense de todas las operaciones críticas ejecutadas en el sistema (inicios de sesión, altas de clientes, ventas, anulaciones, ajustes de inventario y cambios de precio).		
Actores	Dueño / Administrador		
Pre condiciones	Usuario con rol ADMIN autenticado.		
Post condiciones	Se despliega la bitácora inmutable de eventos con opciones de filtrado cronológico y por operador.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador ingresa a 'Auditoría y Bitácora' en la barra lateral.	El frontend solicita GET /api/bitacora con parámetros de paginación.
	2	El backend consulta la tabla inmutable Bitacora_Actividad.	Retorna los registros con fecha exacta, nombre de usuario, acción realizada y detalle técnico.
	3	El operador filtra por rango de fechas o usuario específico.	La grilla se actualiza facilitando la trazabilidad de cualquier anomalía o error operativo.

Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Usuario no administrador intenta acceder al registro de auditoría.	El sistema rechaza la petición con código HTTP 403 Forbidden.
Rendimiento	La carga de la bitácora inmutable paginada debe responder en menos de 200 ms.		
Frecuencia	Periódica (revisión de seguridad y control forense)		
Importancia	**Importante**		
Urgencia	Alta		
Comentarios	La tabla Bitacora_Actividad es estrictamente append-only (solo inserciones); ninguna transacción ni usuario posee permisos para modificar o eliminar asientos de auditoría. **Relaciones:** <<extend>> CU36 (Descargar Respaldo de Base de Datos).		

CU29: Gestionar Garantía (90 días corridos)

CU29	**Gestionar Garantía (90 días corridos)**		
Descripción	Permite verificar el estado de la garantía oficial de un rodado ingresando su número de cuadro o DNI del cliente, calculando los días transcurridos desde la venta y habilitando la recepción de taller bonificada si está dentro del plazo legal.		
Actores	Dueño, Empleado		
Pre condiciones	La bicicleta fue comercializada en DN Bikes y se encuentra registrada en la base de datos.		
Post condiciones	El sistema determina la vigencia legal de la garantía (90 días corridos desde la fecha de factura) y habilita la cobertura de taller sin cargo.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador ingresa al módulo 'Garantías y Posventa' o pulsa 'Verificar Garantía' en el historial de ventas.	Se abre la herramienta de verificación de cobertura.

	2	Ingresa el número de serie de la bicicleta o selecciona la venta original.	El frontend consulta el padrón de garantías mediante GET /api/ventas/garantias y filtra por bicicleta o cliente.
	3	El backend busca la venta original y calcula la diferencia entre la fecha de compra y la fecha actual.	Determina si los días transcurridos son menores o iguales a 90 días corridos.
	4	El sistema despliega el diagnóstico de garantía: 'Garantía Vigente (Quedan X días)' o 'Garantía Vencida'.	Si está vigente, ofrece un botón directo para 'Abrir Orden de Garantía en Taller' con costo bonificado.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Bicicleta no encontrada o vendida fuera del sistema.	El sistema alerta: 'No se registra venta oficial en el sistema para este número de cuadro'.
	2	Garantía vencida (más de 90 días corridos).	Informa: 'El plazo legal de garantía de 90 días ha expirado. El servicio de taller deberá presupuestarse con tarifa regular'.
Rendimiento	El cálculo de vigencia temporal de 90 días corridos desde la fecha de factura debe realizarse en menos de 150 ms.		
Frecuencia	Media (ante reclamos técnicos de clientes post-venta)		
Importancia	**Importante**		
Urgencia	Alta		
Comentarios	El marco legal de garantía cubre 90 días corridos contra defectos de fabricación o ensamblado mecánico conforme a la legislación de protección al consumidor. **Relaciones:** <<extend>> CU10 (Consulta Registro de Ventas), <<include>> CU30 (Registrar Primer Service Gratuito).		

CU30: Registrar Primer Service Gratuito

CU30	Registrar Primer Service Gratuito		
Descripción	Permite aplicar el beneficio comercial de primer service gratuito de ajuste (calibración de cambios, centrado de ruedas y ajuste de tornillería) que se otorga con la compra de una bicicleta nueva en DN Bikes.		
Actores	Dueño, Empleado		
Pre condiciones	Bicicleta nueva adquirida en el local comercial con garantía activa y sin registro previo de uso del beneficio.		
Post condiciones	Se genera una orden de servicio preventivo en taller con costo de mano de obra 100% bonificado y se marca el beneficio como 'utilizado'.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El cliente acude al taller para el service preventivo de su bicicleta nueva.	El operador verifica la orden de venta original.
	2	El sistema comprueba que el beneficio no haya sido consumido con anterioridad.	Habilita la acción 'Aplicar Primer Service Gratuito'.
	3	El operador confirma la apertura de la orden técnica con el beneficio de garantía.	El frontend envía PATCH /api/ventas/garantias/:id_detalle_venta/primer-service; el backend asienta el primer service realizado y crea la orden en Reparacion con costo bonificado al 100%. .
	4	Se emite el comprobante de recepción para el cliente sin cargo.	El sistema actualiza el registro impidiendo la duplicación del beneficio.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	El beneficio del primer service ya fue utilizado previamente para esta bicicleta.	El sistema bloquea la bonificación indicando: 'El beneficio del primer service gratuito ya fue consumido en fecha DD/MM/AAAA'.

Rendimiento	La generación de la orden de ajuste con costo bonificado al 100% debe procesarse en menos de 200 ms.
Frecuencia	Media (beneficio de compra de bicicleta nueva)
Importancia	**Importante**
Urgencia	Alta
Comentarios	El primer service gratuito es válido una única vez por unidad comercializada durante los primeros 90 días desde la fecha de compra. **Relaciones:** <<include>> de CU29 (Gestionar Garantía).

CU31: Registrar Proveedor Comercial

CU31	**Registrar Proveedor Comercial**		
Descripción	Permite registrar proveedores mayoristas de bicicletas, componentes y repuestos (razón social, CUIT, teléfono, correo electrónico, dirección y persona de contacto).		
Actores	Dueño / Administrador		
Pre condiciones	Usuario con rol ADMIN autenticado en el sistema.		
Post condiciones	El nuevo distribuidor queda guardado en la tabla Proveedor con sus datos comerciales y fiscales.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador pulsa 'Nuevo Proveedor' en el módulo Proveedores.	Se abre el formulario modal de alta comercial.
	2	Completa la Razón Social, CUIT (con validación de formato tributario), Teléfono, Email y Dirección.	El formulario valida que los campos requeridos no se encuentren vacíos.
	3	Presiona 'Guardar Proveedor'.	El cliente envía solicitud POST a /api/proveedores.
	4	El backend verifica la unicidad del CUIT y persiste el registro en PostgreSQL.	Retorna confirmación HTTP 201 Created y refresca la grilla de proveedores.

Excepciones	#	Acción (actor)	Reacción (sistema)
	1	CUIT duplicado en la base de datos.	El sistema rechaza la operación informando: 'Ya existe un proveedor registrado con ese número de CUIT'.
	2	Formato de CUIT inválido.	El formulario alerta: 'El CUIT debe contener 11 dígitos numéricos válidos'.
Rendimiento	La verificación de formato CUIT y persistencia comercial debe completarse en menos de 180 ms.		
Frecuencia	Baja (al incorporar nuevas firmas proveedoras)		
Importancia	**Media**		
Urgencia	Puede esperar		
Comentarios	El CUIT es el identificador fiscal único para toda transacción comercial con proveedores en el territorio nacional. **Relaciones:** <<extend>> CU32 (Consultar Proveedores).		

CU32: Consultar Proveedores

CU32	**Consultar Proveedores**		
Descripción	Permite consultar el directorio de distribuidores mayoristas con buscador por nombre o CUIT, facilitando el control de cuentas corrientes y emisión de pagos.		
Actores	Dueño / Administrador		
Pre condiciones	Existen proveedores registrados en el sistema.		
Post condiciones	Se visualiza la grilla comercial de proveedores con saldos acumulados de compras y accesos directos a pagos.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El operador accede a 'Proveedores' en el menú de navegación.	El frontend solicita GET /api/proveedores.

	2	El backend consulta PostgreSQL y retorna el directorio ordenado por razón social.	La interfaz renderiza la tabla con datos de contacto y total acumulado de operaciones.
	3	El usuario puede buscar por nombre comercial o filtrar por estado activo.	La lista se actualiza dinámicamente permitiendo ver el historial o registrar un nuevo pago.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	No hay proveedores registrados.	Muestra aviso con botón de invitación para cargar el primer proveedor.
Rendimiento	La carga del directorio de proveedores y cuentas corrientes debe tardar menos de 180 ms.		
Frecuencia	Media (gestión de compras y logística)		
Importancia	**Media**		
Urgencia	Puede esperar		
Comentarios	Solo los usuarios con rol de Dueño o Administrador tienen visibilidad del módulo de Proveedores y sus balances asociados. **Relaciones:** <<extend>> CU31 (Registrar Proveedor Comercial), <<extend>> CU35 (Consultar Historial de Pagos a Proveedores).		

CU33: Registrar Pago a Proveedor

CU33	**Registrar Pago a Proveedor**
Descripción	Permite asentar egresos de dinero vinculados a compras de mercadería mayorista o insumos de taller, especificando monto abonado, medio de pago, fecha y número de factura cancelada.
Actores	Dueño / Administrador
Pre condiciones	El proveedor debe existir en el sistema. Se cuenta con el importe y comprobante a cancelar.
Post condiciones	Se registra el egreso en la tabla Pago_Proveedor, impactando negativamente en el balance financiero y asentando la auditoría.

Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador selecciona un proveedor en la grilla y presiona 'Registrar Pago'.	Se abre el modal de egresos comerciales.
	2	Ingresa el importe total abonado, fecha de la operación y número de factura/remito asociado.	El sistema valida que el importe sea un número positivo mayor a cero.
	3	Selecciona el método de pago utilizado (CU34: Efectivo, Transferencia Bancaria o Cheque).	Asocia la forma de pago al asiento contable.
	4	Presiona 'Confirmar Pago'.	El backend inserta el registro en Pago_Proveedor en transacción atómica.
	5	El sistema confirma el guardado.	Notifica: 'Pago a proveedor registrado exitosamente' y actualiza el saldo del proveedor.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Monto ingresado inválido o menor o igual a cero.	El formulario bloquea la confirmación indicando: 'El monto del pago debe ser mayor a cero'.
	2	Fallo de conexión con la base de datos.	El sistema aborta la transacción y muestra: 'Error al registrar el pago. Intente nuevamente'.
Rendimiento	El asiento de egreso y cómputo en el saldo del proveedor debe ejecutarse en menos de 200 ms.		
Frecuencia	Media (liquidación de facturas de mercadería)		
Importancia	**Importante**		
Urgencia	Alta		

Comentarios	Los pagos a proveedores se consolidan automáticamente como 'Egresos Comerciales' en el cálculo del Balance Neto del módulo Reportes (CU20). **Relaciones:** <<include>> CU34 (Seleccionar Método de Pago).

CU34: Seleccionar Método de Pago

CU34	**Seleccionar Método de Pago**		
Descripción	Componente transversal que permite seleccionar la modalidad financiera de la transacción: Efectivo, Transferencia Bancaria, Tarjeta de Débito o Tarjeta de Crédito, garantizando discriminación precisa en los arqueos de caja.		
Actores	Dueño, Empleado		
Pre condiciones	Operación de venta (CU09) o pago a proveedor (CU33) en proceso de confirmación.		
Post condiciones	El método seleccionado queda grabado en el comprobante fiscal y en el asiento correspondiente de la base de datos.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	Al confirmar una transacción, el sistema presenta las opciones de medios de pago disponibles.	El operador selecciona la modalidad correspondiente.
	2	Si es 'Efectivo', el sistema habilita la calculadora de vuelto para asistencia en caja.	Calcula el cambio exacto a entregar.
	3	Si es 'Transferencia' o 'Tarjeta', permite ingresar el número de referencia/cupón de pago.	Registra el identificador de la transacción electrónica.
	4	El método elegido se incluye en el payload de la petición transaccional.	Se persiste en la columna metodo_pago de la tabla correspondiente.
Excepciones	#	Acción (actor)	Reacción (sistema)

	1	No se selecciona ningún medio de pago.	El sistema exige seleccionar al menos una modalidad antes de permitir confirmar la operación.
Rendimiento	La imputación del medio de pago en el comprobante fiscal debe ser inmediata (< 50 ms).		
Frecuencia	Muy Alta (en cada cobro de mostrador o pago a proveedor)		
Importancia	**Importante**		
Urgencia	Inmediatamente		
Comentarios	Los ingresos por Efectivo y los ingresos bancarios (Transferencia/Tarjetas) se totalizan de forma discriminada en los reportes de cierre de caja diario. **Relaciones:** <<include>> de CU11 (Ventas) y <<include>> de CU33 (Pagos a Proveedores).		

CU35: Consultar Historial de Pagos a Proveedores

CU35	**Consultar Historial de Pagos a Proveedores**		
Descripción	Permite auditar el historial completo de pagos realizados a proveedores, comprobando fechas, montos, números de factura asociados y comprobantes de transferencia.		
Actores	Dueño / Administrador		
Pre condiciones	Existen pagos asentados en la base de datos.		
Post condiciones	Se presenta la grilla cronológica de egresos comerciales con totalizadores y filtros por proveedor y período.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)
	1	El Administrador accede a la pestaña 'Historial de Pagos' dentro del módulo Proveedores.	El frontend solicita GET /api/pagos-proveedores
	2	El backend consulta PostgreSQL ordenando los asientos por fecha decreciente.	Renderiza la grilla con Proveedor, CUIT, Fecha, Monto, Método de Pago y Factura.

	3	El operador puede filtrar por un distribuidor específico o exportar el listado a PDF (CU22).	Facilita la conciliación con los extractos bancarios.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	No se registran pagos en el período consultado.	Muestra notificación informativa: 'No se encontraron pagos registrados para el filtro seleccionado'.
Rendimiento	La carga cronológica de egresos y facturas comerciales debe responder en menos de 220 ms.		
Frecuencia	Periódica (auditoría contable y conciliación bancaria)		
Importancia	**Media**		
Urgencia	Puede esperar		
Comentarios	Los registros de pagos a proveedores no pueden ser eliminados; ante un error de imputación, se debe asentar un ajuste compensatorio. **Relaciones:** <<extend>> CU32 (Consultar Proveedores).		

CU 36: Descargar Respaldo de Base de Datos (SQL Dump)

CU 36	**Descargar Respaldo de Base de Datos (SQL Dump)**		
Descripción	Permite generar y descargar una copia de seguridad física completa de la base de datos relacional para resguardo ante contingencias de hardware, formateos de equipo o auditorías técnicas.		
Actores	Dueño / Administrador		
Pre condiciones	Usuario autenticado con permisos de máximo nivel (ADMIN). Motor PostgreSQL en ejecución.		
Post condiciones	Se genera un volcado íntegro de la estructura (DDL) y datos (DML) de todas las tablas en un archivo bikesystem_backup_YYYY-MM-DD.sql descargado al disco local.		
Secuencia Normal	#	Acción (actor)	Reacción (sistema)

	1	El Administrador ingresa a la sección 'Auditoría y Respaldos' en el panel de control.	El sistema despliega el botón de acción 'Descargar Respaldo de Base de Datos'.
	2	Presiona el botón 'Generar Respaldo SQL'.	El frontend solicita GET /api/backup/descargar
	3	El backend ejecuta una consulta de extracción completa de metadatos y datos de las 14 tablas relacionales.	Empaqueta las sentencias CREATE TABLE, INSERT INTO, índices y triggers en una cadena SQL formateada.
	4	El servidor envía el contenido con cabecera Content-Disposition: attachment; filename=bikesystem_backup_FECHA.sql.	Electron despliega el diálogo nativo de guardado de archivo en el explorador de Windows.
	5	El Administrador selecciona la carpeta de resguardo (disco externo, pendrive o carpeta segura).	El archivo se descarga íntegro en disco y se asienta el evento 'BACKUP_DOWNLOAD' en la bitácora inmutable.
Excepciones	#	Acción (actor)	Reacción (sistema)
	1	Usuario sin rol ADMIN intenta solicitar la descarga.	El middleware RBAC bloquea la petición con HTTP 403 Forbidden: 'Acceso no autorizado a respaldos de base de datos'.
	2	Fallo durante la lectura o streaming de PostgreSQL.	El servidor retorna HTTP 500 informando: 'Error al generar el respaldo de base de datos'.
Rendimiento	La exportación completa del DDL y datos de las 14 tablas en archivo .sql descargable debe completarse en menos de 500 ms.		
Frecuencia	Periódica (semanal para copias de seguridad de resguardo)		
Importancia	**Importante**		
Urgencia	Alta		

Comentarios	El volcado SQL generado es totalmente autocontenido y estándar, permitiendo la restauración completa del sistema en cualquier servidor PostgreSQL compatible mediante psql. **Relaciones:** <<extend>> CU28 (Busca Registro de Actividades / Auditoría).

Diagrama entidad-relación(E/R).

Figura 3: diagrama entidad relación.

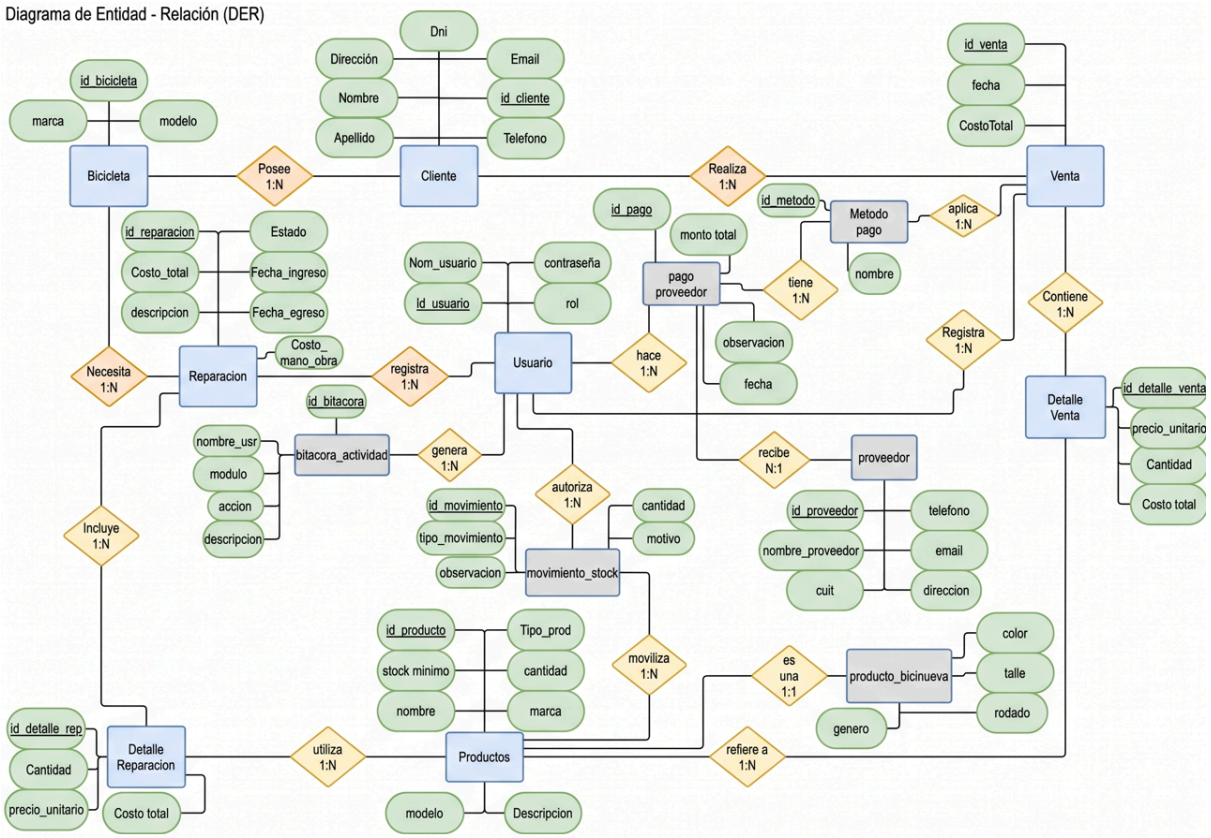
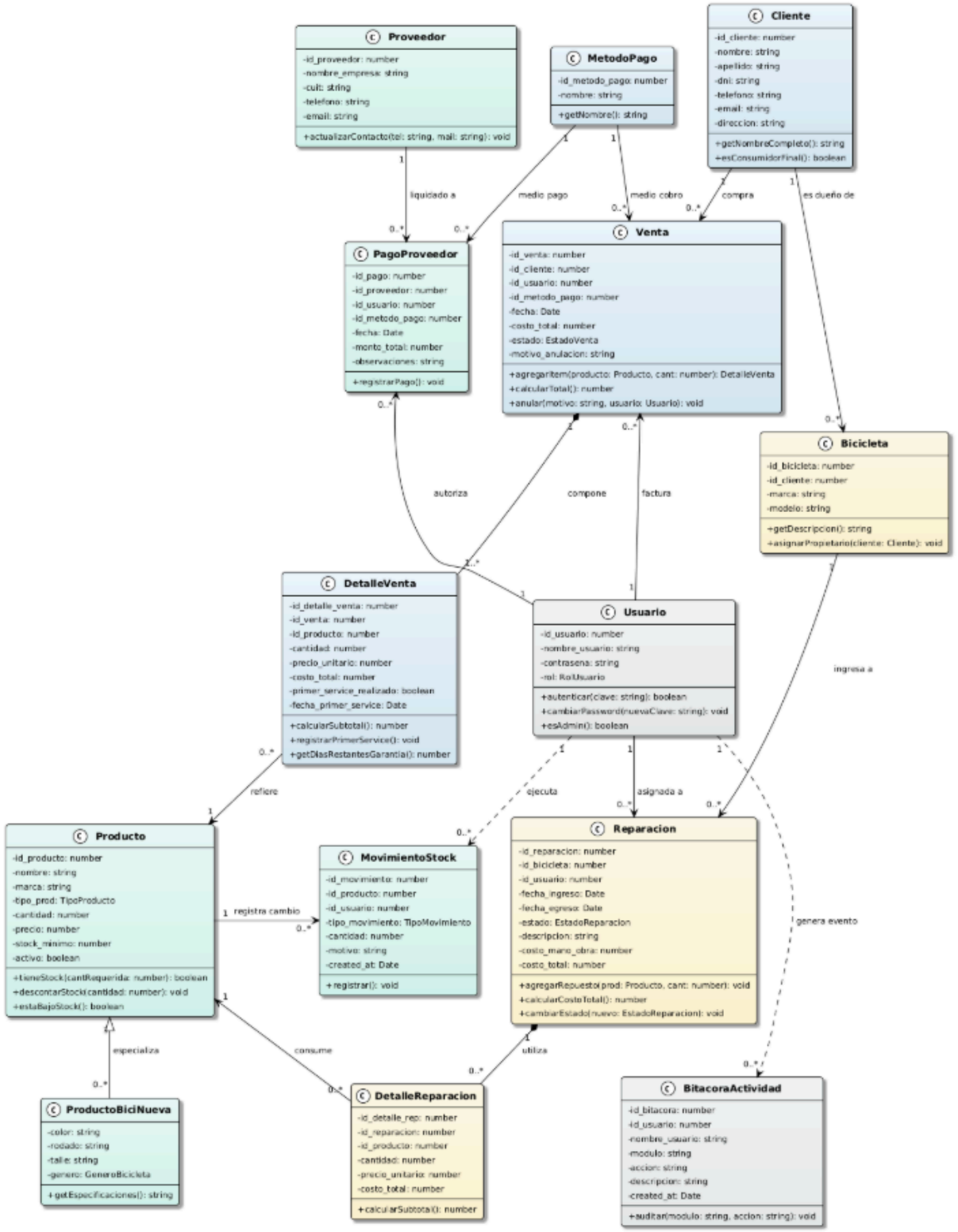


Diagrama de clases.

Figura 4: diagrama de clases.



Relaciones y Multiplicidades del Diagrama de Clases.

Clase Origen	Mult.	Tipo / Rol	Mult.	Clase Destino	Descripción y Regla de Negocio
Cliente	1	Asociación (*posee*)	0..*	**Bicicleta**	Un cliente puede registrar múltiples bicicletas para servicio técnico.
Cliente	1	Asociación (*compra*)	0..*	**Venta**	Un cliente puede efectuar múltiples compras en el salón comercial.
Bicicleta	1	Asociación (*ingresa*)	0..*	**Reparación**	Una bicicleta acumula un historial clínico de reparaciones.
Venta	1	Composición (♦ *compone*)	1..*	**Detalle_Venta**	Una venta comercial se compone obligatoriamente de uno o más ítems.
Reparación	1	Composición (♦ *utiliza*)	0..*	**Detalle_Reparacion**	Una reparación en taller puede consumir repuestos mecánicos.
Producto	1	Generalización (Δ *especializa*)	0..1	**Producto_BiciNueva**	Una bicicleta nueva hereda y extiende los atributos de un producto base.
Producto	1	Asociación (*referencia*)	0..*	**Movimiento_Stock**	Cada alteración física de producto genera un asiento de Kardex.

Proveedor	1	Asociación (*percibe*)	0..*	**Pago_Proveedor**	A un proveedor se le liquidan pagos vinculados a compras.

Stack Tecnológico

Para el desarrollo de la aplicación se utilizaron las siguientes tecnologías y herramientas:

- Entorno de Escritorio: Electron (versión 42). Empaqueta la aplicación como binario ejecutable de escritorio para Windows, permitiendo operar de manera autónoma en los puestos de trabajo.
- Interfaz de Usuario (UI): React 19 con TypeScript, empleando Tailwind CSS para estilos utilitarios y Lucide React para iconografía.
- Herramienta de Construcción: Vite (versión 6) con Hot Module Replacement (HMR) para un entorno de desarrollo ágil y empaquetado optimizado para distribución.
- Servidor API Backend: Express 5 sobre Node.js con TypeScript, ofreciendo enrutamiento modular y middlewares de intercepción.
- Base de Datos Relacional:PostgreSQL 16 alojada en la nube, con soporte de índices GIN Trigram (pg_trgm) para búsquedas fonéticas ultrarrápidas, transacciones ACID concurrentes y procedimientos almacenados para auditoría.
- Seguridad y Criptografía: Bcrypt para el hasheo con salt unidireccional de contraseñas y JSON Web Tokens (JWT) para la autenticación sin estado.
- Gestor de Paquetes y Monorepo: pnpm workspaces para la gestión unificada de dependencias del frontend y del backend en un único repositorio.

Modelo lógico de la base de datos.

A continuación se detalla formalmente la estructura lógica de las 14 tablas relacionales que integran la base de datos de BikeSystem en PostgreSQL. Para cada entidad se especifica su propósito funcional, el diccionario de datos con tipos y restricciones, y sus relaciones de cardinalidad explícitas:

1. Tabla: Usuario

Propósito: Almacena las cuentas de operadores autorizados del sistema, gestionando sus credenciales hasheadas y su nivel de privilegios mediante roles.

Campo	Tipo de Dato	Restricciones	Descripción
id_usuario	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador único correlativo del usuario.
nombre_usuario	VARCHAR(50)	NOT NULL, UNIQUE, CHECK (len ≥ 3)	Nombre de inicio de sesión único.
contraseña	VARCHAR(255)	NOT NULL	Hash criptográfico generado con Bcrypt.
rol	VARCHAR(20)	NOT NULL, CHECK (rol IN ('EMPLEADO','ADMIN','SUPERADMIN'))	Nivel de privilegios asignado.
created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Marcas de tiempo de auditoría de creación y modificación.

Relaciones Explícitas: 1 a N con Venta, Reparacion, Pago_Proveedor, Movimiento_Stock y Bitacora_Actividad.

2. Tabla: Cliente

Propósito: Almacena los datos personales, de contacto y de identificación de la cartera de clientes de DN Bikes.

Campo	Tipo de Dato	Restricciones	Descripción
id_cliente	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador único correlativo del cliente.
nombre	VARCHAR(100)	NOT NULL, CHECK (len ≥ 2)	Nombres del cliente.
apellido	VARCHAR(100)	NOT NULL, CHECK (len ≥ 2)	Apellidos del cliente.
dni	VARCHAR(20)	UNIQUE, CHECK (dni IS NULL OR len ≥ 6)	Documento Nacional de Identidad único (acepta nulo para consumidor final).
telefono	VARCHAR(20)	NULL	Número de teléfono o celular para contacto.
email	VARCHAR(100)	NULL	Correo electrónico de contacto.
direccion	VARCHAR(200)	NULL	Domicilio particular o comercial.
created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT	Auditoría de timestamps.

		CURRENT_TIMESTAMP	

Relaciones Explícitas: 1 a N con Bicicleta y 1 a N con Venta.

3. Tabla: Proveedor

Propósito: Directorio de empresas distribuidoras y fabricantes mayoristas de repuestos y bicicletas.

Campo	Tipo de Dato	Restricciones	Descripción
id_proveedor	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador único del proveedor comercial.
nombre_empresa	VARCHAR(100)	NOT NULL, CHECK (len ≥ 2)	Razón social o nombre comercial del mayorista.
cuit	VARCHAR(20)	UNIQUE, NULL	Clave Única de Identificación Tributaria.
telefono	VARCHAR(20)	NULL	Teléfono de pedidos comerciales.
email	VARCHAR(100)	NULL	Correo electrónico de compras.
direccion	VARCHAR(200)	NULL	Ubicación física o depósito central.

created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Auditoría de timestamps.

Relaciones Explícitas: 1 a N con Pago_Proveedor.

4. Tabla: Productos

Propósito: Catálogo unificado de artículos comerciales (bicicletas, repuestos mecánicos y accesorios de ciclismo).

Campo	Tipo de Dato	Restricciones	Descripción
id_producto	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador único del producto.
nombre	VARCHAR(100)	NOT NULL, CHECK (len ≥ 2)	Denominación comercial del producto.
marca	VARCHAR(50)	NULL	Marca fabricante (ej. Shimano, Venzo, Vairo).
modelo	VARCHAR(50)	NULL	Línea o modelo específico del artículo.
tipo_prod	VARCHAR(50)	NOT NULL, CHECK (IN ('bicicleta','repuesto','accesorio'))	Categoría clasificatoria del catálogo.
cantidad	INT	DEFAULT 0, CHECK (cantidad ≥ 0)	Stock físico actual en salón o taller.

precio	DECIMAL(10,2)	DEFAULT 0, CHECK (precio ≥ 0)	Precio de venta comercial unitario al público.
stock_minimo	INT	DEFAULT 0, CHECK (stock_minimo ≥ 0)	Umbral de seguridad para disparar alertas visuales.
activo	BOOLEAN	NOT NULL, DEFAULT true	Bandera de borrado lógico (soft-delete).
created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Auditoría de timestamps.

Relaciones Explícitas: 1 a 1 con Producto_BiciNueva; 1 a N con Detalle_Venta, Detalle_Reparacion y Movimiento_Stock.

5. Tabla: Producto_BiciNueva (Especialización)

Propósito: Extiende la tabla Productos cuando el artículo comercializado es una bicicleta nueva para salón de ventas.

Campo	Tipo de Dato	Restricciones	Descripción
id_producto	INT	PK, FK REFERENCES Productos(id_producto) ON DELETE CASCADE	Clave primaria compartida que vincula la especialización.
marca	VARCHAR(50)	NULL	Marca del rodado.
color	VARCHAR(30)	NULL	Acabado o color de la pintura.

rodado	VARCHAR(20)	NULL	Tamaño de rueda (ej. 29, 26, 20).
talle	VARCHAR(20)	NULL	Medida del cuadro (ej. S, M, L, 17", 19").
genero	VARCHAR(20)	NOT NULL, CHECK (IN ('hombre','mujer','unisex'))	Segmento de diseño ergonómico.

Relaciones Explícitas: Especialización 1 a 1 (Herencia en BD) con Productos.

6. Tabla: Metodo_Pago

Propósito: Catálogo normalizado de formas de pago aceptadas en el mostrador comercial y taller.

Campo	Tipo de Dato	Restricciones	Descripción
id_metodo_pago	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador único del medio de pago.
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre (Efectivo, Transferencia, Cheque, Mercado Pago).

Relaciones Explícitas: 1 a N con Venta y Pago_Proveedor.

7. Tabla: Bicicleta

Propósito: Registra las bicicletas particulares pertenecientes a clientes que ingresan para servicio técnico o reparación.

Campo	Tipo de Dato	Restricciones	Descripción

id_bicicleta	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador único de la bicicleta en el sistema.
id_cliente	INT	NOT NULL, FK REFERENCES Cliente(id_cliente) ON DELETE RESTRICT	Cliente titular propietario del rodado.
marca	VARCHAR(50)	NOT NULL, CHECK (len ≥ 1)	Marca del rodado.
modelo	VARCHAR(50)	NULL	Modelo o descripción técnica adicional.
created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Auditoría de timestamps.

Relaciones Explícitas: N a 1 con Cliente; 1 a N con Reparacion.

8. Tabla: Venta (Cabecera)

Propósito: Asienta la cabecera de las transacciones comerciales concretadas en el mostrador comercial.

Campo	Tipo de Dato	Restricciones	Descripción
id_venta	INT	PK, GENERATED ALWAYS AS IDENTITY	Número de comprobante interno de venta.

id_cliente	INT	NOT NULL, FK REFERENCES Cliente(id_cliente) ON DELETE RESTRICT	Cliente comprador (o Consumidor Final).
id_usuario	INT	NOT NULL, FK REFERENCES Usuario(id_usuario) ON DELETE RESTRICT	Vendedor u operador que procesó la venta.
id_metodo_pago	INT	NOT NULL, FK REFERENCES Metodo_Pago(id_metodo_pago) ON DELETE RESTRICT	Medio de pago utilizado.
fecha	DATE	NOT NULL, DEFAULT CURRENT_DATE	Fecha calendario de emisión de la venta.
costo_total	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (costo_total ≥ 0)	Importe final total liquidado al cliente.
estado	VARCHAR(20)	NOT NULL, DEFAULT 'COMPLETADA', CHECK (IN ('COMPLETADA','ANULADA'))	Estado de vigencia de la operación comercial.
fecha_anulacion	TIMESTAMPTZ	NULL	Momento exacto en que se anuló la venta (si aplica).

motivo_anulacion	TEXT	NULL	Justificación escrita obligatoria de la anulación.
cliente_nombre / cliente_apellido / cliente_dni	VARCHAR	NULL	Instantánea (snapshot) de los datos del cliente al facturar.
created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Auditoría de timestamps.

Relaciones Explícitas: N a 1 con Cliente, Usuario y Metodo_Pago; 1 a N con Detalle_Venta (ON DELETE CASCADE).

9. Tabla: Detalle_Venta (Líneas de Venta)

Propósito: Renglones individuales de artículos facturados en una venta, gestionando además la garantía post-venta de bicicletas nuevas.

Campo	Tipo de Dato	Restricciones	Descripción
id_detalle_venta	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador del ítem de venta.
id_venta	INT	NOT NULL, FK REFERENCES Venta(id_venta) ON DELETE CASCADE	Venta cabecera asociada.

id_producto	INT	NOT NULL, FK REFERENCES Productos(id_producto) ON DELETE RESTRICT	Artículo comercial facturado.
cantidad	INT	NOT NULL, CHECK (cantidad > 0)	Unidades vendidas.
precio_unitario	DECIMAL(10,2)	NOT NULL	Precio unitario fijado al momento de la venta.
costo_total	DECIMAL(10,2)	NOT NULL	Subtotal de la línea (cantidad × precio_unitario).
primer_service_realizado	BOOLEAN	NOT NULL, DEFAULT false	Indica si se redimió el 1° service oficial.
fecha_primer_service	DATE	NULL	Fecha en que se llevó a cabo el service de garantía.
observaciones_service	TEXT	NULL	Notas técnicas del mecánico sobre el service.
id_usuario_service	INT	NULL, FK REFERENCES Usuario(id_usuario)	Mecánico que ejecutó el service de garantía.

		ON DELETE SET NULL	
service_excepcion	BOOLEAN	NOT NULL, DEFAULT false	Bandera de excepción si se otorgó fuera de los 90 días por ADMIN.

Relaciones Explícitas: N a 1 con Venta, Productos y Usuario.

10. Tabla: Reparacion (Órdenes de Trabajo)

Propósito: Administra las órdenes de trabajo del taller mecánico de servicio técnico.

Campo	Tipo de Dato	Restricciones	Descripción
id_reparacion	INT	PK, GENERATED ALWAYS AS IDENTITY	Número de orden de trabajo de reparación.
id_bicicleta	INT	NOT NULL, FK REFERENCES Bicicleta(id_bicicleta) ON DELETE RESTRICT	Rodado ingresado al taller.
id_usuario	INT	NOT NULL, FK REFERENCES Usuario(id_usuario) ON DELETE RESTRICT	Mecánico responsable asignado.
fecha_ingreso	DATE	NOT NULL, DEFAULT CURRENT_DATE	Fecha en que el rodado entró al taller.

fecha_egreso	DATE	NULL	Fecha en que la bicicleta fue entregada al cliente.
estado	VARCHAR(50)	NOT NULL, CHECK (IN ('Recibida','En Reparación','Lista','Entregada'))	Columna activa del tablero Kanban.
descripcion	TEXT	NULL	Diagnóstico técnico inicial y tareas a realizar.
costo_mano_obra	DECIMAL(10,2)	DEFAULT 0, CHECK (costo_mano_obra ≥ 0)	Honorarios de trabajo del mecánico.
costo_total	DECIMAL(10,2)	DEFAULT 0, CHECK (costo_total ≥ 0)	Importe total (mano de obra + repuestos consumidos).
created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Auditoría de timestamps.

Relaciones Explícitas: N a 1 con Bicicleta y Usuario; 1 a N con Detalle_Reparacion.

11. Tabla: Detalle_Reparacion (Repuestos Consumidos)

Propósito: Detalla cada una de las piezas y repuestos mecánicos utilizados en una orden de servicio técnico.

Campo	Tipo de Dato	Restricciones	Descripción

id_detalle_rep	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador del repuesto consumido.
id_reparacion	INT	NOT NULL, FK REFERENCES Reparacion(id_reparacion) ON DELETE CASCADE	Orden de trabajo asociada.
id_producto	INT	NOT NULL, FK REFERENCES Productos(id_producto) ON DELETE RESTRICT	Repuesto debitado de stock.
cantidad	INT	NOT NULL, CHECK (cantidad > 0)	Unidades instaladas en la bicicleta.
precio_unitario	DECIMAL(10,2)	NOT NULL	Precio de la pieza liquidado al cliente.
costo_total	DECIMAL(10,2)	NOT NULL	Subtotal de repuestos (cantidad × precio_unitario).

Relaciones Explícitas: N a 1 con Reparacion y Productos.

12. Tabla: Pago_Proveedor

Propósito: Asienta las salidas de dinero por compras mayoristas de repuestos o insumos comerciales.

Campo	Tipo de Dato	Restricciones	Descripción

id_pago	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador del recibo de pago comercial.
id_proveedor	INT	NOT NULL, FK REFERENCES Proveedor(id_proveedor) ON DELETE RESTRICT	Proveedor beneficiario del pago.
id_usuario	INT	NOT NULL, FK REFERENCES Usuario(id_usuario) ON DELETE RESTRICT	Operador administrativo que autorizó el pago.
id_metodo_pago	INT	NOT NULL, FK REFERENCES Metodo_Pago(id_metodo_pago) ON DELETE RESTRICT	Forma de pago utilizada (Cheque, Transferencia, etc.).
fecha	DATE	NOT NULL, DEFAULT CURRENT_DATE	Fecha calendario de emisión del pago.
monto_total	DECIMAL(10,2)	NOT NULL, CHECK (monto_total > 0)	Suma monetaria total abonada.
observaciones	TEXT	NULL	Número de factura o concepto de la compra.

created_at / updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Auditoría de timestamps.

Relaciones Explícitas: N a 1 con Proveedor, Usuario y Metodo_Pago.

13. Tabla: Movimiento_Stock (Kardex Unificado)

Propósito: Garantiza la trazabilidad atómica e inmutable de cualquier alteración física en las cantidades de inventario.

Campo	Tipo de Dato	Restricciones	Descripción
id_movimiento	INT	PK, GENERATED ALWAYS AS IDENTITY	Número de asiento de Kardex.
id_producto	INT	NOT NULL, FK REFERENCES Productos(id_producto) ON DELETE RESTRICT	Artículo cuyo stock se modificó.
id_usuario	INT	NOT NULL, FK REFERENCES Usuario(id_usuario) ON DELETE RESTRICT	Operador que causó el movimiento.
tipo_movimiento	VARCHAR(20)	NOT NULL, CHECK (tipo_movimiento IN ('INGRESO','EGRESO'))	Dirección física del movimiento de mercadería.
cantidad	INT	NOT NULL, CHECK (cantidad > 0)	Volumen de unidades físicas movilizadas.
motivo	VARCHAR(100)	NOT NULL	Causa formal (VENTA, REPARACIÓN,

			COMPRA, AJUSTE_ROTURA).
observaciones	TEXT	NULL	Comentarios ampliatorios o número de comprobante.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Instante temporal inmutable del registro.

Relaciones Explícitas: N a 1 con Productos y Usuario.

14. Tabla: Bitacora_Actividad (Auditoría de Seguridad)

Propósito: Bitácora forense de seguridad que registra eventos sensibles y modificaciones críticas.

Campo	Tipo de Dato	Restricciones	Descripción
id_bitacora	INT	PK, GENERATED ALWAYS AS IDENTITY	Identificador del evento de auditoría.
id_usuario	INT	NULL, FK REFERENCES Usuario(id_usuario) ON DELETE SET NULL	Operador que ejecutó la acción (si aplica).
nombre_usuario	VARCHAR(50)	NOT NULL	Nombre de usuario inmutable registrado al momento del suceso.
modulo	VARCHAR(50)	NOT NULL	Módulo afectado (AUTH, VENTAS,

			STOCK, REPARACIONES).
accion	VARCHAR(100)	NOT NULL	Operación ejecutada (LOGIN, ANULACIÓN, ELIMINACIÓN).
descripcion	TEXT	NULL	Detalle pormenorizado de los datos modificados.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp exacto inmutable del evento.

Relaciones Explícitas: N a 1 con Usuario (ON DELETE SET NULL).

Modelo fisco de la base de datos(DDL).

Script ejecutado en PostgreSQL 16 para erigir la estructura formal de tablas, claves primarias identitarias, claves foráneas con reglas de integridad referencial y restricciones CHECK:

-- Habilitar extensión oficial de PostgreSQL para búsqueda por similitud y subcadenas masivas

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 1. TABLAS INDEPENDIENTES

CREATE TABLE Usuario (

id_usuario INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

```
nombre_usuario VARCHAR(50) NOT NULL UNIQUE CONSTRAINT chk_usuario_nombre_min CHECK (length(trim(nombre_usuario)) >= 3),

contrasena VARCHAR(255) NOT NULL,

rol VARCHAR(20) NOT NULL CONSTRAINT chk_usuario_rol CHECK (rol IN ('EMPLEADO', 'ADMIN', 'SUPERADMIN')),

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE Cliente (

id_cliente INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

nombre VARCHAR(100) NOT NULL CONSTRAINT chk_cliente_nombre_min CHECK (length(trim(nombre)) >= 2),

apellido VARCHAR(100) NOT NULL CONSTRAINT chk_cliente_apellido_min CHECK (length(trim(apellido)) >= 2),

dni VARCHAR(20) UNIQUE CONSTRAINT chk_cliente_dni_min CHECK (dni IS NULL OR length(trim(dni)) >= 6),

telefono VARCHAR(20),

email VARCHAR(100),

direccion VARCHAR(200),

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE Proveedor (

id_proveedor INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

nombre_empresa VARCHAR(100) NOT NULL CONSTRAINT chk_prov_nombre_min CHECK (length(trim(nombre_empresa)) >= 2),
```

```
cuit VARCHAR(20) UNIQUE,

telefono VARCHAR(20),

email VARCHAR(100),

direccion VARCHAR(200),

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE Productos (

id_producto INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

nombre VARCHAR(100) NOT NULL CONSTRAINT chk_prod_nombre_min CHECK (length(trim(nombre)) >= 2),

marca VARCHAR(50),

modelo VARCHAR(50) NULL,

tipo_prod VARCHAR(50) NOT NULL CONSTRAINT chk_prod_tipo_valido CHECK (LOWER(tipo_prod) IN ('bicicleta', 'repuesto', 'accesorio')),

cantidad INT DEFAULT 0 CONSTRAINT chk_productos_cantidad CHECK (cantidad >= 0),

precio DECIMAL(10, 2) DEFAULT 0 CONSTRAINT chk_productos_precio CHECK (precio >= 0),

stock_minimo INT DEFAULT 0 CONSTRAINT chk_productos_stock_minimo CHECK (stock_minimo >= 0),

activo BOOLEAN NOT NULL DEFAULT true,

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP

);

CREATE TABLE Producto_BiciNueva (
```

```
id_producto INT PRIMARY KEY,

marca VARCHAR(50) NULL,

color VARCHAR(30) NULL,

rodado VARCHAR(20) NULL,

talle VARCHAR(20) NULL,

genero VARCHAR(20) NOT NULL CONSTRAINT chk_bicinueva_genero CHECK (LOWER(genero) IN ('hombre', 'mujer', 'unisex')),

CONSTRAINT fk_bicinueva_prod FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE CASCADE

);

CREATE TABLE Metodo_Pago (

id_metodo_pago INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

nombre VARCHAR(50) NOT NULL UNIQUE

);

INSERT INTO Metodo_Pago (nombre) VALUES

('Efectivo'),

('Transferencia Bancaria'),

('Cheque'),

('Mercado Pago');

-- 2. TABLAS DEPENDIENTES

CREATE TABLE Bicicleta (

id_bicicleta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_cliente INT NOT NULL,

marca VARCHAR(50) NOT NULL CONSTRAINT chk_bici_marca_min CHECK (length(trim(marca)) >= 1),
```

```
modelo VARCHAR(50),

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_bici_cliente FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente) ON DELETE RESTRICT

);

-- 3. TABLAS TRANSACCIONALES (Cabeceras)

CREATE TABLE Venta (

id_venta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_cliente INT NOT NULL,

id_usuario INT NOT NULL,

id_metodo_pago INT NOT NULL DEFAULT 1,

fecha DATE NOT NULL DEFAULT CURRENT_DATE,

costo_total DECIMAL(10, 2) NOT NULL DEFAULT 0 CONSTRAINT chk_venta_costo_total CHECK (costo_total >= 0),

estado VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA' CONSTRAINT chk_venta_estado CHECK (estado IN ('COMPLETADA', 'ANULADA')),

fecha_anulacion TIMESTAMPTZ NULL,

motivo_anulacion TEXT NULL,

cliente_nombre VARCHAR(100) NULL,

cliente_apellido VARCHAR(100) NULL,

cliente_dni VARCHAR(20) NULL,

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
CONSTRAINT fk_venta_cliente FOREIGN KEY (id_cliente) REFERENCES Cliente(id_cliente) ON DELETE RESTRICT,

CONSTRAINT fk_venta_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE RESTRICT,

CONSTRAINT fk_venta_metodo FOREIGN KEY (id_metodo_pago) REFERENCES Metodo_Pago(id_metodo_pago) ON DELETE RESTRICT

);

CREATE TABLE Reparacion (

id_reparacion INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_bicicleta INT NOT NULL,

id_usuario INT NOT NULL,

fecha_ingreso DATE NOT NULL DEFAULT CURRENT_DATE,

fecha_egreso DATE,

estado VARCHAR(50) NOT NULL CONSTRAINT chk_reparacion_estado_valido CHECK (estado IN ('Recibida', 'En Reparación', 'Lista', 'Entregada')),

descripcion TEXT,

costo_mano_obra DECIMAL(10, 2) DEFAULT 0 CONSTRAINT chk_reparacion_mano_obra CHECK (costo_mano_obra >= 0),

costo_total DECIMAL(10, 2) DEFAULT 0 CONSTRAINT chk_reparacion_costo_total CHECK (costo_total >= 0),

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_rep_bicicleta FOREIGN KEY (id_bicicleta) REFERENCES Bicicleta(id_bicicleta) ON DELETE RESTRICT,

CONSTRAINT fk_rep_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE RESTRICT

);
```

```
CREATE TABLE Pago_Proveedor (

id_pago INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_proveedor INT NOT NULL,

id_usuario INT NOT NULL,

id_metodo_pago INT NOT NULL,

fecha DATE NOT NULL DEFAULT CURRENT_DATE,

monto_total DECIMAL(10, 2) NOT NULL CONSTRAINT chk_pago_monto_total CHECK (monto_total > 0),

observaciones TEXT,

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_pago_prov FOREIGN KEY (id_proveedor) REFERENCES Proveedor(id_proveedor) ON DELETE RESTRICT,

CONSTRAINT fk_pago_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE RESTRICT,

CONSTRAINT fk_pago_metodo FOREIGN KEY (id_metodo_pago) REFERENCES Metodo_Pago(id_metodo_pago) ON DELETE RESTRICT

);

-- 4. TABLAS TRANSACCIONALES (Detalles)

CREATE TABLE Detalle_Venta (

id_detalle_venta INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_venta INT NOT NULL,

id_producto INT NOT NULL,

cantidad INT NOT NULL CONSTRAINT chk_detventa_cantidad CHECK (cantidad > 0),

precio_unitario DECIMAL(10, 2) NOT NULL,
```

costo_total DECIMAL(10, 2) NOT NULL,

primer_service_realizado BOOLEAN NOT NULL DEFAULT false,

fecha_primer_service DATE NULL,

observaciones_service TEXT NULL,

id_usuario_service INT NULL,

service_excepcion BOOLEAN NOT NULL DEFAULT false,

CONSTRAINT fk_detventa_venta FOREIGN KEY (id_venta) REFERENCES Venta(id_venta) ON DELETE CASCADE,

CONSTRAINT fk_detventa_prod FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE RESTRICT,

CONSTRAINT fk_detventa_usuario_service FOREIGN KEY (id_usuario_service) REFERENCES Usuario(id_usuario) ON DELETE SET NULL

);

CREATE TABLE Detalle_Reparacion (

id_detalle_rep INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_reparacion INT NOT NULL,

id_producto INT NOT NULL,

cantidad INT NOT NULL CONSTRAINT chk_detrep_cantidad CHECK (cantidad > 0),

precio_unitario DECIMAL(10, 2) NOT NULL,

costo_total DECIMAL(10, 2) NOT NULL,

CONSTRAINT fk_detrep_rep FOREIGN KEY (id_reparacion) REFERENCES Reparacion(id_reparacion) ON DELETE CASCADE,

CONSTRAINT fk_detrep_prod FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE RESTRICT

);

CREATE TABLE Movimiento_Stock (

```
id_movimiento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_producto INT NOT NULL,

id_usuario INT NOT NULL,

tipo_movimiento VARCHAR(20) NOT NULL CONSTRAINT chk_mov_tipo CHECK (tipo_movimiento IN ('INGRESO', 'EGRESO')),

cantidad INT NOT NULL CONSTRAINT chk_mov_cant CHECK (cantidad > 0),

motivo VARCHAR(100) NOT NULL,

observaciones TEXT,

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_mov_prod FOREIGN KEY (id_producto) REFERENCES Productos(id_producto) ON DELETE RESTRICT,

CONSTRAINT fk_mov_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE RESTRICT

);

CREATE TABLE Bitacora_Actividad (

id_bitacora INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

id_usuario INT NULL,

nombre_usuario VARCHAR(50) NOT NULL,

modulo VARCHAR(50) NOT NULL,

accion VARCHAR(100) NOT NULL,

descripcion TEXT,

created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_bitacora_usuario FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE SET NULL

);
```

```
-- 5. FUNCIÓN TRIGGER DE AUDITORÍA


CREATE OR REPLACE FUNCTION actualizar_timestamp_modificacion()

RETURNS TRIGGER

LANGUAGE plpgsql

SET search_path = public

AS $$

BEGIN

NEW.updated_at = CURRENT_TIMESTAMP;

RETURN NEW;

END;

$$;


CREATE TRIGGER trg_productos_updated_at BEFORE UPDATE ON Productos FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();

CREATE TRIGGER trg_cliente_updated_at BEFORE UPDATE ON Cliente FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();

CREATE TRIGGER trg_venta_updated_at BEFORE UPDATE ON Venta FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();

CREATE TRIGGER trg_reparacion_updated_at BEFORE UPDATE ON Reparacion FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();

CREATE TRIGGER trg_pago_updated_at BEFORE UPDATE ON Pago_Proveedor FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();

CREATE TRIGGER trg_usuario_updated_at BEFORE UPDATE ON Usuario FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();
```

```
CREATE TRIGGER trg_bici_updated_at BEFORE UPDATE ON Bicicleta FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();

CREATE TRIGGER trg_prov_updated_at BEFORE UPDATE ON Proveedor FOR EACH ROW EXECUTE FUNCTION actualizar_timestamp_modificacion();


-- 6. ÍNDICES DE RENDIMIENTO (B-Tree)


-- Índices en Claves Foráneas (Acelera búsquedas relacionadas y JOINs)

CREATE INDEX idx_detalle_venta_id_venta ON Detalle_Venta(id_venta);

CREATE INDEX idx_detalle_venta_id_producto ON Detalle_Venta(id_producto);

CREATE INDEX idx_detalle_rep_id_reparacion ON Detalle_Reparacion(id_reparacion);

CREATE INDEX idx_detalle_rep_id_producto ON Detalle_Reparacion(id_producto);

CREATE INDEX idx_bicicleta_id_cliente ON Bicicleta(id_cliente);

CREATE INDEX idx_venta_id_cliente ON Venta(id_cliente);

CREATE INDEX idx_venta_id_usuario ON Venta(id_usuario);

CREATE INDEX idx_reparacion_id_bicicleta ON Reparacion(id_bicicleta);

CREATE INDEX idx_reparacion_id_usuario ON Reparacion(id_usuario);

CREATE INDEX idx_pago_prov_id_proveedor ON Pago_Proveedor(id_proveedor);

CREATE INDEX idx_pago_prov_id_usuario ON Pago_Proveedor(id_usuario);

CREATE INDEX idx_pago_prov_id_metodo ON Pago_Proveedor(id_metodo_pago);

CREATE INDEX idx_venta_id_metodo ON Venta(id_metodo_pago);

CREATE INDEX idx_bitacora_id_usuario ON Bitacora_Actividad(id_usuario);


-- Índices en Fechas y Estados (Acelera generación de Dashboard y Reportes)

CREATE INDEX idx_venta_fecha ON Venta(fecha);
```

```
CREATE INDEX idx_venta_estado ON Venta(estado);

CREATE INDEX idx_venta_fecha_activas ON Venta(fecha DESC) WHERE estado != 'ANULADA';

CREATE INDEX idx_pago_prov_fecha ON Pago_Proveedor(fecha DESC);

CREATE INDEX idx_mov_producto ON Movimiento_Stock(id_producto);

CREATE INDEX idx_bitacora_fecha ON Bitacora_Actividad(created_at DESC);

CREATE INDEX idx_bitacora_modulo ON Bitacora_Actividad(modulo);

CREATE INDEX idx_reparacion_estado ON Reparacion(estado);

CREATE INDEX idx_reparacion_fechas ON Reparacion(fecha_ingreso, fecha_egreso);

CREATE INDEX idx_reparacion_entregadas_fecha ON Reparacion(fecha_egreso DESC) WHERE estado = 'Entregada';

CREATE INDEX idx_bicicleta_marca_modelo ON Bicicleta(marca, modelo);


-- Índice Parcial (Acelera catálogo visible en stock/ventas ocupando mínima RAM)

CREATE INDEX idx_productos_activos ON Productos(tipo_prod) WHERE activo = true;


-- 7. ÍNDICES GIN TRIGRAM (Búsqueda Masiva Instantánea con pg_trgm)


CREATE INDEX idx_productos_nombre_trgm ON Productos USING gin (nombre gin_trgm_ops);

CREATE INDEX idx_productos_marca_trgm ON Productos USING gin (marca gin_trgm_ops);

CREATE INDEX idx_productos_modelo_trgm ON Productos USING gin (modelo gin_trgm_ops);

CREATE INDEX idx_cliente_busqueda_trgm ON Cliente USING gin ((nombre || ' ' || apellido) gin_trgm_ops);
```

CREATE INDEX idx_cliente_dni_trgm ON Cliente USING gin (dni gin_trgm_ops);

CREATE INDEX idx_proveedor_nombre_trgm ON Proveedor USING gin (nombre_empresa gin_trgm_ops);

Arquitectura del sistema.

El sistema de gestión BikeSystem fue diseñado siguiendo una Arquitectura de Escritorio Conectada con Persistencia Centralizada en la Nube (Cloud-Backed Desktop Architecture). El sistema desacopla estrictamente la interfaz de usuario de escritorio en React/Electrón, el servidor de API y servicios en Express, y el motor de base de datos relacional PostgreSQL alojado en la nube. Este enfoque garantiza que las reglas de negocio y transacciones críticas sean independientes de los componentes visuales, gocen de alta mantenibilidad y permitan la concurrencia multi-terminal en tiempo real entre los puestos de venta y el taller mecánico.

La centralización de la base de datos en la nube proporciona sincronización instantánea del stock y las órdenes de trabajo entre múltiples terminales, resguardo integral de la información comercial frente a fallas de hardware en las terminales físicas, y ejecución automática de respaldos en la nube, manteniendo al mismo tiempo la agilidad, fluidez y ergonomía de una aplicación de escritorio nativa para el operador de mostrador y los mecánicos

La arquitectura del sistema se divide en tres capas principales:

- Capa de Presentación (Presentation): Interfaz de usuario interactiva construida con React 19 y TypeScript, estilizada con Tailwind CSS y ejecutada dentro del proceso de renderizado de Electrón. Captura las acciones del usuario, gestiona el estado visual y se comunica con el backend mediante llamadas HTTP asíncronas.
- Capa de Backend y Servicios (Service / API): Desarrollada en Node.js con Express 5 y TypeScript. Expone una API RESTful local que centraliza las validaciones de negocio, la seguridad basada en roles (RBAC) y la orquestación de transacciones atómicas.

- Capa de Persistencia y Datos (Data): Motor de base de datos relacional PostgreSQL 16 alojado en infraestructura cloud, con pool de conexiones gestionado por el driver nativo pg (pg.Pool). Garantiza persistencia transaccional ACID, integridad referencial mediante claves foráneas y búsquedas predictivas mediante índices trigram (pg_trgm).

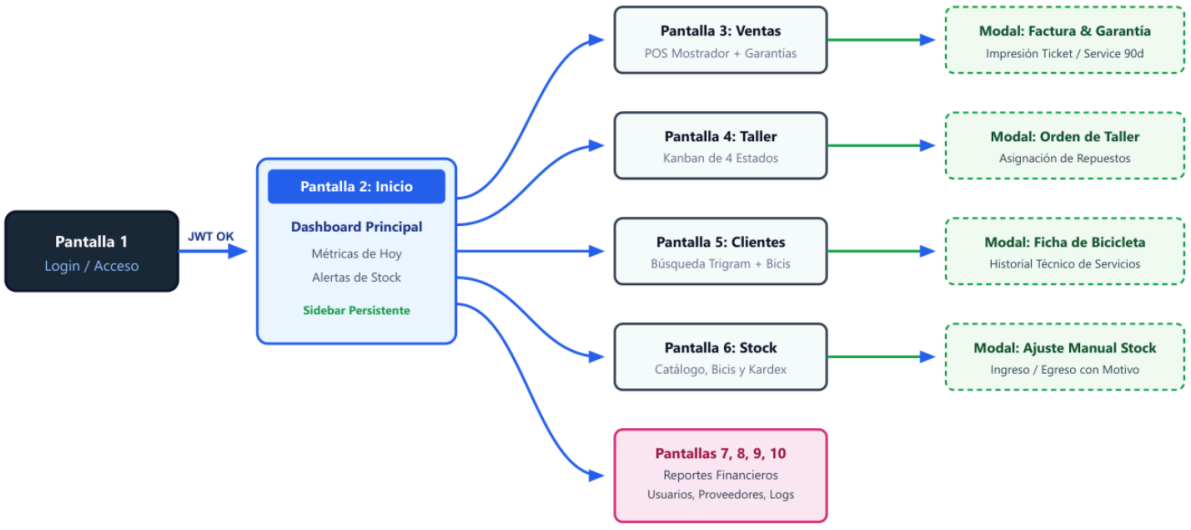
Diseño y prototipo

Interfaces

El diseño de interfaces se orientó a brindar una experiencia de usuario clara, eficiente y optimizada para la atención ágil en el mostrador y el registro de trabajo en el taller mecánico. La aplicación se estructura a partir de un flujo de navegación intuitivo que organiza las funcionalidades en módulos claramente diferenciados.

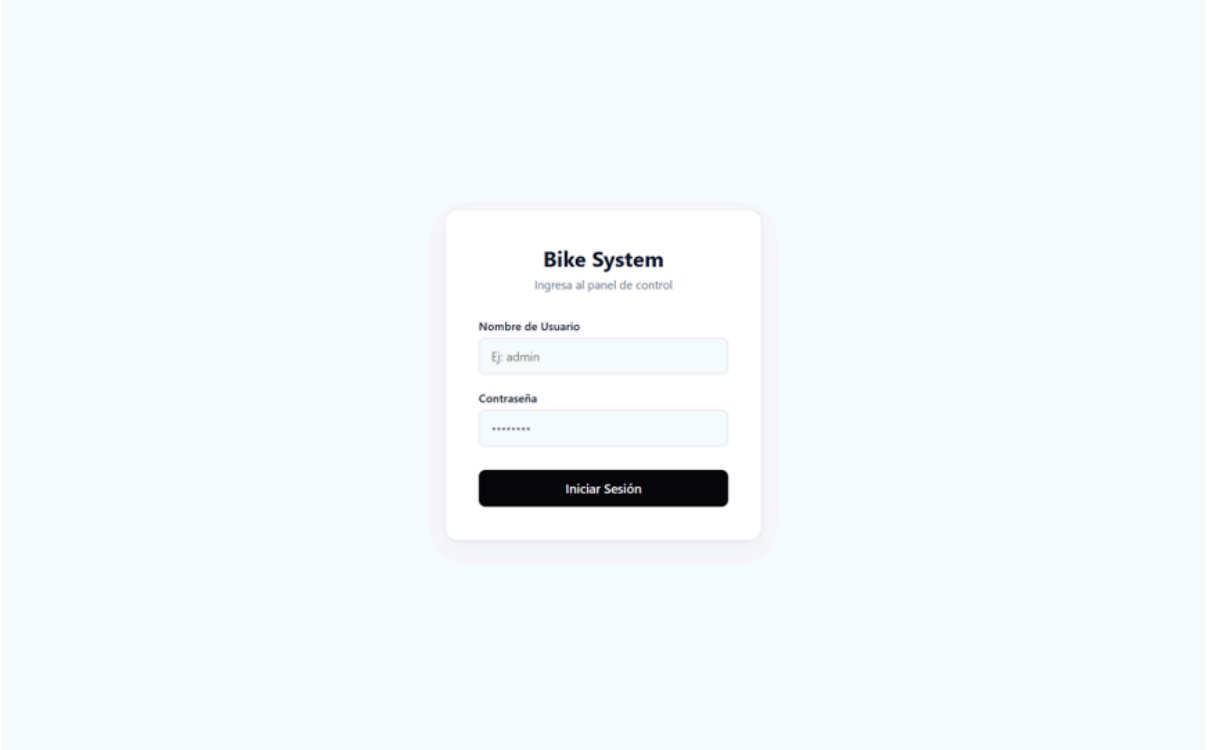
Flujo de navegación entre pantallas

Figura 5: flujo de navegación entre pantallas



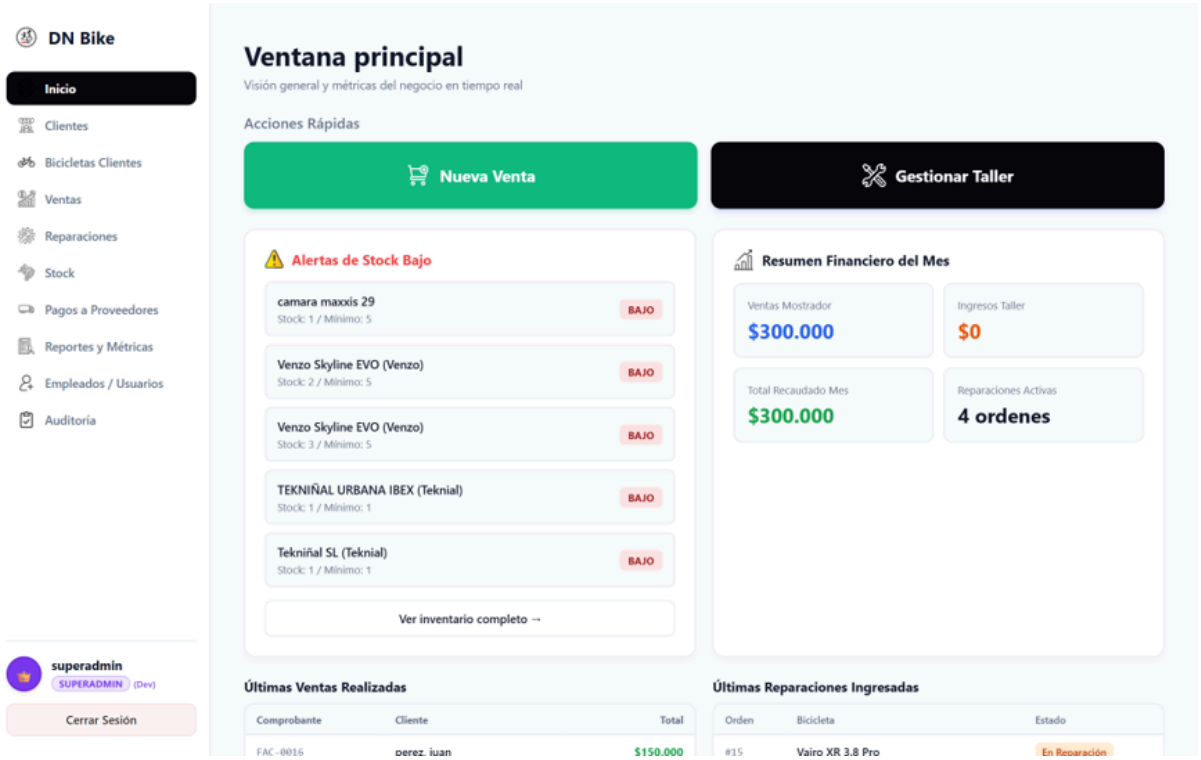
A continuación se presentan las pantallas de bike system:

Figura 6: pantalla de inicio de sesión



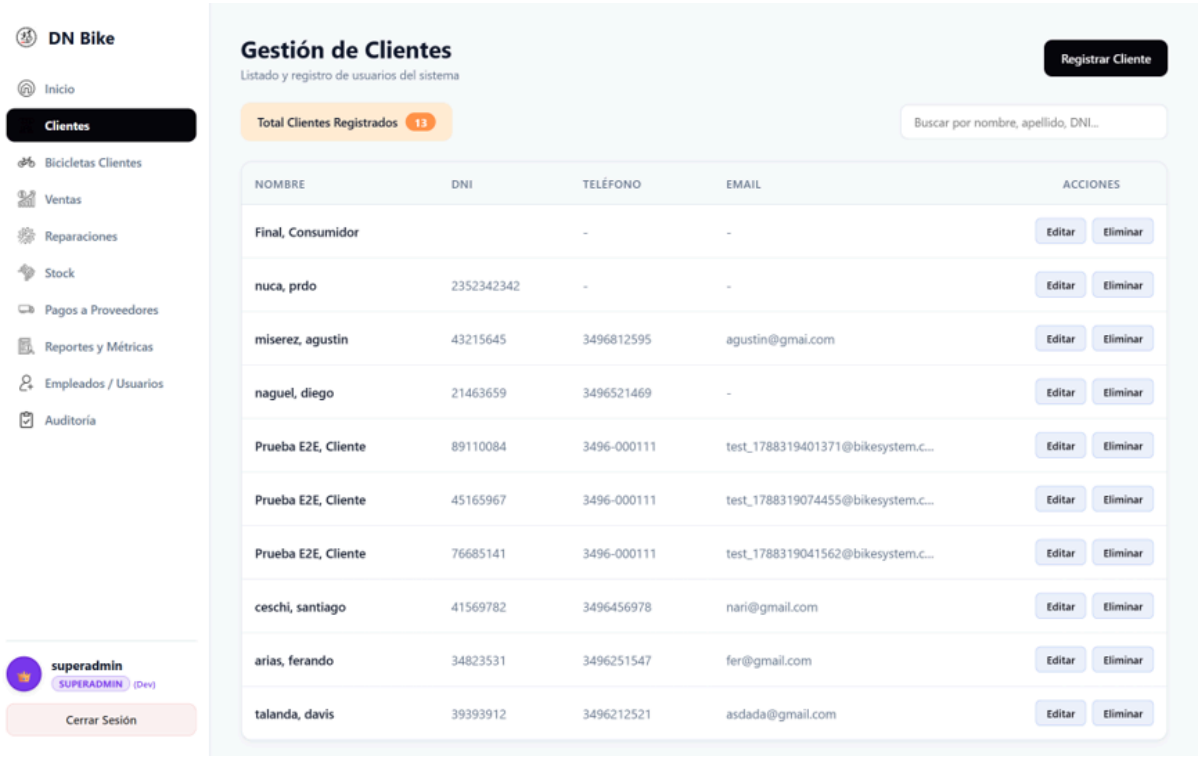
Nota: Pantalla de Inicio de Sesión permite el acceso seguro al sistema solicitando usuario y contraseña. Implementa autenticación con tokens JWT y verificación de contraseñas hasheadas mediante Bcrypt. Incluye mensajes de validación ante credenciales incorrectas.

Figura 7: Ventana principal de operaciones



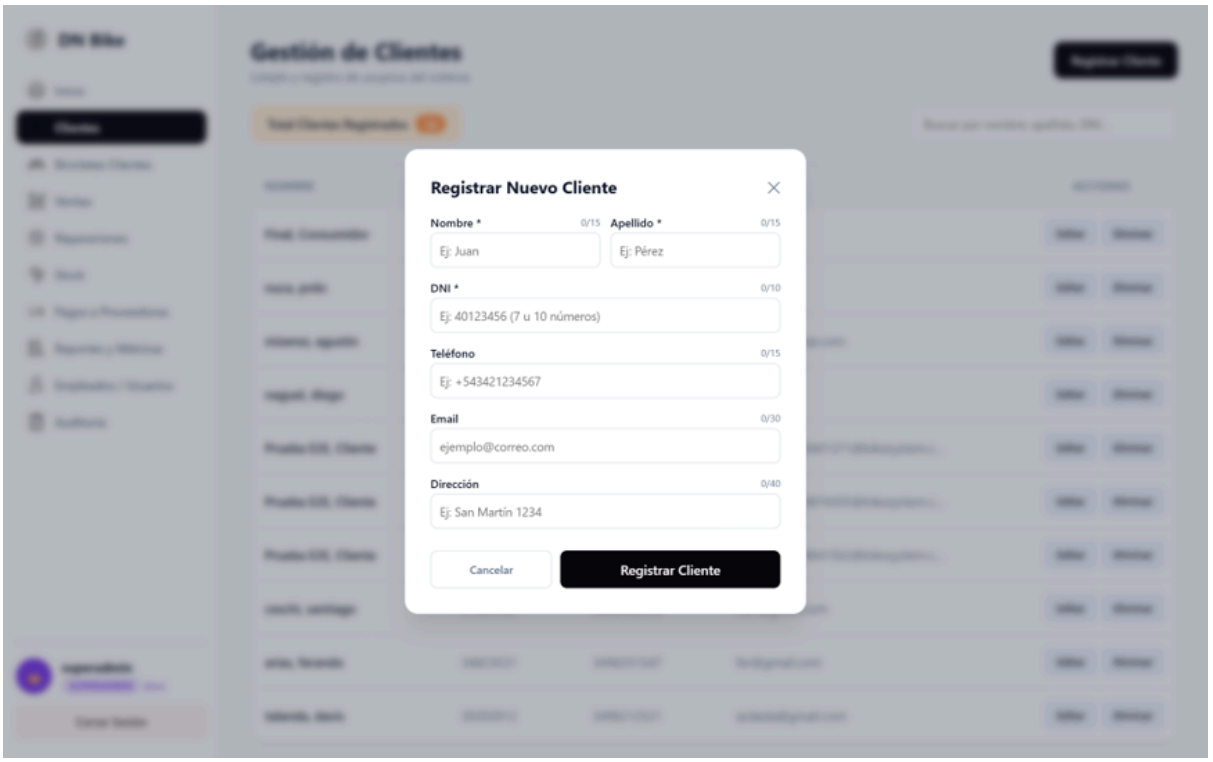
Nota: Ventana principal donde está panel de control central que brinda métricas clave del día: recaudación total, cantidad de reparaciones activas, alertas de stock mínimo y accesos directos a los principales módulos del sistema.

Figura 8: Directorio y gestor de clientes



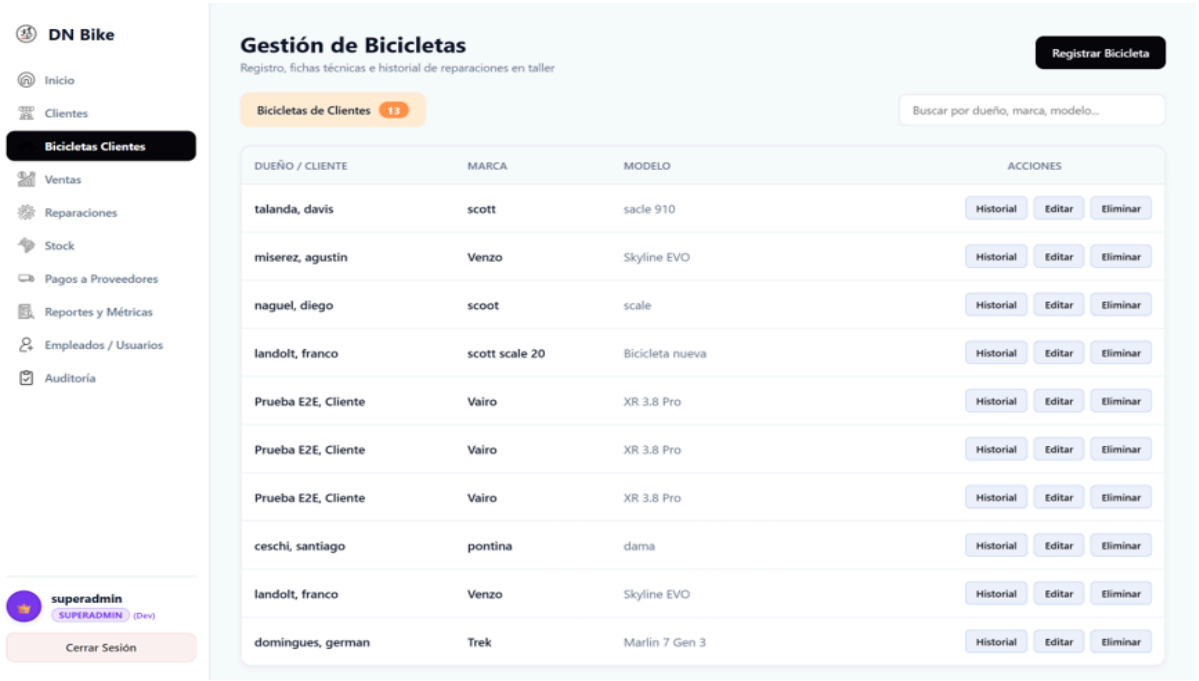
Nota: Gestión de Clientes: Listado completo de la cartera de clientes con buscador en tiempo real por nombre o DNI. Permite visualizar datos de contacto, registrar nuevos clientes y acceder directamente a sus bicicletas vinculadas.

Figura 9: Formulario de alta y edición de clientes.



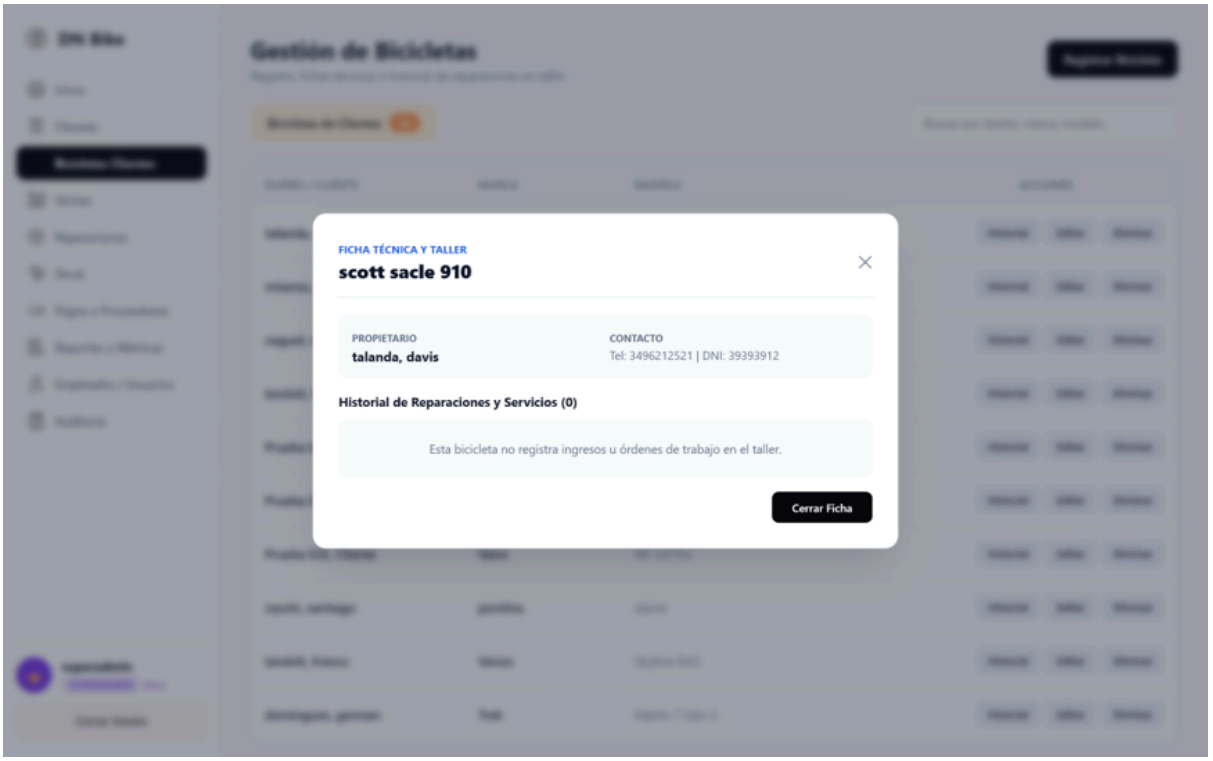
Nota: Ventana modal para la carga y actualización de datos personales (nombre, apellido, DNI, teléfono, correo electrónico y dirección particular) con validaciones en los campos obligatorios.

Figura 10: Bicicletas asociadas a su cliente.



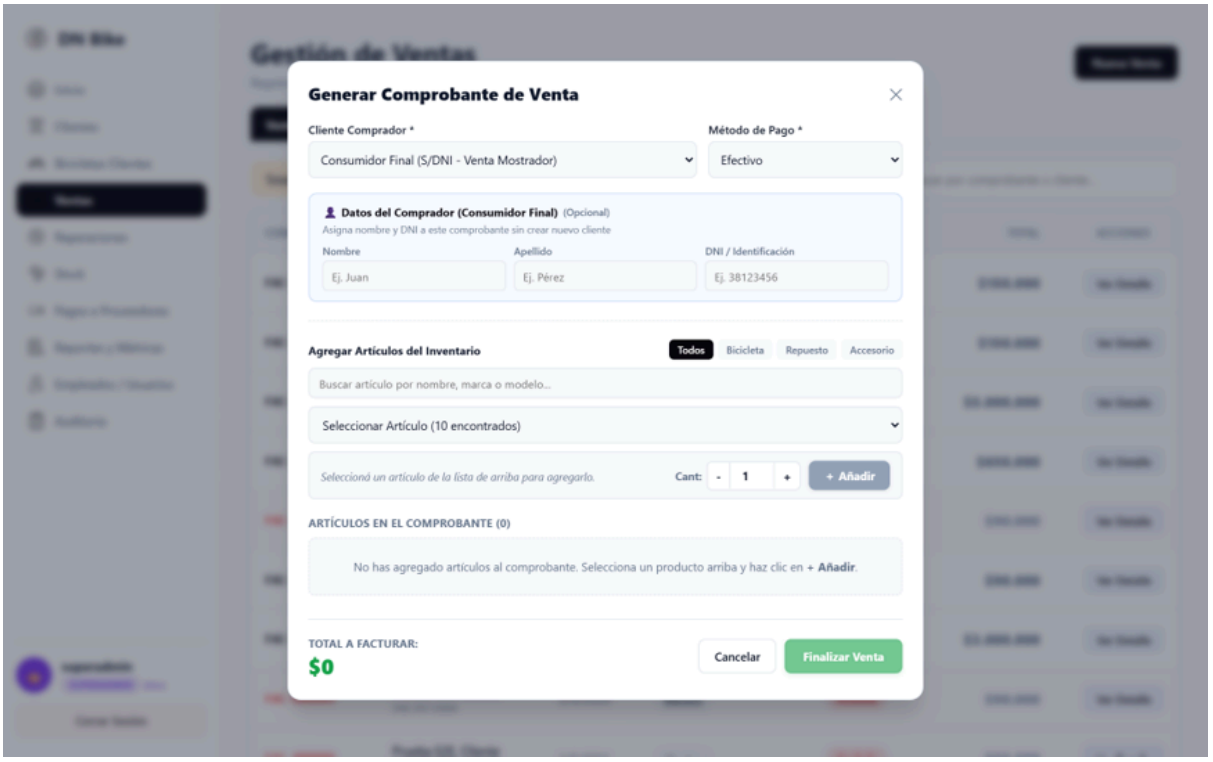
Nota: Administra las bicicletas registradas en el sistema vinculadas a sus respectivos dueños, consignando marca, modelo, rodado, color y historial de reparaciones en el taller.

Figura 11: Historial de servicios de la bicicleta



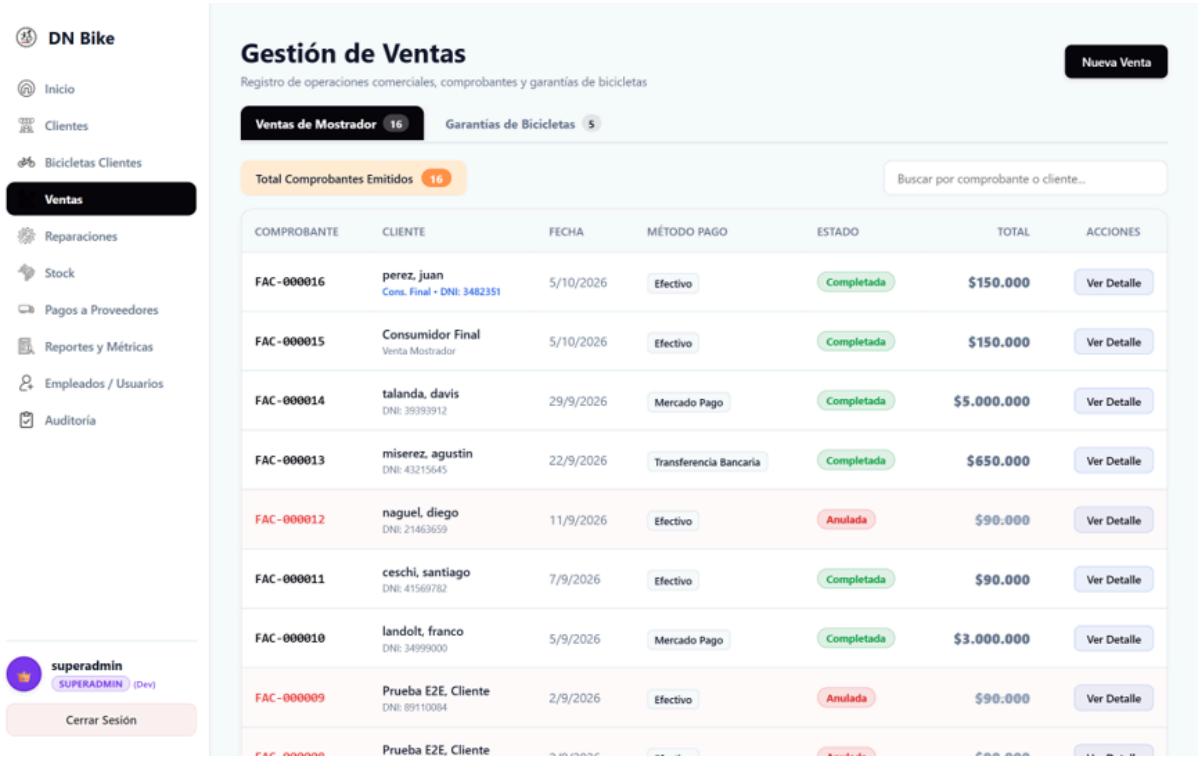
Nota: Historial de Servicios línea temporal interactiva que detalla todas las reparaciones, servicios de mantenimiento y cambios de piezas efectuados sobre un rodado específico, permitiendo un seguimiento técnico continuo.

Figura 12: Punto de venta.



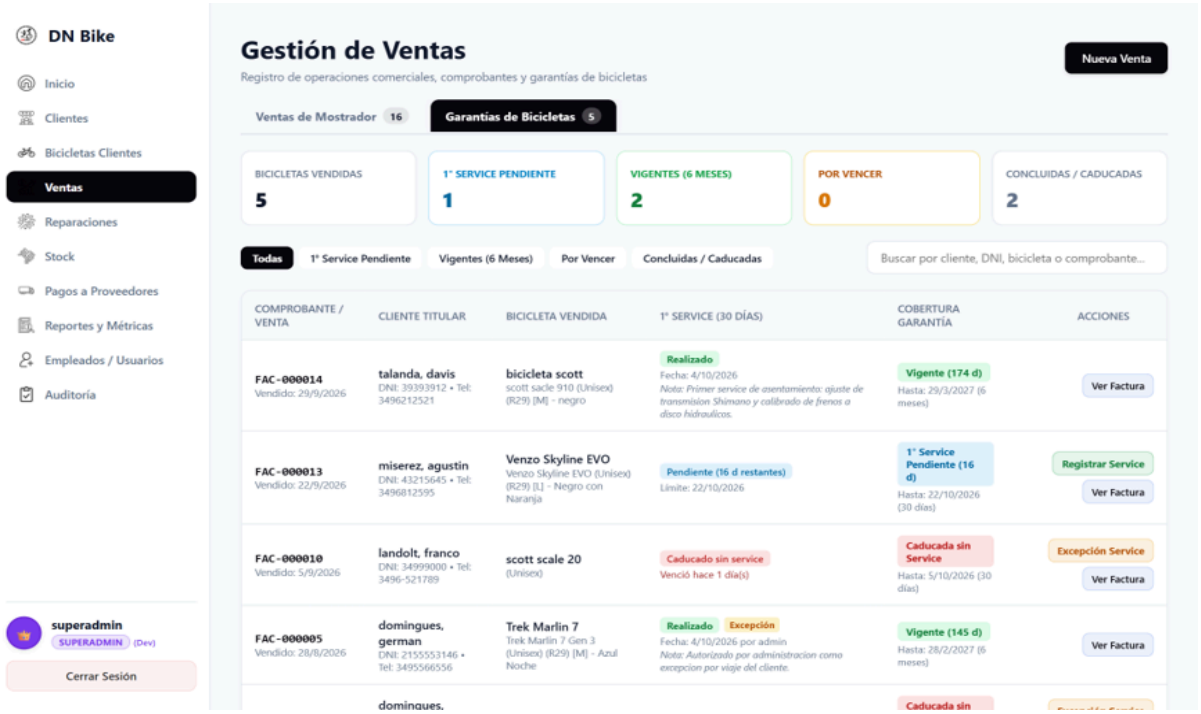
Nota: Interfaz ágil para la facturación en mostrador con buscador predictivo de repuestos y bicicletas, cálculo automático de importes, selección de cliente y múltiples métodos de pago (efectivo, transferencia, tarjeta).

Figura 13: Historial de ventas y comprobantes emitidos.



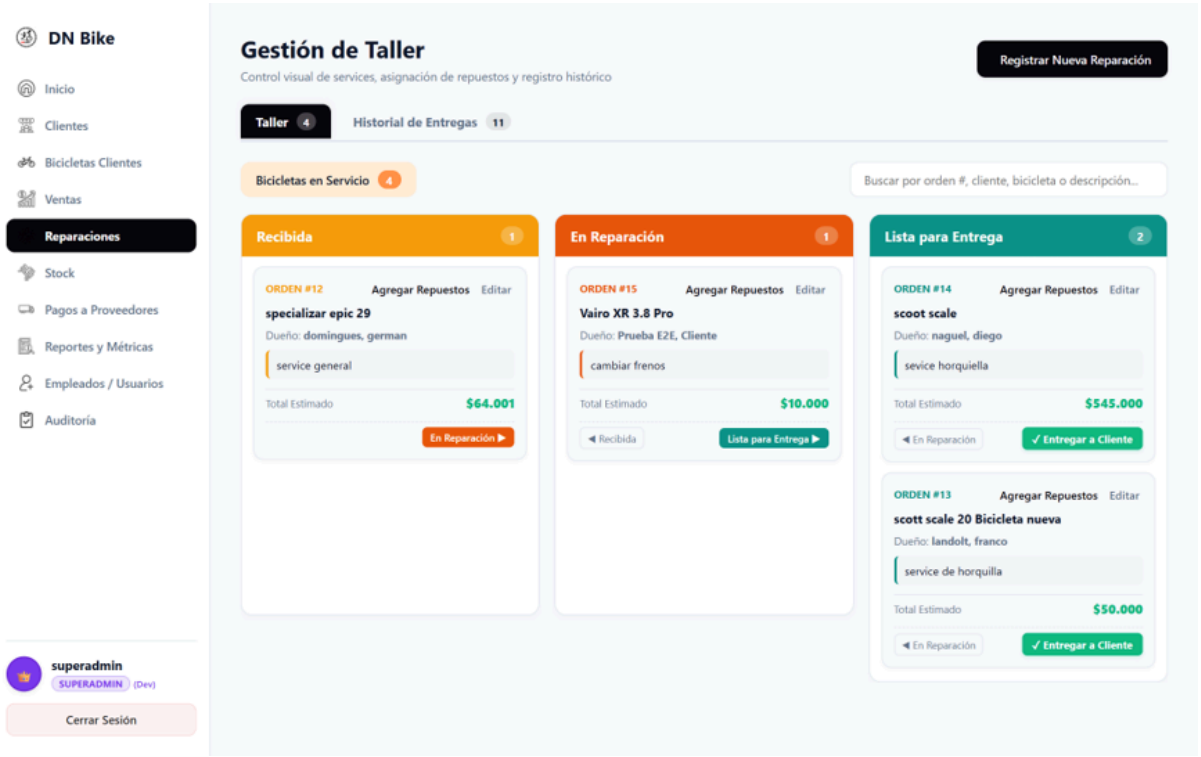
*Nota:*Registro cronológico de operaciones comerciales realizadas, con filtros por fecha, estado de pago y opción de reimpresión de comprobantes en PDF.

Figura 14: Gestión de garantías y services.



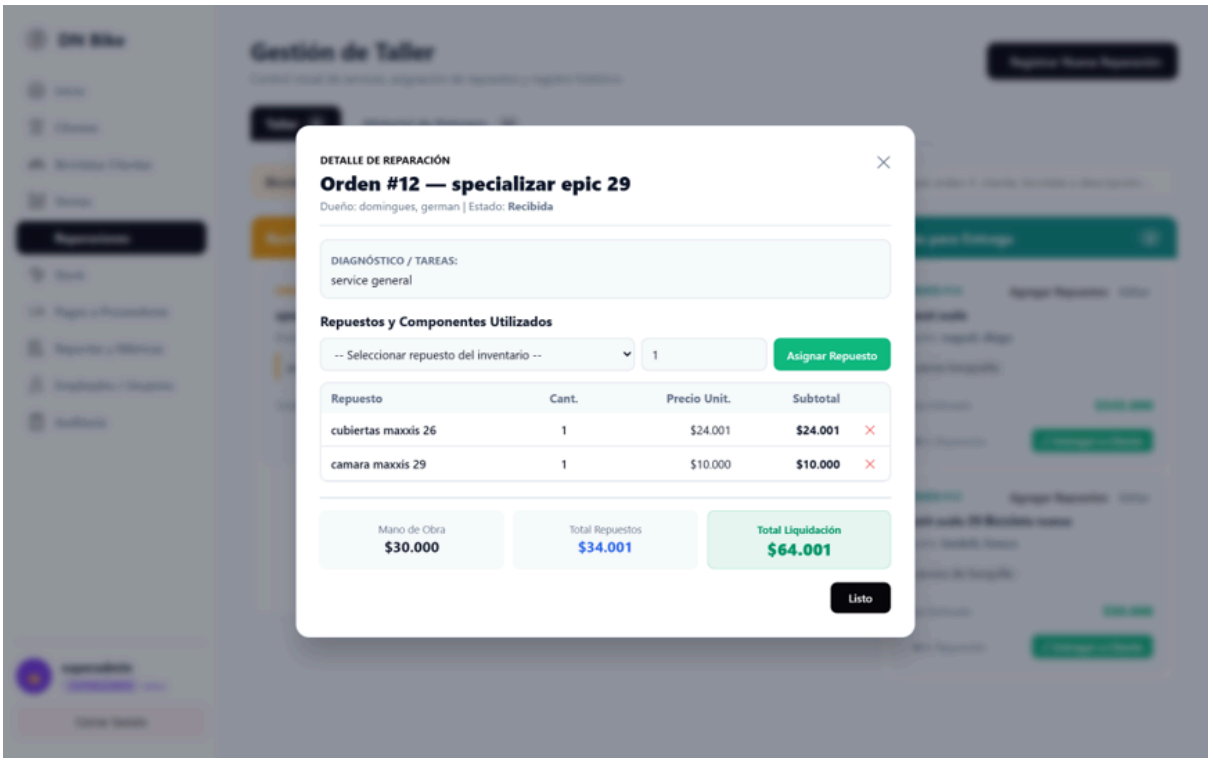
Nota: Monitoreo automatizado del primer service de mantenimiento preventivo oficial para bicicletas nuevas vendidas, controlando los 90 días de su garantía.

Figura 15: Tablero Kanban de reparaciones en taller.



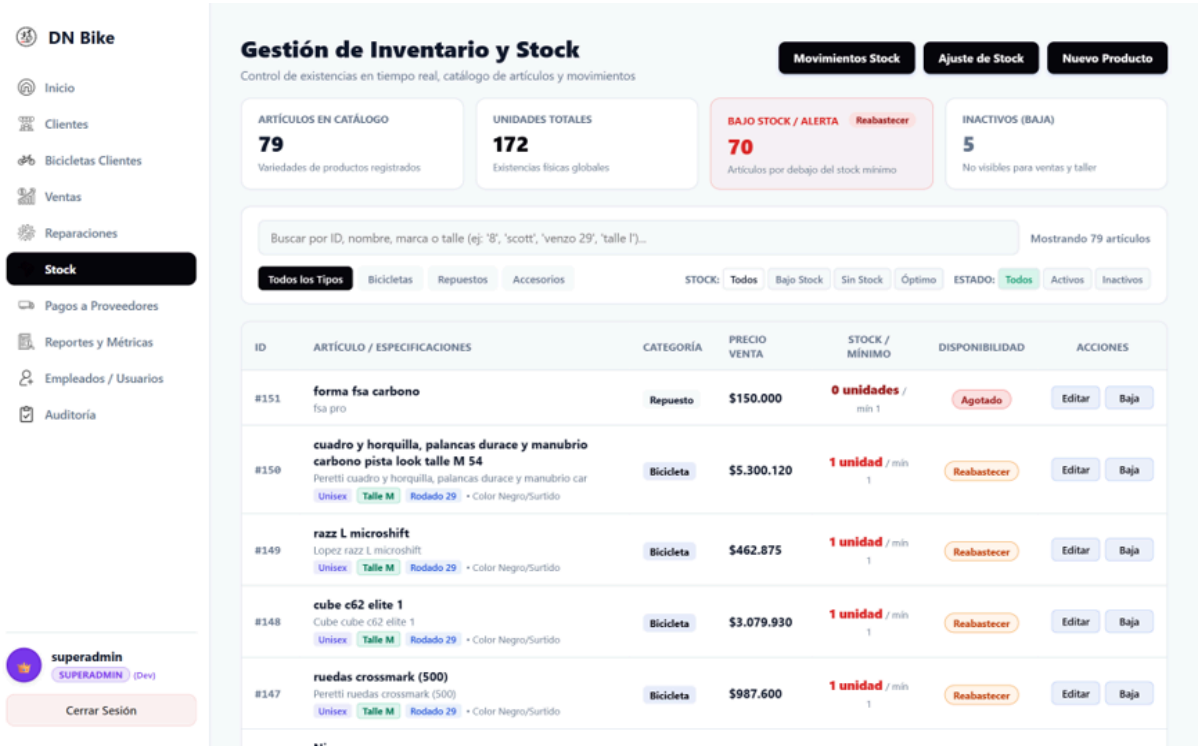
Nota: Tablero visual interactivo que organiza las órdenes de trabajo mecánicas en columnas según su estado: Pendiente, En Proceso, Esperando Repuesto y Lista para Entrega.

Figura 16: Detalle y asignación de repuestos.



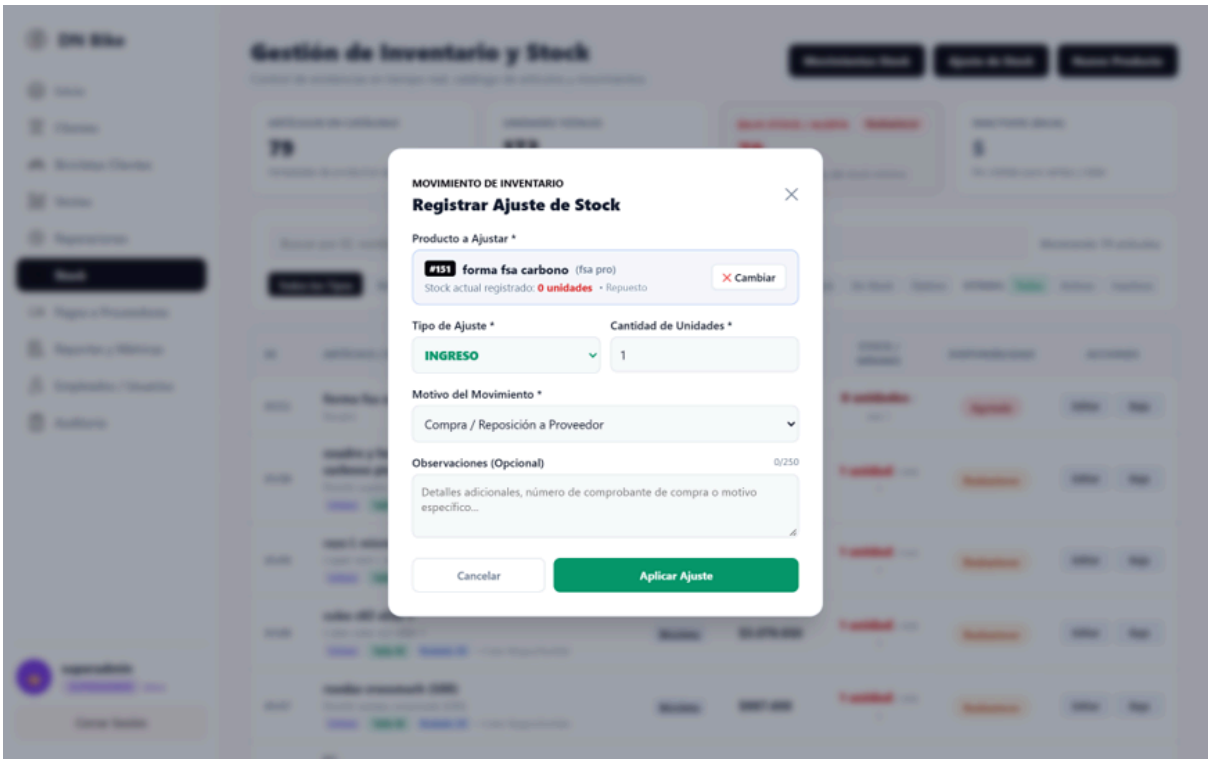
Nota: Ventana modal para asignar repuestos del inventario a una orden de reparación, descontados automáticamente del stock y sumando el costo de mano de obra del mecánico.

Figura 17: Gestión de inventario y catálogo de productos.



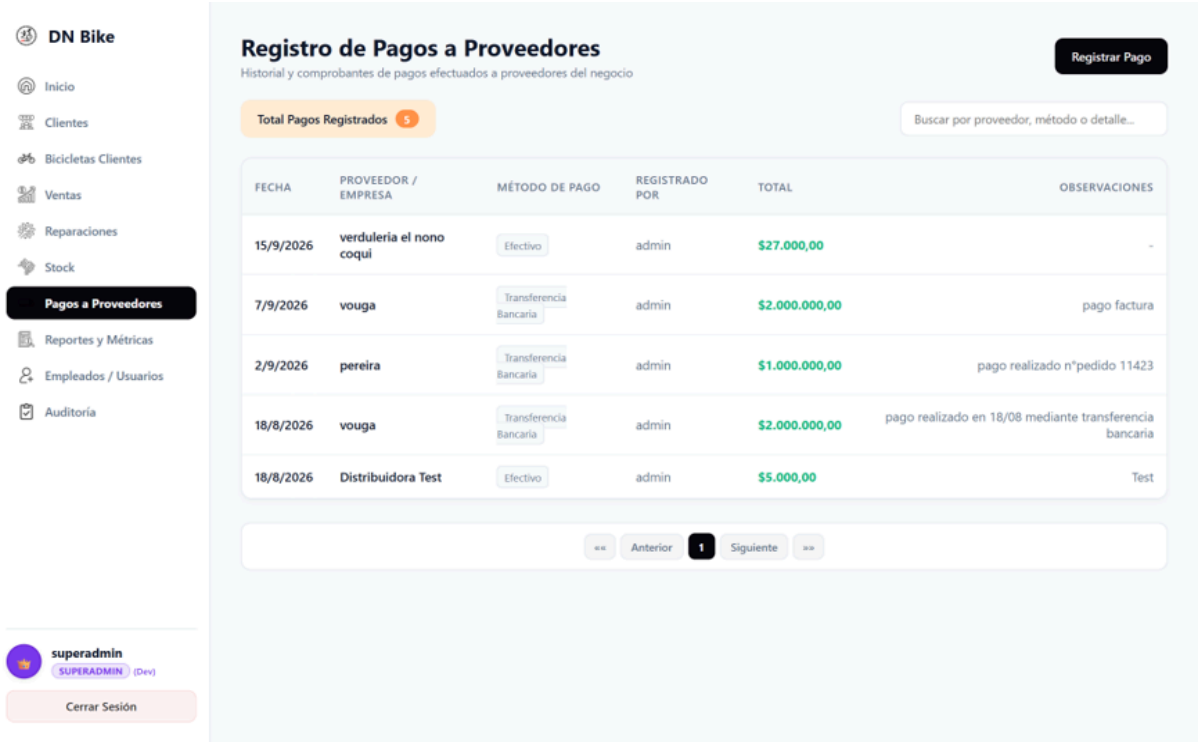
Nota: Grilla con búsqueda y filtrado por categoría de productos (bicicletas, repuestos, indumentaria, accesorios), con visualización clara del stock actual y alertas de stock mínimo.

Figura 18: Formulario de ajuste manual de inventario.



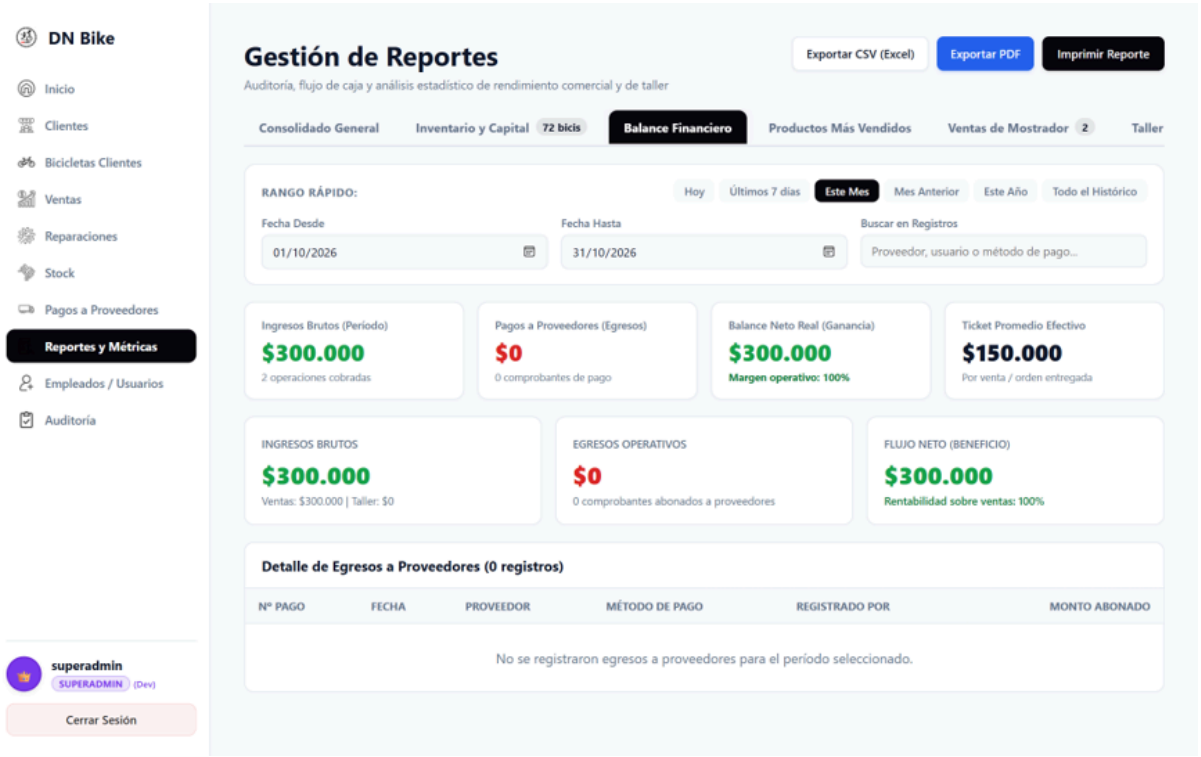
Nota: Formulario para ingresar o egresar unidades físicas de mercadería justificando el motivo (rotura, desvío de conteo, uso interno) con registro para el kardex.

Figura 19: Gestión de Proveedores y Pagos Comerciales.



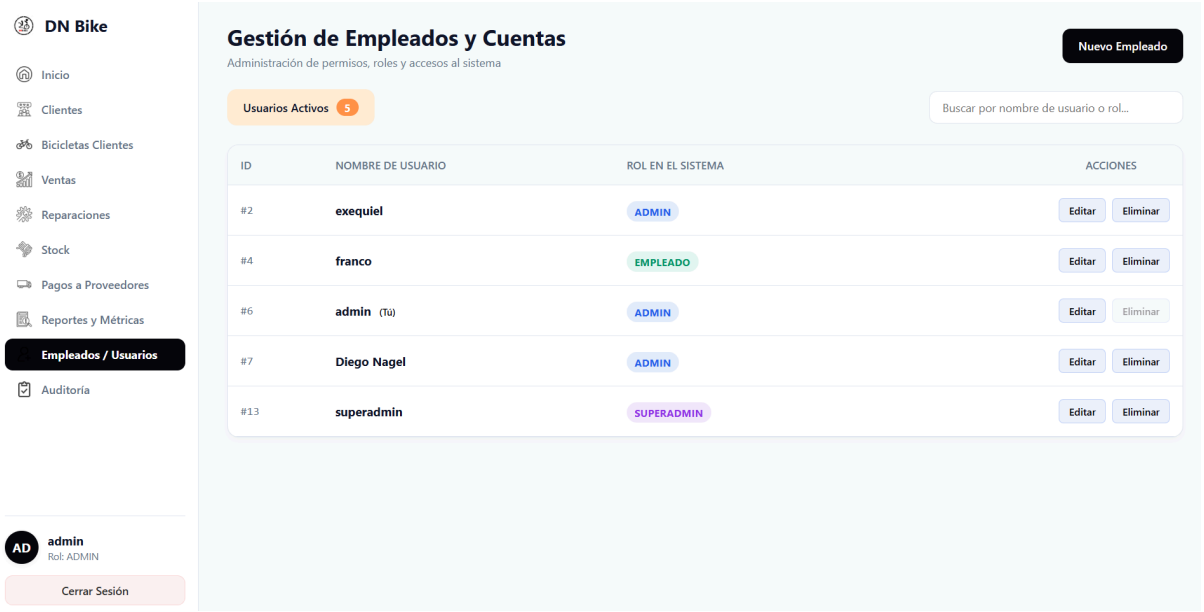
Nota: Directorio de empresas mayoristas de repuestos y componentes, con registro de compras efectuadas y pagos asentados.

Figura 20: Panel de reportes financieros.



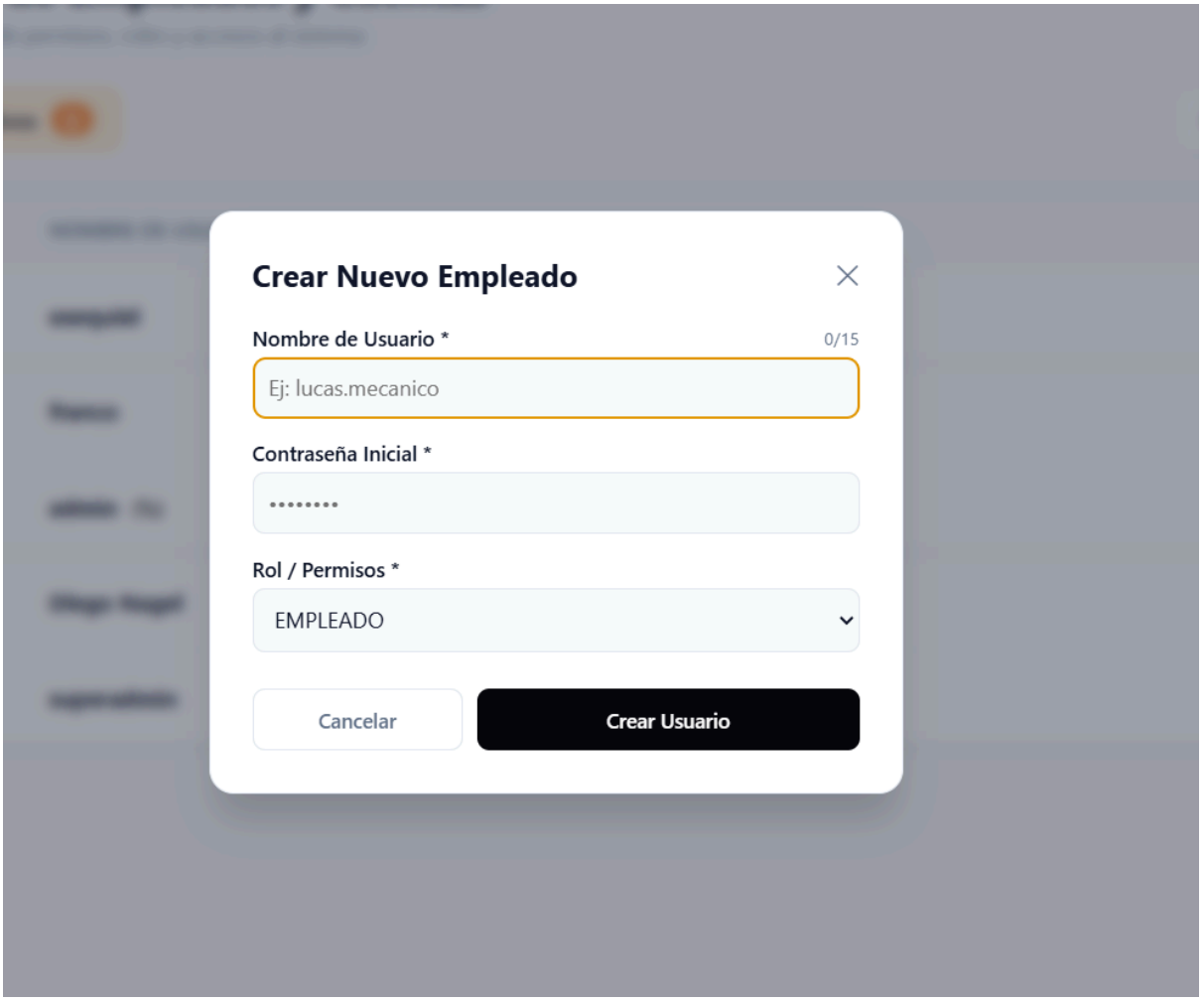
Nota: Gráficos y tablas comparativas de ingresos por taller versus ventas de mostrador, egresos por compras, cálculo de rentabilidad neta y ranking de productos más demandados.

Figura 21: Panel de creación de usuarios.



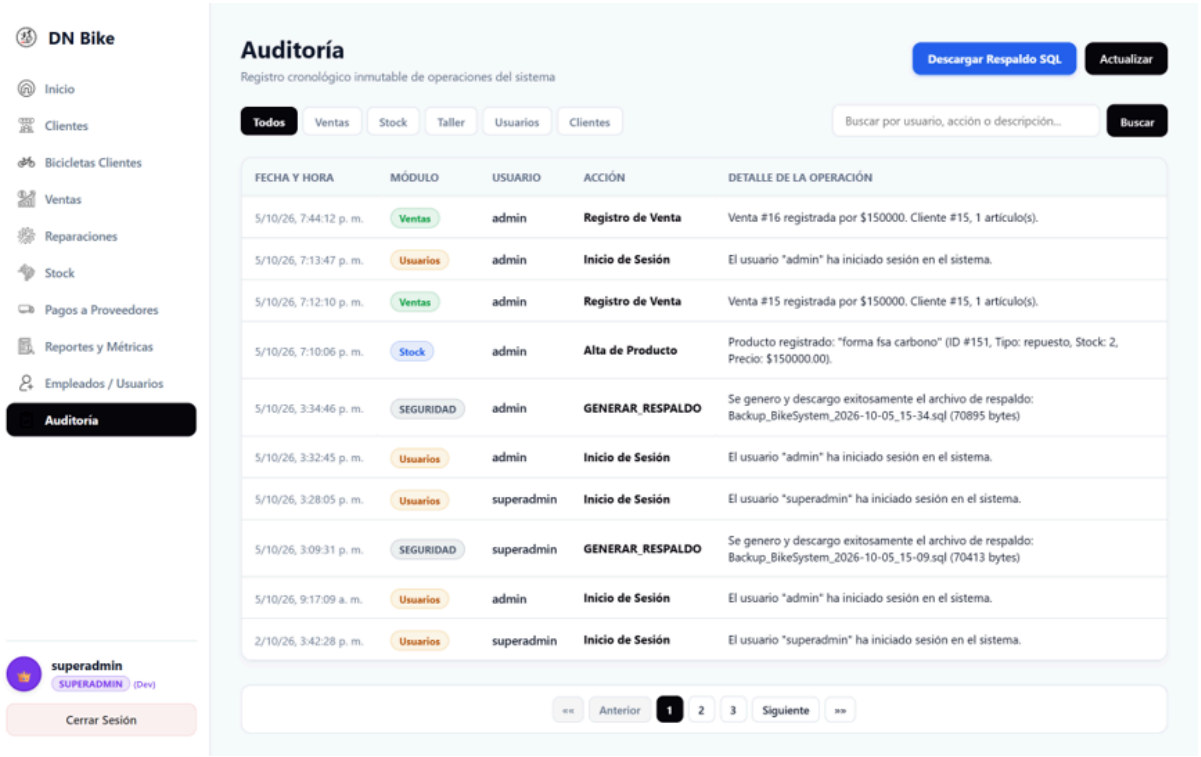
Nota: Panel donde se observan todos los usuarios registrados en el sistema y su rol y edicion de datos dentro del sistema.

Figura 22: ventana modal para crear usuarios.



Nota: Ventana modal para la creación de usuarios con sus correspondientes campos y rol asignado.

Figura 23: Bitácora de auditoría y respaldo de base de datos.



Nota: Registro inmutable de eventos del sistema (inicios de sesión, cambios en inventario, ventas anuladas) y módulo para generar y restaurar copias de respaldo de PostgreSQL.

Estructura del Proyecto

El código fuente del proyecto se organiza bajo la arquitectura de monorepo en carpetas claramente delimitadas:

- frontend/ (Capa de Presentación y Proceso Electron):
  - electron/: Proceso principal de Electron (main.ts), gestión de ventanas nativas del sistema operativo, scripts de precarga (preload.ts) y canales IPC.
  - public/: Recursos estáticos servidos directamente por el navegador (favicon, logos vectoriales).
  - src/views/: 11 Vistas modulares del sistema (Inicio/Dashboard, Ventas/POS con solapa de Garantías y Primer Service, Taller/Kanban, Stock/Inventario, Clientes, Bicicletas, PagoProveedores, Reportes, Auditoría, Usuarios, Login), organizadas internamente con sus propios components/, hooks/ y tabs/.
  - src/components/: Componentes reutilizables de UI (tablas con paginación, modales, alertas, selectores predictivos, layout y navegación lateral Sidebar).

- src/contexts/: Contexto global de autenticación (AuthContext.tsx) para control de sesión, token JWT y rol del usuario.
- src/services/: Cliente HTTP unificado y tipado (api.ts) basado en fetch nativo con inyección automática de cabeceras Authorization: Bearer <token>.
- src/hooks/: Custom hooks transversales y reutilizables (e.g. useDebounce, helpers de sincronización).
- src/types/: Definiciones de interfaces y tipos TypeScript para todas las entidades y DTOs del dominio comercial.
- src/utils/: Funciones utilitarias comunes (formateo de moneda argentina formatearMoneda, parseo de fechas y validaciones de interfaz).
- src/assets/: Recursos estáticos empaquetados, fotografías, iconos y estilos gráficos complementarios.
- package.json & vite.config.ts: Scripts de compilación (dev, build, dist para instalador Windows NSIS) y orquestación con vite-plugin-electron.

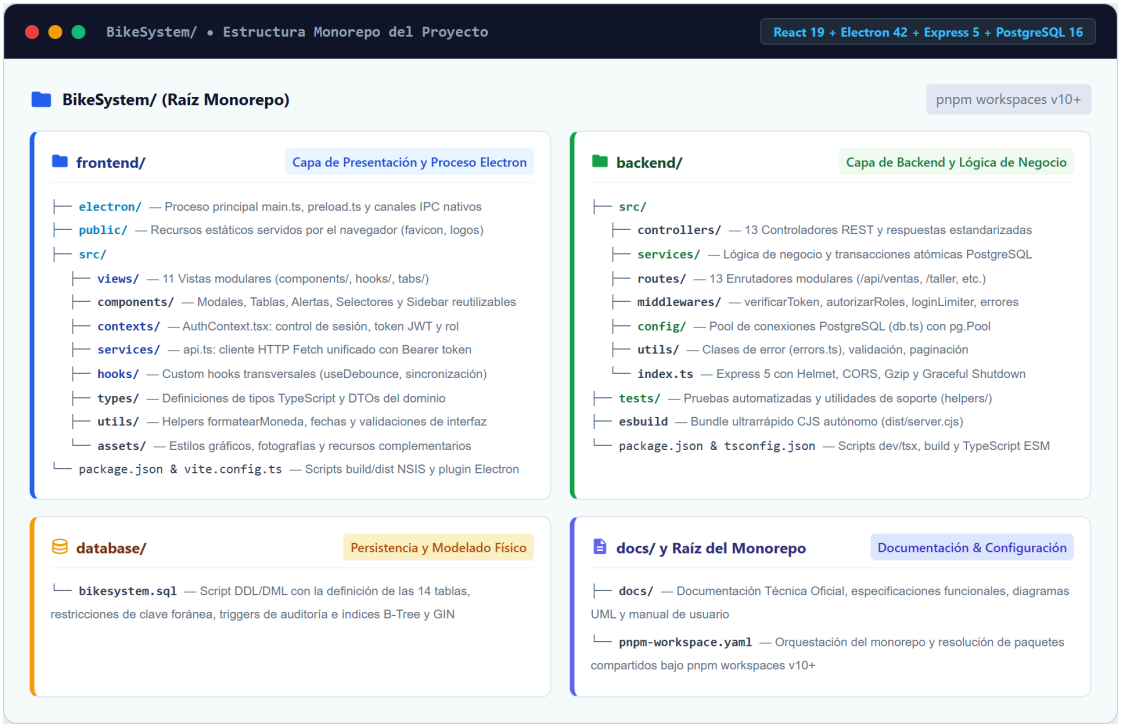
- backend/ (Capa de Backend y Lógica de Negocio):
  - src/controllers/: 13 Controladores REST que procesan las peticiones HTTP, validan los parámetros de entrada y responden con estructuras JSON estandarizadas.
  - src/services/: Capa de lógica de negocio, validaciones transaccionales y operaciones atómicas con PostgreSQL.
  - src/routes/: 13 Enrutadores modulares organizados por entidad comercial (/api/ventas, /api/reparaciones, /api/clientes, etc.) y protegidos por middlewares.
  - src/middlewares/: Interceptores de autenticación JWT (verificarToken), control granular por roles (autorizarRoles), limitador de tasa (loginLimiter) y manejador global de errores (manejarErrores).
  - src/config/: Configuración del pool de conexiones a PostgreSQL (db.ts) mediante pg.Pool y variables de entorno del sistema.
  - src/utils/: Clases de error centralizadas (errors.ts: BadRequestError, UnauthorizedError, NotFoundError), validaciones lógicas (validation.ts),

helpers de paginación (pagination.ts) y respuestas JSON uniformes (response.ts).

- src/index.ts: Punto de entrada del servidor Express 5 con seguridad Helmet, CORS parametrizado, compresión Gzip y cierre ordenado (Graceful Shutdown).
- tests/: Directorio de pruebas automatizadas y utilidades de soporte de testing (tests/helpers/).
- esbuild: Herramienta de compilación ultrarrápida integrada en scripts de compilación de npm que empaqueta todo el backend en un único archivo CJS autónomo (dist/server.cjs) para la distribución final.
- package.json & tsconfig.json: Scripts operativos (dev con tsx watch, build, start) y configuración de TypeScript en modo ESM.

- database/ (Persistencia y Modelado Físico):
  - bikesystem.sql: Script DDL/DML con la definición de las 14 tablas, restricciones de clave foránea, triggers de auditoría e índices B-Tree y GIN.
- docs/, scripts/ y Raíz del Monorepo:
  - docs/: Documentación Técnica Oficial, especificaciones funcionales, diagramas UML y manual de usuario.
  - pnpm-workspace.yaml: Orquestación del monorepo y resolución de paquetes compartidos bajo pnpm workspaces v10+.

Figura 24: estructura de carpetas.



Nota: Se visualiza la estructura de carpetas de la aplicación.

Diagrama de arquitectura de flujo del sistema.

Figura 25: flujo del sistema



Nota: Arquitectura multicapa de BikeSystem que detalla el flujo transaccional de ventas, abarcando la interfaz de usuario (React/Electron), los servicios de backend (Node.js/Express) y la capa de persistencia con transacciones atómicas ACID (PostgreSQL).

Descripción técnica de Funcionalidades.

En esta sección se detalla la implementación técnica, arquitectura interna, endpoints RESTful y mecanismos de persistencia para cada uno de los 11 módulos funcionales que conforman el ecosistema Bike System. Para cada módulo se especifica su propósito operativo, pantallas asociadas, contratos de API, implementación en el frontend (React), lógica de negocio en el backend (Express) y sentencias SQL / transacciones de base de datos (PostgreSQL 16):

Módulo: Login / Autenticación

Descripción General y Reglas de Negocio: Constituye la primera barrera de seguridad de la aplicación de escritorio. Implementa autenticación sin estado mediante tokens JWT (JSON Web Tokens) firmados criptográficamente y control de acceso basado en roles (RBAC) con tres perfiles jerárquicos: SUPERADMIN, ADMIN (Dueño/Encargado) y EMPLEADO (Vendedor/Mecánico). No permite el acceso a ninguna vista operativa sin una sesión activa validada.

Pantalla Asociada: Figura 2: Pantalla de Inicio de Sesión (Login).

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Respuesta Exitosa
POST	/api/login	Público (Rate Limit)	{ nombre_usuario, contrasena }	{ token, usuario: { id, nombre, rol } }
GET	/api/health	Público	Ninguna	{ status: 'ok', message: 'BikeSystem API está corriendo' }
GET	/api/db-test	Público	Ninguna	{ status: 'ok', database: 'Conexión activa con PostgreSQL' }

Implementación en Capa Frontend (React)

El estado de sesión es gobernado de forma centralizada por el contexto AuthContext (src/contexts/AuthContext.tsx). Al autenticarse satisfactoriamente, el token se almacena de forma segura en el almacenamiento local del proceso de renderizado y el cliente HTTP unificado (src/services/api.ts, basado en fetch nativo tipado) inyecta de manera automática el encabezado Authorization: Bearer <token> en cada llamada subsiguiente. Las rutas privadas y las acciones sensibles validan el rol activo antes de renderizar vistas protegidas.

Código Backend Representativo (Servicio de Autenticación y Auditoría)

```
// backend/src/services/auth.service.ts
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { pool } from '../config/db.js';
import { UnauthorizedError, BadRequestError } from '../utils/errors.js';

export class AuthService {
  static async login(nombreUsuario: string, contrasena: string) {
    if (!nombreUsuario?.trim() || !contrasena) {
      throw new BadRequestError('Completá el usuario y la contraseña.');
    }

    const query = `SELECT id_usuario, nombre_usuario, contrasena, rol FROM Usuario
WHERE LOWER(nombre_usuario) = LOWER($1);`;
    const result = await pool.query(query, [nombreUsuario.trim()]);
    const usuario = result.rows[0];

    if (!usuario || !(await bcrypt.compare(contrasena, usuario.contrasena))) {
      throw new UnauthorizedError('Credenciales inválidas.');
    }

    const token = jwt.sign(
      { id: usuario.id_usuario, rol: usuario.rol, nombre_usuario: usuario.nombre_usuario },
      process.env.JWT_SECRET!,
      { expiresIn: '8h' }
    );

    // Registro inmutable en bitácora de auditoría
    await pool.query(
      `INSERT INTO Bitacora_Actividad (id_usuario, nombre_usuario, modulo, accion,
descripcion)
      VALUES ($1, $2, 'Usuarios', 'Inicio de Sesión', $3);`,
```

```
[usuario.id_usuario, usuario.nombre_usuario, `El usuario "${usuario.nombre_usuario}" ha iniciado sesión.`]
    );

    return { token, usuario: { id: usuario.id_usuario, nombre: usuario.nombre_usuario, rol: usuario.rol } };
  }
}
```

Módulo: Home / Dashboard de Operaciones

Descripción General y Reglas de Negocio: Panel de control central que brinda visibilidad en tiempo real sobre la salud comercial y técnica del negocio. Consolida cuatro indicadores clave (KPIs): recaudación total acumulada en el mes, bicicletas actualmente en taller, alertas críticas de artículos bajo stock mínimo y balance financiero neto. El dashboard permite a los operadores y al dueño conocer el estado general del negocio en un solo vistazo sin navegar por múltiples pantallas.

Pantalla Asociada: Figura 3: Dashboard Principal de Operaciones.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Respuesta y Métricas
GET	/api/reportes/dashboard	EMPLEADO / ADMIN	Ninguna	Consolida: ingresos mensuales de ventas y taller, conteo de órdenes activas por estado Kanban y lista de artículos bajo stock mínimo.

Implementación en Capa Frontend (React)

La vista principal InicioView consume los datos mediante el hook de dashboard y renderiza tarjetas métricas de acceso rápido. Implementa la función utilitaria formatearMoneda(valor) adaptada a la moneda de curso legal argentina (ARS), formateando valores como $1.250.000,00. Si existen productos con existencia menor o igual al stock mínimo, se despliega una tabla de alertas prioritarias con insignias en color rojo para

reposición inmediata.

Consulta SQL Agregada de Métricas de Dashboard

```
-- Agregación SQL para ingresos del mes en curso (Ventas + Taller)
SELECT
  COALESCE((SELECT SUM(costo_total) FROM Venta
            WHERE DATE_TRUNC('month', fecha) = DATE_TRUNC('month', CURRENT_DATE)
              AND estado != 'ANULADA'), 0) AS total_ventas_mes,
  COALESCE((SELECT SUM(costo_total) FROM Reparacion
            WHERE DATE_TRUNC('month', fecha_ingreso) = DATE_TRUNC('month', CURRENT_DATE)), 0) AS total_taller_mes,
  (SELECT COUNT(*) FROM Reparacion WHERE estado != 'Entregada') AS reparaciones_activas,
  (SELECT COUNT(*) FROM Productos WHERE cantidad <= stock_minimo AND activo = true) AS productos_bajo_stock;
```

Módulo: Gestión de Clientes

Descripción General y Reglas de Negocio: Administra el padrón de clientes particulares y corporativos de DN Bikes. Todo cliente registrado debe contar con DNI único en el sistema. Incorpora un motor de búsqueda instantáneo optimizado mediante la extensión pg_trgm de PostgreSQL, permitiendo encontrar personas aún con errores tipográficos o coincidencias parciales de nombre o documento.

Pantallas Asociadas: Figura 7: Directorio y Gestión de Clientes y Figura 8: Formulario de Alta y Edición de Cliente.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción
GET	/api/clientes	EMPLEADO / ADMIN	Query: ?busqueda=...	Listado con soporte de paginación y búsqueda predictiva acelerada por trigramas.

GET	/api/clientes/:id	EMPLEADO / ADMIN	Ninguna	Detalle de cliente por identificador primario.
POST	/api/clientes	EMPLEADO / ADMIN	{ nombre, apellido, dni, telefono, email, direccion }	Alta de cliente con validación estricta de unicidad de DNI.
PUT	/api/clientes/:id	EMPLEADO / ADMIN	{ nombre, apellido, dni, telefono, email, direccion }	Actualización de datos personales y de contacto.
DELETE	/api/clientes/:id	ADMIN / SUPERADMIN	Ninguna	Baja física condicionada a que no posea ventas o bicicletas asociadas (ON DELETE RESTRICT).

Implementación en Capa Frontend (React)

La vista ClientesView implementa una tabla con paginación y búsqueda con *debouncing* de 300 ms para evitar sobrecargar el backend con cada pulsación de teclado. La creación y edición se gestionan a través de la ventana modal ModalCliente, la cual valida en tiempo de edición que el nombre y apellido tengan al menos 2 caracteres y que el DNI sea numérico.

Consulta SQL Optimizada con Trigramas

```
-- Búsqueda fonética ultrarrápida combinando nombre, apellido y DNI mediante pg_trgm
SELECT id_cliente, nombre, apellido, dni, telefono, email, direccion
FROM Cliente
WHERE (nombre || ' ' || apellido) ILIKE ('%' || $1 || '%')
   OR dni ILIKE ('%' || $1 || '%')
ORDER BY similarity((nombre || ' ' || apellido), $1) DESC
LIMIT 20;
```

Módulo: Gestión de Parque Rodado y Bicicletas

Descripción General y Reglas de Negocio: Administra el parque de bicicletas de clientes que acuden al taller de DN Bikes. Cada rodado consigna cliente titular, marca, modelo, color, rodado y número de serie del cuadro. La entidad bicicleta se encuentra desacoplada de la compra original, permitiendo desplegar en pantalla su Historial Clínico de Servicios para conocer todas las intervenciones técnicas previas, repuestos consumidos y fechas de entrega.

Pantallas Asociadas: Figura 10: Parque Rodado de Clientes (Bicicletas) y Figura 11: Historial Clínico de Servicios de la Bicicleta.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción
GET	/api/bicicletas	EMPLEADO / ADMIN	Query: ?id_cliente=...	Listado de rodados con filtros por cliente titular y buscador por marca/modelo.
GET	/api/bicicletas/:id	EMPLEADO / ADMIN	Ninguna	Obtiene la ficha técnica de un rodado específico.
GET	/api/bicicletas/:id/historial	EMPLEADO / ADMIN	Ninguna	Historial clínico: órdenes de reparación concluidas, diagnósticos y repuestos aplicados.
POST	/api/bicicletas	EMPLEADO / ADMIN	{ id_cliente, marca, modelo, rodado, color, numero_serie }	Registra una nueva bicicleta vinculada a un cliente.
PUT	/api/bicicletas/:id	EMPLEADO / ADMIN	{ marca, modelo, rodado, color, numero_serie }	Modifica especificaciones técnicas de la bicicleta.

DELETE	/api/bicicletas/:id	ADMIN / SUPERADMIN	Ninguna	Baja de bicicleta condicionada a no tener reparaciones históricas vinculadas.

Consulta SQL de Historial Clínico de Servicios

```
-- Consulta relacional del historial clínico de una bicicleta
SELECT
    r.id_reparacion,
    r.fecha_ingreso,
    r.fecha_egreso,
    r.motivo,
    r.estado,
    r.costo_mano_obra,
    r.costo_total,
    u.nombre_usuario AS mecanico,
    COALESCE(json_agg(json_build_object(
        'repuesto', p.nombre,
        'cantidad', dr.cantidad,
        'precio', dr.precio_unitario
    )) FILTER (WHERE dr.id_detalle_rep IS NOT NULL), '[]') AS repuestos_aplicados
FROM Reparacion r
JOIN Usuario u ON r.id_usuario = u.id_usuario
LEFT JOIN Detalle_Reparacion dr ON r.id_reparacion = dr.id_reparacion
LEFT JOIN Productos p ON dr.id_producto = p.id_producto
WHERE r.id_bicicleta = $1
GROUP BY r.id_reparacion, u.nombre_usuario
ORDER BY r.fecha_ingreso DESC;
```

Módulo: Punto de Venta (Ventas y Presupuestos)

Descripción General y Reglas de Negocio: Optimizado para la atención rápida en el mostrador comercial. Permite buscar productos por nombre o código, gestionar cantidades en un carrito reactivo en memoria, seleccionar cliente registrado o Consumidor Final predeterminado, elegir medio de pago y generar comprobante de venta. Las operaciones se

ejecutan bajo estricta transaccionalidad ACID con prevención de bloqueos mutuos (deadlocks) y asientan simultáneamente el egreso en el Kardex.

Pantallas Asociadas: Figura 12: Punto de Venta (POS) y Facturación de Mostrador y Figura 13: Historial de Ventas y Comprobantes Emitidos.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción
POST	/api/ventas	EMPLEADO / ADMIN	{ id_cliente, id_metodo_pago, detalles: [...] }	Procesa la venta en transacción ACID: descuenta stock físico, asienta Kardex y activa garantías.
GET	/api/ventas	EMPLEADO / ADMIN	Query: ?desde=...&hasta=...	Historial de comprobantes con filtros por fecha, cliente o estado (COMPLETADA / ANULADA).
GET	/api/ventas/:id	EMPLEADO / ADMIN	Ninguna	Detalle de ítems facturados y comprobante de una venta.
GET	/api/ventas/metodos-pago	EMPLEADO / ADMIN	Ninguna	Catálogo de medios de pago disponibles (Efectivo, Tarjeta, Transferencia).
PUT / PATCH	/api/ventas/:id/anular	EMPLEADO / ADMIN	{ motivo }	Anula la venta, reintegra existencias al inventario y audita motivo en bitácora.

Implementación en Capa Frontend (React)

El flujo comercial está gobernado por el hook local useCarritoVenta (src/views/Ventas/hooks/useCarritoVenta.ts), el cual administra los ítems en memoria sin

acoplar estado global superfluo. Al finalizar la compra, se abre una ventana modal con el resumen para confirmar el medio de cobro y emitir el ticket.

Código Backend Representativo (Transacción ACID y Prevención de Deadlocks)

```
// backend/src/services/venta.service.ts
export class VentaService {
  static async crearVenta(datos: { id_cliente: number; id_metodo_pago?: number; detalles: any[]; idUsuarioOperador: number }) {
    // 1. Prevención de Deadlocks: ordenar productos por ID antes de bloquear con FOR UPDATE
     const detallesOrdenados = [...datos.detalles].sort((a, b) => Number(a.id_producto) - Number(b.id_producto));

   const client = await pool.connect();
   try {
    await client.query('BEGIN'); // Inicio de Transacción ACID

    // 2. Insertar cabecera de la venta
    const resVenta = await client.query(
     `INSERT INTO Venta (id_cliente, id_usuario, id_metodo_pago, costo_total)
      VALUES ($1, $2, $3, 0) RETURNING id_venta;`,
     [datos.id_cliente, datos.idUsuarioOperador, datos.id_metodo_pago || 1]
    );
    const idVenta = resVenta.rows[0].id_venta;
    let totalCalculado = 0;

    // 3. Procesar artículos con bloqueo concurrente FOR UPDATE
    for (const item of detallesOrdenados) {
     const { rows: prod } = await client.query(
          `SELECT precio, cantidad FROM Productos WHERE id_producto = $1 FOR UPDATE;`,
       [item.id_producto]
      );
```

```
        if (!prod.length || prod[0].cantidad < item.cantidad) {
                    throw new BadRequestError(`Stock insuficiente para el producto ID ${item.id_producto}`);
        }
        const subtotal = Number(prod[0].precio) * item.cantidad;
        totalCalculado += subtotal;

        // Descontar inventario físico
            await client.query(`UPDATE Productos SET cantidad = cantidad - $1 WHERE id_producto = $2;`, [item.cantidad, item.id_producto]);

        // Registrar detalle de venta
        await client.query(
            `INSERT INTO Detalle_Venta (id_venta, id_producto, cantidad, precio_unitario, costo_total)
             VALUES ($1, $2, $3, $4, $5);`,
           [idVenta, item.id_producto, item.cantidad, prod[0].precio, subtotal]
        );

        // Asiento inmutable en Kardex (Movimiento_Stock)
        await client.query(
            `INSERT INTO Movimiento_Stock (id_producto, id_usuario, tipo_movimiento, cantidad, motivo, observaciones)
             VALUES ($1, $2, 'EGRESO', $3, 'VENTA', $4);`,
              [item.id_producto, datos.idUsuarioOperador, item.cantidad, `Venta Comprobante #${idVenta}`]
        );
      }

      // Consolidar monto total definitivo
        await client.query(`UPDATE Venta SET costo_total = $1 WHERE id_venta = $2;`, [totalCalculado, idVenta]);
      await client.query('COMMIT'); // Consolidación atómica
      return { id_venta: idVenta, total: totalCalculado };
```

```
    } catch (error) {
      await client.query('ROLLBACK'); // Reversión total ante cualquier anomalía
      throw error;
    } finally {
      client.release();
    }
  }
}
```

Módulo: Gestión de Garantías y Primer Service

Descripción General y Reglas de Negocio: Administra el beneficio posventa distintivo de DN Bikes: toda bicicleta nueva vendida incluye por derecho un primer service oficial gratuito dentro de los 90 días corridos posteriores a la fecha de compra. El módulo monitorea automáticamente el plazo transcurrido mediante funciones de calendario en PostgreSQL, clasifica el estado en tiempo real (Vigente, Vencida o Realizada) y permite asentar su redención técnica con un solo clic.

Pantalla Asociada: Figura 14: Gestión de Garantías y Primer Service (90 días).

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción
GET	/api/ventas/garantias	EMPLEADO / ADMIN	Ninguna	Listado de garantías con cálculo dinámico de vigencia y días transcurridos.
PATCH	/api/ventas/garantias/:id_detalle_venta/primer-service	EMPLEADO / ADMIN	Ninguna	Registra la ejecución del primer service preventivo gratuito oficial.

Lógica de Negocio y Regla de los 90 Días en PostgreSQL

```
-- Cálculo dinámico de vigencia de garantía en PostgreSQL
SELECT
```

```
    dv.id_detalle_venta,
    v.id_venta,
    v.fecha AS fecha_venta,
    c.nombre || ' ' || c.apellido AS cliente,
    p.nombre AS bicicleta,
    dv.primer_service_realizado,
    dv.fecha_primer_service,
    (CURRENT_DATE - v.fecha) AS dias_transcurridos,
    CASE
        WHEN dv.primer_service_realizado THEN 'REALIZADA'
        WHEN (CURRENT_DATE - v.fecha) <= 90 THEN 'VIGENTE'
        ELSE 'VENCIDA'
    END AS estado_garantia
FROM Detalle_Venta dv
JOIN Venta v ON dv.id_venta = v.id_venta
JOIN Productos p ON dv.id_producto = p.id_producto
JOIN Cliente c ON v.id_cliente = c.id_cliente
WHERE p.tipo_prod = 'bicicleta' AND v.estado != 'ANULADA'
ORDER BY v.fecha DESC;
```

Módulo: Taller de Reparaciones y Órdenes de Trabajo

Descripción General y Reglas de Negocio: Corazón operativo del taller mecánico de DN Bikes. Organiza las órdenes de servicio en un tablero visual Kanban interactivo compuesto por tres estados en curso: Recibida, En Reparación y Lista (con soporte de arrastre drag-and-drop). Permite diagnosticar averías, consumir repuestos del almacén en tiempo real debitando inventario y calcular la liquidación final sumando honorarios de mano de obra. Una vez abonada y retirada la bicicleta, la orden pasa al estado Entregada y se archiva en la pestaña de Historial de Entregadas con comprobante.

Pantallas Asociadas: Figura 15: Tablero Kanban de Reparaciones de Taller y Figura 16: Detalle y Asignación de Repuestos en Taller.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción

GET	/api/reparaciones	EMPLEADO / ADMIN	Query: ?estado=...	Retorna órdenes activas para el tablero Kanban e históricas con filtros por estado.
POST	/api/reparaciones	EMPLEADO / ADMIN	{ id_bicicleta, motivo, costo_mano_obra }	Abre una nueva orden de servicio técnico en estado inicial 'Recibida'.
GET	/api/reparaciones/:id	EMPLEADO / ADMIN	Ninguna	Detalle de la orden, mano de obra, diagnóstico y repuestos empleados.
PUT	/api/reparaciones/:id	EMPLEADO / ADMIN	{ estado, costo_mano_obra, motivo }	Actualiza estado Kanban, mano de obra estimada y observaciones técnicas.
POST	/api/detalle-reparacion	EMPLEADO / ADMIN	{ id_reparacion, id_producto, cantidad, precio_unitario }	Asigna un repuesto a la orden, descontándolo del stock físico y asentando Kardex.
GET	/api/detalle-reparacion/:id_reparacion	EMPLEADO / ADMIN	Ninguna	Listado de repuestos imputados a una orden de reparación.
DELETE	/api/detalle-reparacion/:id_detalle_rep	EMPLEADO / ADMIN	Ninguna	Quita un repuesto de la orden y reintegra su cantidad al stock físico.

Implementación en Capa Frontend (React)

El componente ReparacionesKanban.tsx implementa la API nativa de Drag-and-Drop de HTML5 (onDragStart, onDragOver, onDrop). Al soltar una tarjeta en una columna diferente, dispara una llamada PUT al servidor actualizando el estado. La ventana modal ModalDetalleReparacion.tsx permite asignar repuestos buscando en el inventario activo, calculando automáticamente el subtotal acumulado.

Código Backend: Consumo de Repuesto en Taller y Descuento Atómico

```
// backend/src/services/detalleReparacion.service.ts
export class DetalleReparacionService {
  static async agregarRepuesto(idReparacion: number, idProducto: number, cantidad: number, idUsuario: number) {
    const client = await pool.connect();
    try {
      await client.query('BEGIN');

      // 1. Verificar existencia de stock
       const { rows: prod } = await client.query('SELECT precio, cantidad FROM Productos WHERE id_producto = $1 FOR UPDATE', [idProducto]);
      if (!prod.length || prod[0].cantidad < cantidad) {
        throw new BadRequestError('Stock insuficiente para este repuesto.');
      }

      // 2. Insertar en Detalle_Reparacion
      await client.query(
            `INSERT INTO Detalle_Reparacion (id_reparacion, id_producto, cantidad, precio_unitario)
        VALUES ($1, $2, $3, $4);`,
        [idReparacion, idProducto, cantidad, prod[0].precio]
      );

      // 3. Descontar inventario físico
```

```
        await client.query('UPDATE Productos SET cantidad = cantidad - $1 WHERE id_producto = $2;', [cantidad, idProducto]);

      // 4. Asentar en Kardex (Movimiento_Stock)
      await client.query(
          `INSERT INTO Movimiento_Stock (id_producto, id_usuario, tipo_movimiento, cantidad, motivo, observaciones)
        VALUES ($1, $2, 'EGRESO', $3, 'TALLER', $4);`,
            [idProducto, idUsuario, cantidad, `Consumo en Orden de Reparación #${idReparacion}`]
      );

      await client.query('COMMIT');
    } catch (e) {
      await client.query('ROLLBACK');
      throw e;
    } finally {
      client.release();
    }
  }
}
```

Módulo: Gestión de Inventario y Stock

Descripción General y Reglas de Negocio: Administra el catálogo unificado de repuestos, accesorios, indumentaria y bicicletas nuevas. Controla existencias físicas en tiempo real, alerta ante niveles inferiores al stock_minimo y registra cada entrada, salida o ajuste manual en una bitácora inmutable de movimientos (Kardex). Ningún cambio de stock puede realizarse sin quedar registrado con motivo y usuario responsable.

Pantallas Asociadas: Figura 17: Gestión de Inventario y Catálogo de Productos y Figura 18: Formulario de Ajuste Manual de Inventario.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción

GET	/api/productos	EMPLEADO / ADMIN	Query: ?categoria=...&busqueda=...	Catálogo completo con paginación, filtros por categoría y búsqueda de texto.
GET	/api/productos/:id	EMPLEADO / ADMIN	Ninguna	Ficha técnica y existencias de un artículo puntual.
POST	/api/productos	EMPLEADO / ADMIN	{ nombre, codigo, precio, costo, cantidad, stock_minimo, ... }	Alta de producto en catálogo comercial.
PUT	/api/productos/:id	EMPLEADO / ADMIN	{ nombre, precio, costo, stock_minimo, ... }	Actualización de datos comerciales y niveles de stock de seguridad.
DELETE	/api/productos/:id	ADMIN / SUPERADMIN	Ninguna	Baja lógica del producto (activo = false).
PUT / PATCH	/api/productos/:id/reactivar	ADMIN / SUPERADMIN	Ninguna	Reactivación del producto en catálogo.
GET	/api/productos/movimientos	EMPLEADO / ADMIN	Query: ?tipo=...&desde=...	Historial general de movimientos Kardex (ingresos, egresos y ajustes).
POST	/api/productos/movimientos	EMPLEADO / ADMIN	{ id_producto, tipo_movimiento, cantidad, motivo }	Registro de ajuste manual de inventario con motivo obligatorio y auditoría.

GET	/api/productos/:id/movimientos	EMPLEADO / ADMIN	Ninguna	Kardex individual de un producto específico.

Módulo: Gestión de Proveedores y Compras

Descripción General y Reglas de Negocio: Directorio de empresas distribuidoras y fabricantes mayoristas de componentes de ciclismo. Permite registrar las compras de mercadería y asentar pagos formalmente con comprobante y método de desembolso, alimentando el flujo de egresos financieros de la bicicletería para el cálculo de la rentabilidad neta.

Pantalla Asociada: Figura 19: Gestión de Proveedores y Pagos Comerciales.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción
GET	/api/proveedores	ADMIN / SUPERADMIN	Ninguna	Directorio de proveedores registrados con datos de contacto comercial.
POST	/api/proveedores	ADMIN / SUPERADMIN	{ nombre_empresa, contacto, telefono, email, direccion }	Alta de proveedor con validación de razón social.
GET	/api/proveedores/:id	ADMIN / SUPERADMIN	Ninguna	Ficha individual de proveedor y compras acumuladas.
PUT	/api/proveedores/:id	ADMIN / SUPERADMIN	{ nombre_empresa, telefono, email, direccion }	Modificación de datos de contacto de proveedor.

DELETE	/api/proveedores/:id	ADMIN / SUPERADMIN	Ninguna	Baja de proveedor condicionada a no poseer pagos vinculados.
GET	/api/pagos-proveedores	ADMIN / SUPERADMIN	Query: ?desde=...&hasta=...	Historial contable de egresos por compras a proveedores.
POST	/api/pagos-proveedores	ADMIN / SUPERADMIN	{ id_proveedor, monto, id_metodo_pago, comprobante }	Asiento de pago a proveedor con fecha, monto, método y número de factura.
GET	/api/pagos-proveedores/metodos-pago	ADMIN / SUPERADMIN	Ninguna	Medios de pago autorizados para egresos de compras.

Módulo: Reportes y Estadísticas

Descripción General y Reglas de Negocio: Módulo analítico gerencial exclusivo para el rol Administrador. Integra la recaudación por ventas de mostrador y servicios de taller contra los egresos por compras a proveedores, calculando el balance financiero neto del período conforme a la fórmula formal: Rentabilidad Neta = (Ingresos Ventas + Ingresos Taller) - Egresos Proveedores. Genera valorizaciones de capital en stock y gráficos de barras comparativos.

Pantalla Asociada: Figura 20: Panel de Reportes Financieros y Rentabilidad.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción
GET	/api/reportes/dashboard	EMPLEADO / ADMIN	Ninguna	KPIs consolidados del mes: ingresos, órdenes de taller y alertas de stock.

GET	/api/reportes/estadisticas	ADMIN / SUPERADMIN	Query: ?desde=...&hasta=...	Totales acumulados de ingresos, egresos y balance neto en rango de fechas.
GET	/api/reportes/ventas	ADMIN / SUPERADMIN	Query: ?desde=...&hasta=...	Desglose temporal de ventas por fecha, cliente y forma de cobro.
GET	/api/reportes/reparaciones	ADMIN / SUPERADMIN	Query: ?desde=...&hasta=...	Ingresos por taller clasificados por mano de obra y repuestos aplicados.
GET	/api/reportes/egresos	ADMIN / SUPERADMIN	Query: ?desde=...&hasta=...	Detalle cronológico de egresos por compras y pagos mayoristas.
GET	/api/reportes/top-productos	ADMIN / SUPERADMIN	Ninguna	Ranking de productos más demandados por volumen de venta y recaudación.
GET	/api/reportes/capital-stock	ADMIN / SUPERADMIN	Ninguna	Valorización económica del inventario actual a precio de costo y venta.

Consulta SQL de Consolidación Financiera y Rentabilidad

```
-- Cálculo formal de balance financiero y rentabilidad neta
SELECT
    v.ingresos_ventas,
    r.ingresos_taller,
    (v.ingresos_ventas + r.ingresos_taller) AS ingresos_totales,
    p.egresos_proveedores,
    ((v.ingresos_ventas + r.ingresos_taller) - p.egresos_proveedores) AS rentabilidad_neta
FROM (
    SELECT COALESCE(SUM(costo_total), 0) AS ingresos_ventas
    FROM Venta WHERE fecha BETWEEN $1 AND $2 AND estado != 'ANULADA'
```

```
) v
CROSS JOIN (
   SELECT COALESCE(SUM(costo_total), 0) AS ingresos_taller
      FROM Reparacion WHERE fecha_egreso BETWEEN $1 AND $2 AND estado =
'Entregada'
) r
CROSS JOIN (
   SELECT COALESCE(SUM(monto), 0) AS egresos_proveedores
   FROM Pago_Proveedor WHERE fecha BETWEEN $1 AND $2
) p;
```

Módulo: Configuración del Sistema y Respaldo

Descripción General y Reglas de Negocio: Núcleo de administración y seguridad del sistema. Permite dar de alta usuarios y asignar roles del personal (RBAC), consultar la bitácora inmutable de eventos sensibles del negocio (Bitacora_Actividad) y generar copias de seguridad de la base de datos PostgreSQL en caliente vía streaming SQL descargable ante contingencias de hardware.

Pantalla Asociada: Figura 23: Bitácora de Auditoría y Respaldo de Base de Datos.

Endpoints de la API REST

Método	Ruta	Acceso	Carga Útil (Body)	Descripción
GET	/api/usuarios	ADMIN / SUPERADMIN	Ninguna	Padrón de cuentas de usuario con roles asignados.
POST	/api/usuarios	ADMIN / SUPERADMIN	{ nombre_usuario, contrasena, rol }	Creación de usuario con contraseña hasheada mediante Bcrypt y rol.
GET	/api/usuarios/:id	ADMIN / SUPERADMIN	Ninguna	Consulta de perfil y estado de un usuario del personal.

PUT	/api/usuarios/:id	ADMIN / SUPERADMIN	{ nombre_usuario, rol, contrasena? }	Actualización de datos personales, cambio de contraseña o cambio de rol.
DELETE	/api/usuarios/:id	ADMIN / SUPERADMIN	Ninguna	Baja física de usuario con restricción de integridad referencial.
GET	/api/bitacora	ADMIN / SUPERADMIN	Query: ?modulo=...&desde=...	Historial inmutable de auditoría con filtros por módulo, acción y fecha.
GET	/api/backup/descargar	ADMIN / SUPERADMIN	Ninguna	Streaming de volcado DDL/DML completo en archivo .sql descargable.

Arquitectura General y Decisiones de Diseño Transversales

Arquitectura de Capas y Separación de Responsabilidades

Para garantizar alta cohesión y bajo acoplamiento, BikeSystem implementa una arquitectura en capas estrictamente desacopladas con dependencias unidireccionales:

- Capa de Presentación (Frontend Desktop): Reside en Electron y React 19. Su única incumbencia es la captura de eventos de usuario, el renderizado reactivo de la interfaz y la validación preliminar de formularios. No contiene consultas SQL directas ni lógica de cálculo contable crítico.
- Capa de Enrutamiento y Middlewares (API Gateway Local): Express 5 intercepta cada solicitud entrante, valida la firma criptográfica del JWT y verifica si el rol asignado al usuario posee los permisos necesarios para ejecutar la acción solicitada.
- Capa de Controladores (Controllers): Extrae los parámetros de la petición HTTP, normaliza los datos y delega la ejecución al servicio correspondiente, retornando códigos de estado HTTP estandarizados (200, 201, 400, 401, 403, 404, 500).

- Capa de Servicios y Lógica de Negocio (Services): Encapsula las reglas operativas del negocio (cálculo de costos de taller, aplicación de descuentos, validación de garantías). Es la única capa autorizada para abrir transacciones ACID con BEGIN ... COMMIT / ROLLBACK sobre el pool de base de datos.
- Capa de Persistencia (PostgreSQL): Ejecuta sentencias SQL preparadas, gestiona índices de búsqueda GIN y B-Tree, y dispara triggers de auditoría.

Transaccionalidad ACID y Consistencia de Inventario

Una de las decisiones arquitectónicas fundamentales del sistema es la implementación del Kardex Unificado de Inventario. Toda mutación física en las existencias de Productos se encuentra protegida por transacciones atómicas que asientan simultáneamente un registro en Movimiento_Stock:

Tipo de Operación	Sentencia sobre Productos	Asiento en Movimiento_Stock	Garantía Transaccional
Venta Comercial	UPDATE ... SET cantidad = cantidad - $1	tipo='EGRESO', motivo='VENTA_MOSTRADOR'	Atómica dentro de la transacción de cabecera y detalles de venta.
Repuestos en Taller	UPDATE ... SET cantidad = cantidad - $1	tipo='EGRESO', motivo='TALLER_REPARACION'	Atómica al asignar o modificar repuestos de la orden de trabajo.
Anulación de Venta	UPDATE ... SET cantidad = cantidad + $1	tipo='INGRESO', motivo='ANULACION_VENTA'	Atómica al cambiar el estado de la venta a 'ANULADA'.
Ajuste de Inventario	UPDATE ... SET cantidad = cantidad ± $1	tipo='INGRESO/EGRESO', motivo=motivo_usuario	Atómica en el endpoint de ajuste manual con auditoría obligatoria.

Mecanismo de Seguridad, Hash Bcrypt y Tokens JWT

La seguridad del sistema se fundamenta en dos mecanismos estándares de la industria:

- Hash de Contraseñas con Bcrypt: Las contraseñas de los operadores nunca se almacenan en texto plano en la base de datos. Se procesan con bcrypt.hash(password, 10), introduciendo un salt aleatorio que previene ataques de diccionario y tablas arcoíris.
- Tokens JWT sin Estado (Stateless): Al iniciar sesión exitosamente, el servidor emite un token firmado mediante HMAC-SHA256 con una clave secreta configurada en el archivo de entorno. El token contiene el identificador de usuario, nombre y rol, evitando realizar consultas redundantes a la tabla de usuarios en cada petición.

Middlewares de Autorización y Roles (RBAC)

El control de acceso basado en roles se implementa mediante middlewares modulares y reutilizables en Express, tipando la solicitud mediante PeticionConUsuario:

```
// backend/src/middlewares/auth.middleware.ts
import type { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

export interface PeticionConUsuario extends Request {
  usuarioToken?: { id: number; rol: string; nombre_usuario?: string };
}

export const verificarToken = (req: PeticionConUsuario, res: Response, next: NextFunction): void => {
  const tokenHeader = req.headers['authorization'];
  if (!tokenHeader) {
    res.status(403).json({ error: 'Tenés que iniciar sesión primero.' });
    return;
  }
```

```
    const token = tokenHeader.split(' ')[1];
    if (!token) {
        res.status(403).json({ error: 'Sesión no válida.' });
        return;
    }
    try {
         const decoded = jwt.verify(token, process.env.JWT_SECRET!) as { id: number; rol:
string; nombre_usuario?: string };
        req.usuarioToken = decoded;
        next();
    } catch {
        res.status(401).json({ error: 'Tu sesión expiró. Volvé a iniciar sesión.' });
    }
};

// backend/src/middlewares/roles.middleware.ts
import type { Response, NextFunction } from 'express';
import type { PeticionConUsuario } from './auth.middleware.js';

export const autorizarRoles = (...rolesPermitidos: string[]) => {
    return (req: PeticionConUsuario, res: Response, next: NextFunction): void => {
        const rolUsuario = req.usuarioToken?.rol;
        if (!rolUsuario || !rolesPermitidos.includes(rolUsuario)) {
            res.status(403).json({ error: 'No tenés permisos para realizar esta operación.' });
            return;
        }
        next();
    };
};
```

Empaquetado y Distribución Desktop (Electron Builder y Monorepo)

El despliegue para la empresa DN Bikes se simplifica radicalmente gracias a la configuración de Electron Builder con instalador NSIS. Mediante el comando pnpm dist en la

carpeta frontend, se ejecutan las siguientes fases automáticas:

- Compilación del backend Express a un único archivo bundle CommonJS autónomo (dist/server.cjs) mediante esbuild.
- Compilación del frontend React 19 a assets estáticos optimizados mediante vite build.
- Compilación del proceso principal y script de precarga de Electron en dist-electron/.
- Generación del ejecutable instalador portátil para Windows (release/BikeSystem Setup 1.0.0.exe).

Conclusiones y Trabajo Futuro

Conclusiones

En síntesis del proyecto y cumplimiento de objetivos el presente trabajo integrador tuvo como punto de partida una necesidad concreta y real: el establecimiento comercial y taller mecánico "DN Bikes" gestionaba su actividad a través de registros en cuadernos físicos y planillas de Excel dispersas, lo que generaba descontrol en el inventario de repuestos, extravío de historiales de servicio en taller, falta de seguimiento en garantías oficiales y ausencia de herramientas analíticas para la toma de decisiones. Frente a esa realidad, nos propusimos desarrollar un sistema de gestión integral de escritorio, adaptado a los flujos reales de trabajo del negocio, que centralizara y automatizara sus procesos cotidianos.

A continuación se verifica el grado de cumplimiento de cada objetivo específico planteado en la sección Relevamiento y Análisis de Requerimientos, reflejando de forma transparente las evoluciones arquitectónicas implementadas respecto a las concepciones preliminares de 2025:

Objetivo Específico Planteado	Estado	Módulo Asociado / Justificación de Evolución
Diseñar un módulo de inventario con control de stock mínimo y kardex	✅ Cumplido	Gestión de Inventario y Kardex: Catálogo clasificado, alertas de stock de seguridad y registro inmutable en Movimiento_Stock.

Desarrollar un módulo de punto de venta ágil con emisión de comprobantes	✅ Cumplido	Punto de Venta: Carrito en memoria, soporte para consumidor final anónimo o cliente registrado, y control transaccional ACID.
Implementar la gestión de clientes y parque de bicicletas con historial clínico	✅ Cumplido	Clientes y Parque Rodado: Búsqueda fonética con trigramas e historial clínico cronológico por rodado.
Crear un módulo de taller mecánico con tablero de estados y consumo de repuestos	✅ Cumplido	Taller de Reparaciones (Kanban): Flujo interactivo con 3 columnas en proceso y vista de entregadas con liquidación de mano de obra y repuestos.
Automatizar el seguimiento del primer service gratuito de garantía a 90 días	✅ Cumplido	Gestión de Garantías: Monitoreo algorítmico de los 90 días corridos posventa con redención técnica en un solo clic.
Registrar compras y pagos a distribuidores mayoristas de repuestos	✅ Cumplido	Proveedores y Pagos: Directorio comercial, imputación de egresos con método de pago y número de factura.
Proveer herramientas de control de flujo de fondos y caja	✅ Cumplido (Evolucionado)	Reportes Financieros y Flujo de Fondos: Evolucionó de una apertura/cierre estático de caja hacia un balance analítico continuo en tiempo real (ingresos menos egresos), más robusto ante cortes de energía y sin riesgo de olvidos de arqueo físico.
Generar reportes analíticos y balances financieros de ingresos y egresos	✅ Cumplido	Reportes y Estadísticas: Dashboard gerencial con gráficos de rentabilidad neta, ranking de productos más vendidos y valorización del stock.

Implementar seguridad con roles y respaldos de base de datos PostgreSQL	✅ Cumplido	Seguridad, Auditoría y Backup: Autenticación JWT, hasheo con Bcrypt, bitácora inmutable de actividad y streaming de volcados SQL.

Evolución y Maduración del Diseño (2025 → 2026)

Siguiendo las pautas metodológicas de la cátedra de Prácticas Profesionalizantes, el sistema atravesó un proceso de maduración técnica entre el anteproyecto inicial de 2do año (PP I, 2025) y la implementación final de 3er año (PP II, 2026). Las principales decisiones de diseño que evolucionaron fueron:

- De Caja Diaria Estática a Flujo Financiero Dinámico: El anteproyecto contemplaba una tabla aislada para aperturas y cierres diarios de caja. En la práctica comercial de DN Bikes, los turnos se solapan y el cobro se diversifica en medios electrónicos. Se optó por una arquitectura de balance financiero continuo que deduce ingresos reales (ventas y reparaciones) y egresos (pagos a proveedores), garantizando consistencia inmutable y eliminando el riesgo de cajas no cerradas.
- Optimización del Tablero Kanban de Taller: La propuesta original planteaba múltiples columnas estáticas. El diseño final concentró el área de trabajo en tres estados activos de alta visibilidad (*Recibida*, *En Reparación*, *Lista*) y separó las órdenes concluidas en un historial paginado con comprobante de entrega, agilizando el rendimiento de la interfaz.
- Normalización del Parque Rodado: La entidad Bicicleta se independizó formalmente del cliente mediante claves foráneas y restricciones relacionales, permitiendo conservar el historial mecánico íntegro aun ante transferencias de titularidad o servicios periódicos.
- Trazabilidad ACID de Kardex: La actualización de inventario dejó de ser un simple campo numérico editable para convertirse en una bitácora auditada mediante transacciones relacionales estrictas que garantizan que ningún producto varíe su stock sin comprobante o motivo justificado.

Decisiones técnicas y aprendizajes

El proceso de desarrollo nos permitió consolidar conocimientos y tomar decisiones de diseño que consideramos valiosas de documentar, tanto por su impacto en la calidad del producto final como por los aprendizajes que dejaron:

- Arquitectura de Escritorio Desacoplada (Electron + React + Express): Separar la interfaz visual de la API de servicios dentro de la misma aplicación de escritorio garantizó un desacoplamiento absoluto. Esto permitió probar endpoints de manera aislada y mantener una estructura de código limpia y modular.
- Base de Datos Relacional PostgreSQL: La elección de PostgreSQL 16 frente a motores SQLite o bases no relacionales fue acertada debido a su sólido soporte de transacciones ACID, integridad referencial estricta y capacidades avanzadas como la extensión pg_trgm, que hizo posible realizar búsquedas fonéticas de clientes y repuestos en milisegundos.
- Transaccionalidad en Capa de Servicios: Administrar transacciones complejas (como una venta con cobro, liquidación de caja y descuento de stock simultáneo) directamente en la capa de servicios de Express aseguró predictibilidad operativa y evitó estados inconsistentes.
- Monorepo con pnpm Workspaces: La adopción de un monorepo coordinado aceleró el desarrollo, facilitó compartir tipos de TypeScript entre componentes y optimizó el consumo de espacio en disco mediante enlaces duros.

Limitaciones y desafíos encontrados

Durante el desarrollo identificamos un conjunto de limitaciones, algunas previstas desde el inicio del proyecto en la sección de Alcance y Límites, y otras surgidas durante la implementación:

Limitaciones previstas en el alcance:

- El sistema opera exclusivamente en entorno local y de escritorio para Windows, sin conectividad con servidores en la nube ni tienda e-commerce web B2C sincronizada en esta fase.

- La facturación electrónica con web services oficiales de AFIP / ARCA no fue incorporada en esta entrega, operando mediante comprobantes internos y tickets térmicos.
- Los cobros con tarjeta o transferencias se asientan de manera declarativa sin validación directa de pasarelas bancarias por webhook online.

Desafíos técnicos durante el desarrollo:

- La orquestación del empaquetado dual de Electron Builder (compilando el backend a un bundle CJS mediante esbuild y el frontend mediante Vite) requirió una afinada configuración de scripts para evitar dependencias faltantes en tiempo de ejecución.
- El cálculo temporal exacto de la vigencia de garantías de 90 días requirió contemplar diferencias de zona horaria y cálculo preciso de fechas en PostgreSQL.

Trabajo Futuro

El sistema desarrollado sienta las bases de una plataforma de gestión integral extensible. A continuación se detallan las mejoras y ampliaciones identificadas como más relevantes, organizadas por prioridad:

Nuevas funcionalidades (alta prioridad)

- Facturación Electrónica AFIP / ARCA: Conectar el backend con el Web Service de Factura Electrónica (WSFE) para emitir facturas legales oficiales con Código de Autorización Electrónico (CAE).
- Notificaciones Automáticas por WhatsApp: Integrar una pasarela para enviar avisos automáticos al celular del cliente cuando su bicicleta pase al estado "Lista para retirar" en el taller.
- Portal Web de Consulta para Clientes: Desarrollar una página web ligera donde los clientes puedan ingresar el número de orden o escanear un código QR impreso en su comprobante para verificar el estado de su reparación en tiempo real.

Mejoras sobre funcionalidades existentes

- Soporte para Múltiples Listas de Precios: Incorporar listas de precios diferenciadas (contado efectivo con descuento, tarjeta de crédito en cuotas y precio mayorista para gremio).

- Sugerencia Inteligente de Compras: Analizar el historial de consumo de repuestos en taller para emitir sugerencias automáticas de pedidos a proveedores según la rotación por temporada.

Mejoras técnicas y de calidad

- Suite de Pruebas Automatizadas: Implementar pruebas unitarias y de integración para la API backend utilizando Jest y Supertest, y pruebas de interfaz con Playwright.
- Sincronización en Tiempo Real con WebSockets: Reemplazar el polling de peticiones por eventos Socket.IO para actualizar el tablero de taller de forma instantánea entre múltiples terminales de mecánicos.
- Respaldos Automáticos en la Nube: Configurar un servicio de copia periódica en caliente que suba los volcados SQL a almacenamiento seguro (Google Drive o Amazon S3).

Reflexión Final

El desarrollo del Sistema de Gestión "BikeSystem" representó un desafío integral que nos permitió aplicar en un escenario real y concreto los conocimientos adquiridos a lo largo de nuestra formación en la Tecnicatura Superior en Desarrollo de Software. Más allá de las decisiones técnicas y arquitectónicas implementadas, el mayor logro de este proyecto radica en haber acercado una herramienta de modernización y orden operativo a un comercio tradicional de nuestra comunidad como DN Bikes, brindándole al titular un recurso tangible para optimizar y resguardar su trabajo diario.

Haber logrado entregar un producto funcional, robusto, adaptado a su contexto y escalable, nos deja una gran satisfacción tanto académica como profesional, sentando una base sólida para nuestro futuro como desarrolladores de software.

Anexos

Entrevista a diego: